TRƯỜNG ĐẠI HỌC CẦN THƠ

TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG

BÁO CÁO BÀI TẬP NHÓM

Mã học phần: CT255

Tên học phần: Nghiệp vụ thông minh

Nhóm sinh viên thực hiện: Nhóm 2

Đào Trần Quốc Phong – B2103470

Đỗ Nguyễn Minh Sang – B2110057

Giảng viên hướng dẫn:

PGS. TS Nguyễn Thái Nghe

Cần Thơ, 03/2025

MỤC LỤC

PHẦN 1: TÌM HIỂU VỀ DATA MINING [1] 1

CHƯƠNG 1: CÂY QUYẾT ĐỊNH – DECISION TREE CLASSIFIER 1

1.1. Một thuật toán cơ bản để xây dựng cây quyết định - A Basic Algorithm to Build a Decision Tree 3

1.2. Phương pháp biểu diễn điều kiện kiểm tra thuộc tính - Methods for Expressing Attribute Test Conditions 6

1.3. Các thước đo để chọn điều kiện kiểm tra thuộc tính - Measures for selecting an attribute test condition 9

1.4. Thuật toán để khai thác cây quyết định - Algorithm for Decision Tree Induction 16

1.5. Ứng dụng ví dụ: Phát hiện Robot Web - Example Application: Web robot detection 17

1.6. Đặc điểm của cây quyết định - Characteristics of decision tree classifiers 20

1.7. Một ví dụ về xây dựng cây quyết định: bài toán play tennis [2] 27

1.7.1. Mô tả bài toán 27

1.7.2. Xây dựng cây quyết định 28

CHƯƠNG 2: MÔ HÌNH KNN – NEAREST-NEIGHBOR CLASSIFIERS 32

2.1. Algorithm 34

2.2. Đặc điểm của láng giềng gần nhất- Characteristics Nearest-Neighbor Classifiers) 35

PHẦN 2: XÂY DỰNG ỨNG DỤNG HỖ TRỢ CHẨN ĐOÁN UNG THƯ PHỔI KHÔNG TẾ BÀO NHỎ 36

CHƯƠNG 1: GIỚI THIỆU VÀ MÔ TẢ BÀI TOÁN 36

1.1. Đặt vấn đề 36

1.2. Mục tiêu đề tài 39

1.3. Mô tả chi tiết đề tài 39

1.4. Hướng tiếp cận giải quyết của đề tài 39

CHƯƠNG 2: THIẾT KẾ VÀ CÀI ĐẶT GIẢI PHÁP 41

2.1. Kiến trúc tổng quát hệ thống 41

2.2. Xây dựng các mô hình 42

2.2.1. Mô hình MobileNetV2 42

2.2.2. Mô hình InceptionV3 44

2.2.3. Mô hình VGG16 45

2.3. Giải pháp cài đặt 46

CHƯƠNG 3: KIỂM THỬ VÀ ĐÁNH GIÁ 47

3.1. Mục tiêu kiểm thử 47
3.2. Kịch bản kiểm thử các mô hình 47
3.2.1. Mô tả tập dữ liệu 47
3.2.2. Môi trường thực nghiệm 48
3.2.3. Cơ sở đánh giá 48
3.2.4. Kết quả thực nghiệm 50
3.3. Kịch bản kiểm thử giao diện web 55
3.3.1. Kiểm thử hiển thị giao diện ban đầu 55
3.3.2. Kiểm thử chức năng chọn mô hình 55
3.3.3. Kiểm thử tải lên file ảnh hợp lệ 56
3.3.4. Kiểm thử nút "Chẩn đoán" khi chưa chọn ảnh 57
3.3.5. Kiểm thử nút "Chẩn đoán" với đầy đủ thông tin 57
3.4. Kết quả kiểm thử 58
3.4.1. Kết quả kiểm thử các mô hình 58
3.4.2. Kết quả kiểm thử giao diện web 60
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 61
4.1. Kết luận 61
4.1.1. Kết quả đạt được 61
4.1.2. Hạn chế của đề tài 61
4.2. Hướng phát triển 61
TÀI LIỆU THAM KHẢO 64

DANH MỤC HÌNH

Hình 1.1: Minh họa về cây quyết định cho bài toán phân loại động vật có vú 2
Hình 1.2: Phân loại một động vật có xương sống chưa được gán nhãn 3
Hình 1.3: Thuật toán Hunt's để xây dựng cây quyết định 5
Hình 1.4: Điều kiện kiểm tra đối với thuộc tính nhị phân 6
Hình 1.5: Điều kiện kiểm tra cho các thuộc tính định danh 7
Hình 1.6: Các cách khác nhau để nhóm các giá trị thuộc tính thứ tự 8
Hình 1.7: Điều kiện kiểm tra cho các thuộc tính liên tục 8
Hình 1.8: So sánh giữa các thước đo độ nhiễu đối với bài toán phân loại nhị phân 10
Hình 1.9: Ví dụ về điều kiện kiểm tra thuộc tính ứng viên 11
Hình 1.10: Tiêu chí phân tách cho bài toán phân loại người vay tiền sử dụng chỉ số Gini 12
Hình 1.11: Phân tách các thuộc tính liên tục 14
Hình 1.12: Một thuật toán khung xương cho việc xây dựng cây quyết định 16
Hình 1.13: Dữ liệu đầu vào để phát hiện robot web 18
Hình 1.14: Mô hình cây quyết định phát hiện robot web 21
Hình 1.15: Cây quyết định cho hàm Boolean (A∧B)∨(C∧D) 22
Hình 1.16: Các phương pháp xử lý giá trị thuộc tính bị thiếu trong bộ phân loại cây quyết định 23
Hình 1.17: Dữ liệu XOR liên quan đến X và Y cùng một thuộc tính không liên quan Z .24
Hình 1.18: Ví dụ về một cây quyết định và các ranh giới quyết định của nó cho một tập dữ liệu hai chiều 26
Hình 1.19: Ví dụ về một tập dữ liệu không thể được phân vùng tối ưu bằng cây quyết định với các điều kiện kiểm tra trên từng thuộc tính riêng lẻ. Ranh giới quyết định thực sự được hiển thị bằng đường nét đứt 26
Hình 1.20: Dữ liệu của bài toán play tennis 28
Hình 1.21: Các công thức để xây dựng cây quyết định 28
Hình 1.22: Cây quyết định xây dựng được sau khi tính toán 31
Hình 1.23: Cách tiếp cận tổng quát để xây dựng mô hình phân loại 32

Hình 1.24: Các hàng xóm gần nhất thứ 1, 2 và 3 của một mẫu 33
Hình 1.25: Phân loại k-láng giềng gần nhất với số lượng lớn 34
Hình 1.26: Thuật toán k-nn 34
Hình 2.1: Biểu đồ thể hiện số lượng ca mắc ung thư trên thế giới năm 2022 36
Hình 2.2: Biểu đồ thể hiện số lượng ca tử vong do ung thư trên thế giới năm 2022 37
Hình 2.3: Biểu đồ thể hiện số lượng ca mắc ung thư tại Việt Nam năm 2022 37
Hình 2.4: Biểu đồ thể hiện số lượng ca tử vong do ung thư tại Việt Nam năm 2022 38
Hình 2.5: Kiến trúc tổng quát của hệ thống 41
Hình 2.6: Minh họa mô hình MobileNetV2 43
Hình 2.7: Minh họa mô hình InceptionV3 44
Hình 2.8: Minh họa mô hình VGG16 45
Hình 2.9: Samples của các lớp trong tập dữ liệu 48
Hình 2.10: Đường cong ROC – AUC 50
Hình 2.11: Biểu đồ Accuracy và Loss của các mô hình 51
Hình 2.12: Báo cáo đánh giá (Classification report) của các mô hình 52
Hình 2.13: Ma trận nhầm lẫn (Confusion Matrix) của các mô hình 53
Hình 2.14: Đường cong ROC và chỉ số AUC của các mô hình 54
Hình 2.15: Giao diện ban đầu 55
Hình 2.16: Giao diện khi chọn Dropdown 56
Hình 2.17: Giao diện khi chọn mô hình khác 56
Hình 2.18: Giao diện khi tải lên một ảnh 57
Hình 2.19: Giao diện khi không chọn ảnh nhưng chẩn đoán 57
Hình 2.20: Giao diện sau khi chẩn đoán thành công 58

DANH MỤC BẢNG

Bảng 1.1: Một bộ dữ liệu mẫu cho bài toán phân loại động vật có xương sống 1
Bảng 1.2: Một bộ dữ liệu mẫu cho bài toán phân loại người vay vốn.....4
Bảng 2.1: Số lượng samples ở mỗi lớp của tập dữ liệu Chest CT-Scan images 47
Bảng 2.2: So sánh độ chính xác giữa ba mô hình.....54
Bảng 2.3: Kết quả kiểm thử mô hình VGG16 59
Bảng 2.4: Kết quả kiểm thử mô hình InceptionV3.....59
Bảng 2.5: Kết quả kiểm thử giao diện web.....60

DANH MỤC TỪ CHUYÊN NGÀNH

Viết tắt	Giải thích
AI	Trí tuệ nhân tạo (Artificial Intelligence)
AUC-ROC	Diện tích dưới đường cong ROC (Area Under the Curve - Receiver Operating Characteristic)
CNN	Mạng nơ-ron tích chập (Convolutional Neural Network)
CT	Chụp cắt lớp vi tính (Computed Tomography)
MRI	Chụp cộng hưởng từ (Magnetic Resonance Imaging)
NSCLC	Ung thư phổi không tế bào nhỏ (Non-Small Cell Lung Cancer)
PET/CT	Chụp cắt lớp phát xạ positron (Positron Emission Tomography/Computed Tomography)
SCLC	Ung thư phổi tế bào nhỏ (Small Cell Lung Cancer)
SGD	Thuật toán tối ưu ngẫu nhiên (Stochastic Gradient Descent)

PHẦN 1: TÌM HIỂU VỀ DATA MINING [1]

CHƯƠNG 1: CÂY QUYẾT ĐỊNH – DECISION TREE CLASSIFIER

Để minh họa cách hoạt động của cây quyết định, hãy xem xét bài toán phân loại động vật có vú (mammals) và không phải động vật có vú (non-mammals) bằng cách sử dụng tập dữ liệu động vật có xương sống được trình bày trong **Bảng 1.1**. Giả sử các nhà khoa học phát hiện một loài mới. Làm thế nào để xác định nó là động vật có vú hay không? Một cách tiếp cận là đặt một loạt câu hỏi về đặc điểm của loài này. Câu hỏi đầu tiên có thể là: loài này là động vật máu lạnh hay máu nóng? Nếu là máu lạnh, thì chắc chắn nó không phải là động vật có vú. Nếu không, nó có thể là chim hoặc động vật có vú. Trong trường hợp này, ta cần đặt thêm một câu hỏi khác: Con cái của loài này có sinh con hay không? Những loài sinh con chắc chắn là động vật có vú, trong khi những loài không sinh con có khả năng cao là không phải động vật có vú (ngoại trừ một số loài đẻ trứng hiếm gặp như thú mỏ vịt và thú lông gai).

Bảng 1.1: Một bộ dữ liệu mẫu cho bài toán phân loại động vật có xương sống

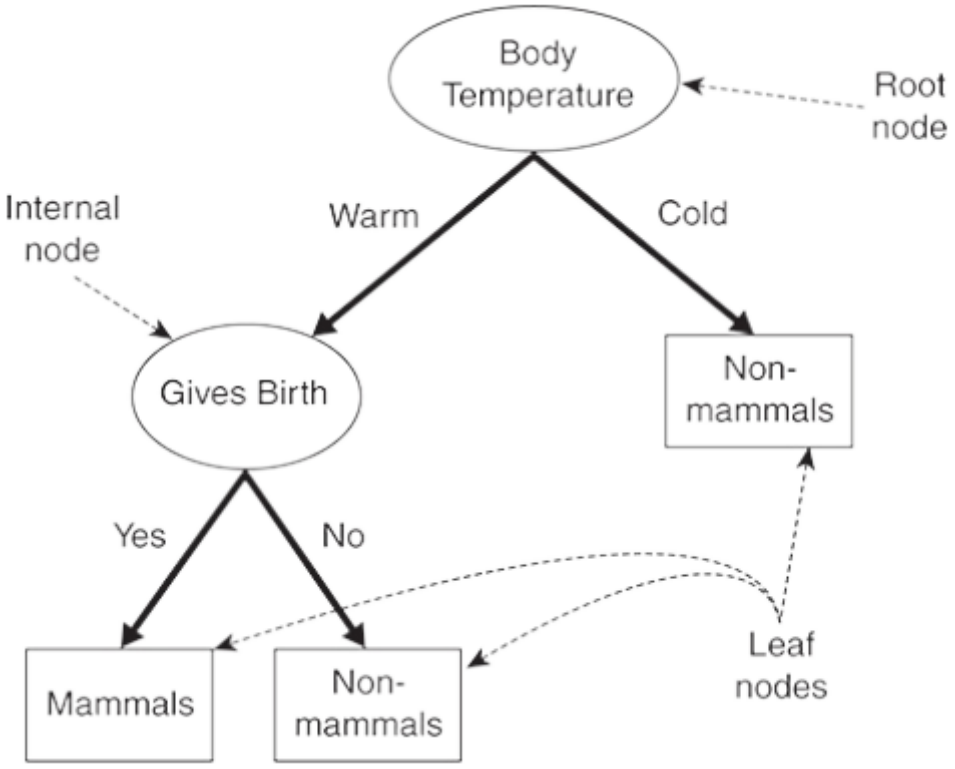
Vertebrate Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class Label
human	warm-blooded	hair	yes	no	no	yes	no	mammal
python	cold-blooded	scales	no	no	no	no	yes	reptile
salmon	cold-blooded	scales	no	yes	no	no	no	fish
whale	warm blooded	hair	yes	yes	no	no	no	mammal
frog	cold-blooded	none	no	semi	no	yes	yes	amphibian
komodo	cold-blooded	scales	no	no	no	yes	no	reptile
dragon								
bat	warm-blooded	hair	yes	no	yes	yes	yes	mammal
pigeon	warm-blooded	feathers	no	no	yes	yes	no	bird
cat	warm-blooded	fur	yes	no	no	yes	no	mammal
leopard	cold-blooded	scales	yes	yes	no	no	no	fish
shark								
turtle	cold-blooded	scales	no	semi	no	yes	no	reptile
penguin	warm-blooded	feathers	no	semi	no	yes	no	bird
porcupine	warm-blooded	quills	yes	no	no	yes	yes	mammal
eel	cold-blooded	scales	no	yes	no	no	no	fish
salamander	cold-blooded	none	no	semi	no	yes	yes	amphibian

Ví dụ trên minh họa cách chúng ta có thể giải quyết một bài toán phân loại bằng cách đặt một chuỗi câu hỏi được thiết kế cẩn thận về các thuộc tính của dữ liệu thử nghiệm. Mỗi khi nhận được một câu trả lời, ta có thể đặt tiếp một câu hỏi khác cho đến khi có thể xác định chắc chắn nhãn phân loại của đối tượng. Chuỗi câu hỏi và các câu trả lời có thể được

tổ chức thành một cấu trúc phân cấp gọi là cây quyết định. **Hình 1.1** minh họa một ví dụ về cây quyết định cho bài toán phân loại động vật có vú. Cây quyết định có ba loại nút:

- **Nút gốc (Root node):** không có liên kết đầu vào và có thể có hoặc không nhiều liên kết đầu ra.
- **Nút bên trong (Internal nodes):** mỗi nút có chính xác một liên kết đầu vào và hai hoặc nhiều liên kết đầu ra.
- **Nút lá hoặc nút kết thúc (Leaf or terminal nodes):** mỗi nút có chính xác một liên kết đầu vào và không có liên kết đầu ra.

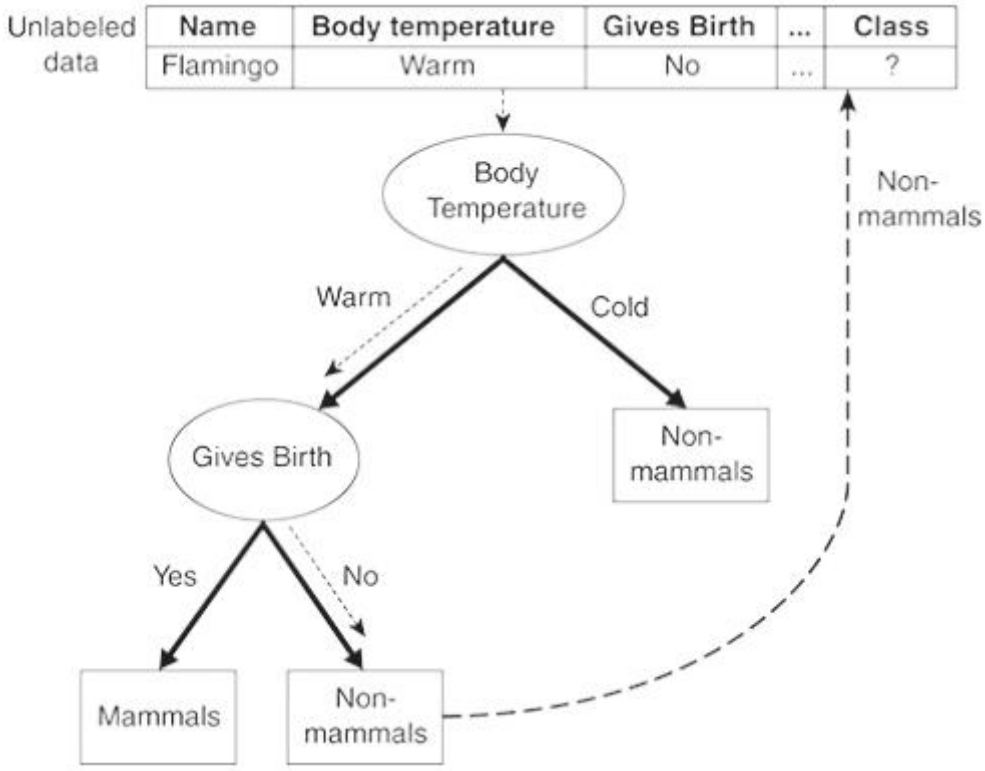
Mỗi nút lá (leaf node) trong cây quyết định đều được gán với một nhãn lớp (class label). Các nút không phải lá (nonterminal nodes), bao gồm nút gốc (root) và các nút nội bộ (internal nodes), chứa các điều kiện kiểm tra thuộc tính thường được xác định dựa trên một thuộc tính duy nhất. Mỗi kết quả khả thi của điều kiện kiểm tra thuộc tính sẽ tương ứng chính xác với một nút con của nút này. Ví dụ, nút gốc của cây quyết định được minh họa trong **Hình 1.1** sử dụng một thuộc tính để xác định điều kiện kiểm tra thuộc tính với hai kết quả khả thi là ấm (warm) và lạnh (cold), dẫn đến hai nút con tương ứng.



Hình 1.1: Minh họa về cây quyết định cho bài toán phân loại động vật có vú

Khi sử dụng cây quyết định, việc phân loại một mẫu thử khá đơn giản. Bắt đầu từ nút gốc, chúng ta áp dụng điều kiện kiểm tra thuộc tính của nó và đi theo nhánh tương ứng dựa trên kết quả của bài kiểm tra. Điều này sẽ dẫn đến một nút bên trong khác, nơi một điều kiện kiểm tra thuộc tính mới được áp dụng, hoặc đến một nút lá. Khi đến một nút lá, chúng ta gán nhãn lớp liên kết với nút đó cho mẫu thử nghiệm. Ví dụ, **Hình 1.2** minh họa cách truy vết con đường được sử dụng để dự đoán nhãn lớp của một con hồng hạc (flamingo).

Đường đi kết thúc tại một nút lá được gán nhãn là "Không phải động vật có vú (Non-mammals)".



Hình 1.2: Phân loại một động vật có xương sống chưa được gán nhãn

Phân loại một động vật có xương sống chưa được gán nhãn. Các đường nét đứt biểu thị kết quả của việc áp dụng các điều kiện kiểm tra thuộc tính khác nhau lên động vật có xương sống chưa được gán nhãn. Cuối cùng, động vật này được gán vào lớp **Không phải động vật có vú (Non-mammals).**

1.1. Một thuật toán cơ bản để xây dựng cây quyết định - A Basic Algorithm to Build a Decision Tree

Có nhiều cây quyết định có thể được tạo ra từ một tập dữ liệu cụ thể. Mặc dù một số cây tốt hơn những cây khác, nhưng việc tìm ra một cây tối ưu là rất tốn kém về mặt tính toán do kích thước không gian tìm kiếm tăng theo cấp số mũ. Các thuật toán hiệu quả đã được phát triển để tạo ra một cây quyết định có độ chính xác hợp lý. Mặc dù chưa tối ưu, nhưng vẫn có thể hoàn thành trong một khoảng thời gian hợp lý. Những thuật toán này thường sử dụng chiến lược tham lam (greedy strategy) để phát triển cây quyết định theo hướng từ trên xuống, bằng cách đưa ra một loạt quyết định tối ưu cục bộ về thuộc tính nào sẽ được sử dụng để chia tập dữ liệu huấn luyện. Một trong những phương pháp sớm nhất là **Thuật toán Hunt (Hunt's algorithm)**, nền tảng của nhiều bộ phân loại cây quyết định hiện đại, bao gồm ID3, C4.5 và CART. Phần này trình bày thuật toán Hunt và mô tả một số vấn đề thiết kế cần xem xét khi xây dựng cây quyết định.

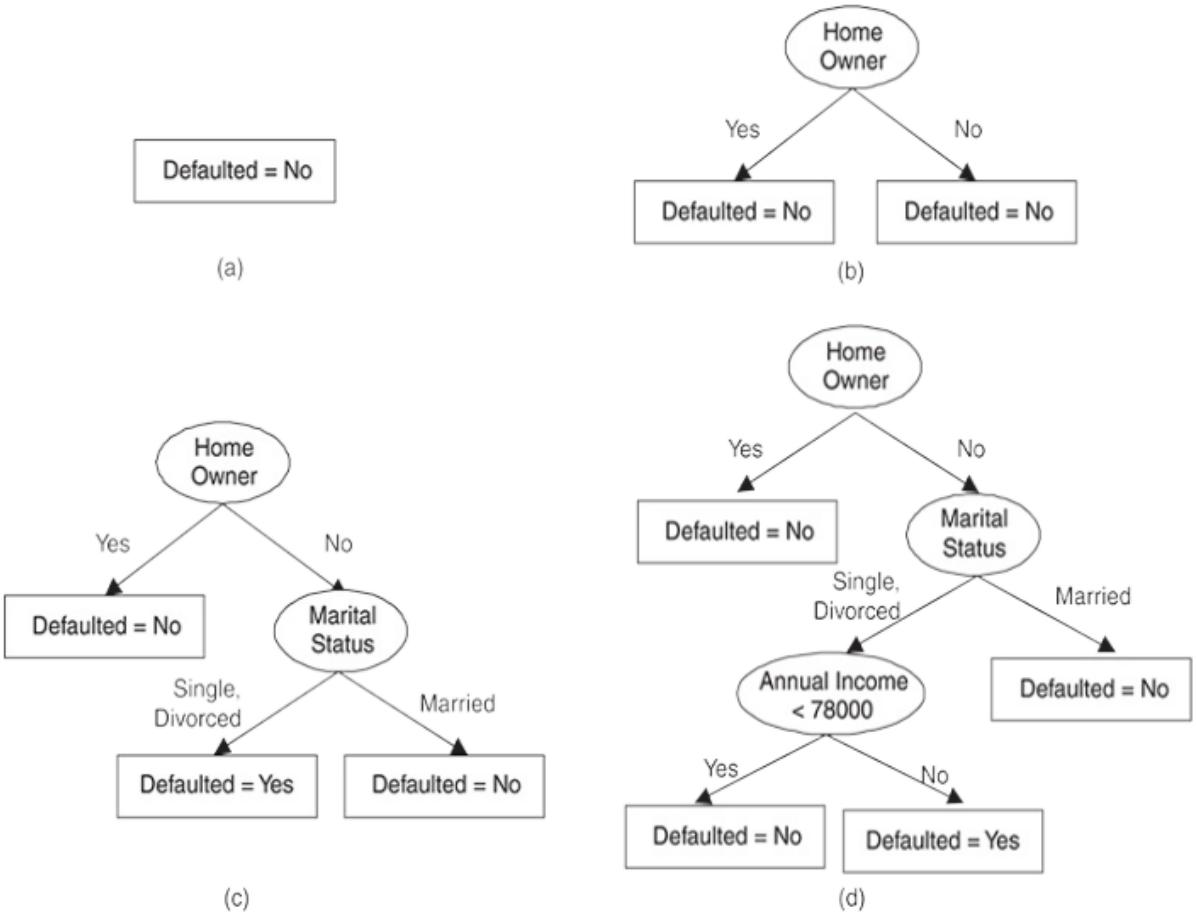
Thuật toán HUNT'S

Trong thuật toán Hunt, cây quyết định được phát triển theo cách đệ quy. Ban đầu, cây chỉ chứa một nút gốc duy nhất, liên kết với tất cả các mẫu huấn luyện. Nếu một nút chứa các mẫu từ nhiều hơn một lớp, nó sẽ được mở rộng bằng cách áp dụng điều kiện kiểm tra thuộc tính, được xác định bởi một **tiêu chí phân tách (splitting criterion)**. Một nút lá con được tạo ra cho mỗi kết quả của điều kiện kiểm tra thuộc tính, và các mẫu liên kết với nút cha sẽ được phân phối đến các nút con dựa trên kết quả kiểm tra. Bước mở rộng nút này có thể được áp dụng đệ quy cho mỗi nút con, miễn là nó vẫn còn chứa các mẫu thuộc nhiều hơn một lớp. Nếu tất cả các mẫu trong một nút lá đều có cùng một nhãn lớp, thì nút đó không được mở rộng thêm nữa. Mỗi nút lá được gán nhãn lớp xuất hiện nhiều nhất trong các mẫu huấn luyện liên kết với nó.

Để minh họa cách thuật toán hoạt động, hãy xem xét tập dữ liệu huấn luyện được trình bày trong **Bảng 1.2**, dùng để phân loại khách hàng vay tiền. Giả sử chúng ta áp dụng thuật toán Hunt để khớp với dữ liệu huấn luyện. Ban đầu, cây chỉ chứa một nút lá duy nhất, như được minh họa trong **Hình 1.3(a)**. Nút này được gán nhãn Defaulted = No, vì phần lớn khách hàng vay không bị vỡ nợ khi thanh toán khoản vay. Lỗi huấn luyện của cây này là 30%, do có ba trong số mười mẫu huấn luyện có nhãn Defaulted = Yes. Vì nút lá này chứa các mẫu huấn luyện thuộc nhiều hơn một lớp, nó có thể được mở rộng thêm.

Bảng 1.2: Một bộ dữ liệu mẫu cho bài toán phân loại người vay vốn

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes



Hình 1.3: Thuật toán Hunt's để xây dựng cây quyết định

Giả sử thuộc tính Home Owner (Chủ sở hữu nhà) được chọn để phân tách tập dữ liệu huấn luyện. Lý do chọn thuộc tính này làm điều kiện kiểm tra sẽ được thảo luận sau. Kết quả của việc phân tách nhị phân dựa trên thuộc tính Home Owner được hiển thị trong **Hình 1.3(b).** Tất cả các mẫu huấn luyện có Home Owner = Yes sẽ được chuyển đến nút con bên trái của nút gốc, và phần còn lại sẽ được chuyển đến nút con bên phải. Thuật toán Hunt sau đó tiếp tục được áp dụng đệ quy cho từng nút con. Nút con bên trái trở thành nút lá được gán nhãn Defaulted = No, vì tất cả các mẫu trong nút này đều có cùng một nhãn lớp Defaulted = No. Nút con bên phải chứa các mẫu thuộc cả hai nhãn lớp khác nhau, do đó cần tiếp tục phân tách. Các cây con thu được sau khi mở rộng đệ quy nút con bên phải được hiển thị trong Hình 1.3**(c)** và **Hình 1.3(d)**.

Thuật toán Hunt, như đã mô tả ở trên, đưa ra một số giả định đơn giản hóa, nhưng trên thực tế, những giả định này thường không đúng. Trong phần tiếp theo, chúng tôi sẽ mô tả các giả định này và đề xuất một số phương pháp để xử lý chúng.

1. Một số nút con được tạo trong thuật toán Hunt có thể rỗng nếu không có mẫu huấn luyện nào có giá trị thuộc tính cụ thể đó. Một cách để xử lý trường hợp này là khai báo mỗi nút con rỗng đó là nút lá, và gán cho nó nhãn lớp xuất hiện nhiều nhất trong số các mẫu huấn luyện liên kết với nút cha của nó.

2. Nếu tất cả các mẫu huấn luyện liên kết với một nút có giá trị thuộc tính giống nhau nhưng nhãn lớp khác nhau, thì không thể tiếp tục mở rộng nút này. Một cách để xử lý trường hợp này là khai báo nó là nút lá và gán cho nó nhãn lớp xuất hiện nhiều nhất trong số các mẫu huấn luyện liên kết với nút này.

Các vấn đề thiết kế trong việc xây dựng cây quyết định

Thuật toán Hunt là một quy trình tổng quát để phát triển cây quyết định theo chiến lược tham lam. Để triển khai thuật toán này, có hai vấn đề thiết kế quan trọng cần được giải quyết.

1. Tiêu chí phân tách (What is the splitting criterion)? Ở mỗi bước đệ quy, cần chọn một thuộc tính để chia tập dữ liệu huấn luyện liên kết với một nút thành các tập con nhỏ hơn, liên kết với các nút con của nó. Tiêu chí phân tách xác định thuộc tính nào được chọn làm điều kiện kiểm tra và cách phân phối các mẫu huấn luyện vào các nút con. Vấn đề này sẽ được thảo luận trong Mục **1.2** và Error! Reference source not found..

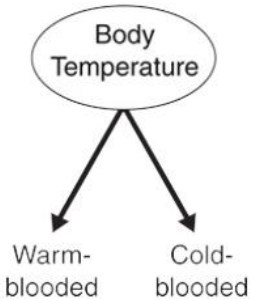
2. Tiêu chí dừng (What is the stopping criterion)? Thuật toán cơ bản chỉ dừng việc mở rộng một nút khi tất cả các mẫu huấn luyện liên kết với nút đó có cùng một nhãn lớp hoặc có giá trị thuộc tính giống hệt nhau. Mặc dù các điều kiện này là đủ, nhưng đôi khi vẫn có lý do để dừng mở rộng sớm hơn, ngay cả khi nút lá chứa các mẫu huấn luyện từ nhiều lớp khác nhau. Quá trình này được gọi là chấm dứt sớm (early termination), và điều kiện để xác định thời điểm dừng mở rộng một nút được gọi là tiêu chí dừng (stopping criterion).

1.2. Phương pháp biểu diễn điều kiện kiểm tra thuộc tính - Methods for Expressing Attribute Test Conditions

Các thuật toán xây dựng cây quyết định phải cung cấp một phương pháp để biểu diễn điều kiện kiểm tra thuộc tính và kết quả tương ứng của chúng đối với các loại thuộc tính khác nhau.

Thuộc tính nhị phân (Binary Attributes)

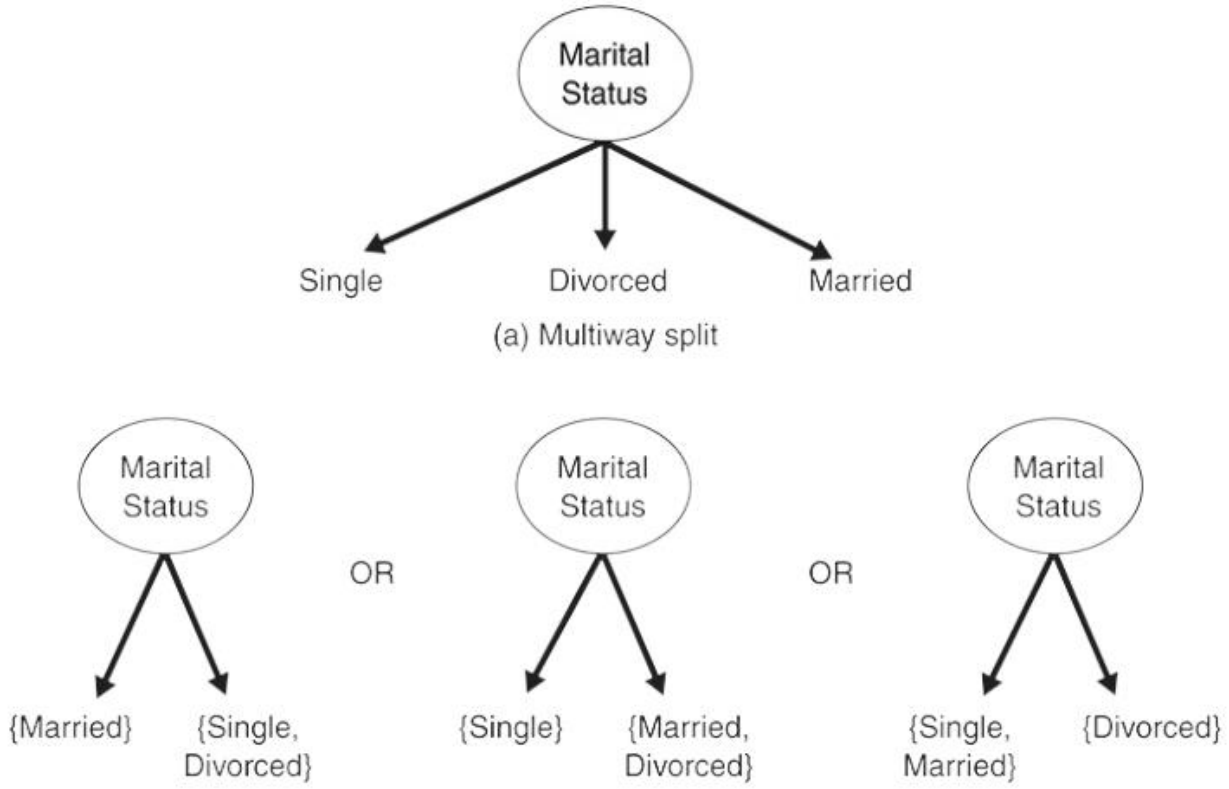
Điều kiện kiểm tra đối với một thuộc tính nhị phân tạo ra hai kết quả có thể có, như được minh họa trong **Hình 1.4**.



Hình 1.4: Điều kiện kiểm tra đối với thuộc tính nhị phân

Thuộc tính định danh (Nominal Attributes)

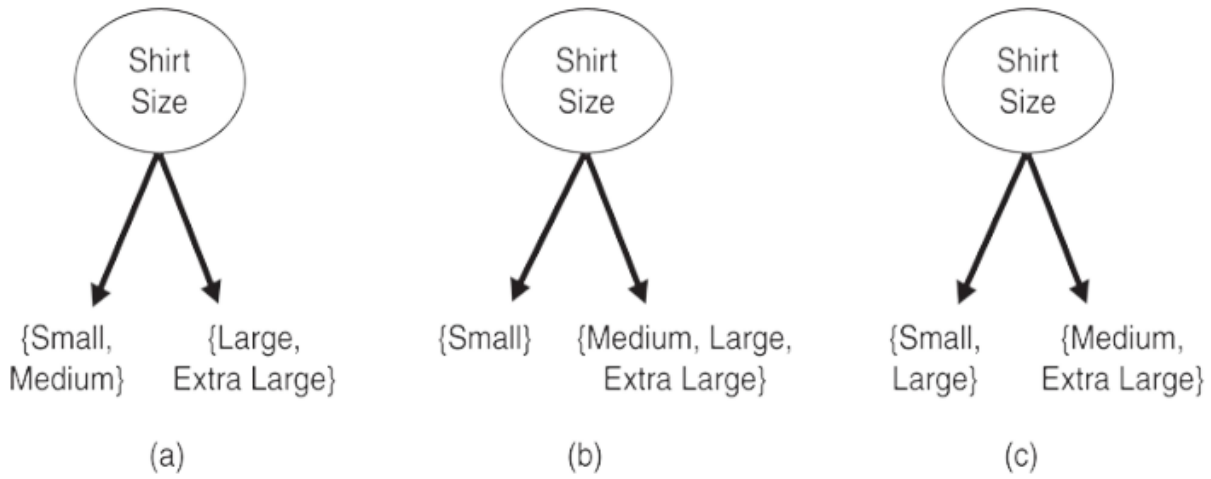
Vì một thuộc tính danh định có thể có nhiều giá trị, điều kiện kiểm tra thuộc tính của nó có thể được biểu diễn theo hai cách: phân tách nhiều nhánh (multiway split) hoặc phân tách nhị phân (binary split), như minh họa trong **Hình 1.5.** Đối với phân tách nhiều nhánh **(Hình 1.5(a))**, số lượng nhánh con phụ thuộc vào số lượng giá trị khác nhau của thuộc tính tương ứng. Ví dụ, nếu một thuộc tính như tình trạng hôn nhân có ba giá trị riêng biệt—độc thân, đã kết hôn, hoặc đã ly hôn—thì điều kiện kiểm tra của nó sẽ tạo ra một phân tách ba nhánh. Ngoài ra, cũng có thể thực hiện phân tách nhị phân bằng cách chia tất cả các giá trị mà thuộc tính danh định có thể nhận được thành hai nhóm. Ví dụ, một số thuật toán cây quyết định như CART chỉ thực hiện phân tách nhị phân bằng cách xem xét 2k - 1 - 1 cách chia nhị phân có thể có đối với một thuộc tính có k giá trị. **Hình 1.5(b)** minh họa ba cách khác nhau để nhóm các giá trị thuộc tính của tình trạng hôn nhân thành hai tập con.



Hình 1.5: Điều kiện kiểm tra cho các thuộc tính định danh

Thuộc tính thứ tự (Ordinal Attributes)

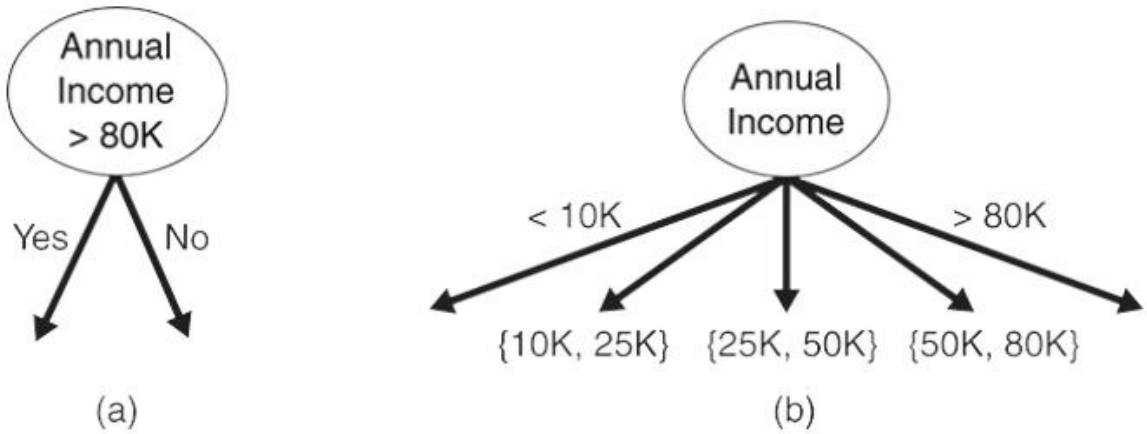
Các thuộc tính thứ tự cũng có thể được phân tách theo nhị phân hoặc đa nhánh. Các giá trị thuộc tính thứ tự có thể được nhóm lại, miễn là việc nhóm này không vi phạm tính chất thứ tự của các giá trị thuộc tính. **Hình 1.6** minh họa nhiều cách khác nhau để phân tách dữ liệu huấn luyện dựa trên thuộc tính Kích cỡ áo (Shirt Size). Các cách phân nhóm trong **Hình 1.6(a) và Hình 1.6(b)** vẫn giữ nguyên thứ tự của các giá trị thuộc tính. Ngược lại, **Hình 1.6(c)** vi phạm tính chất này vì nó nhóm Small (Nhỏ) và Large Lớn) vào cùng một nhóm, trong khi Medium (Vừa) và Extra Large (Rất lớn) lại thuộc một nhóm khác.



Hình 1.6: Các cách khác nhau để nhóm các giá trị thuộc tính thứ tự

Thuộc tính liên tục (Continuous Attributes)

Đối với các thuộc tính liên tục, điều kiện kiểm tra thuộc tính có thể được biểu diễn dưới dạng so sánh (ví dụ: A < v), tạo ra một phân tách nhị phân, hoặc dưới dạng truy vấn phạm vi của tập giá trị (với A<vi<A<vi+1, cho i = 1, ..., k), tạo ra một phân tách nhiều nhánh. Sự khác biệt giữa hai phương pháp này được minh họa trong **Hình 1.7**. Phân tách nhị phân, bất kỳ giá trị nào **v** nằm giữa giá trị thuộc tính nhỏ nhất và lớn nhất trong tập dữ liệu huấn luyện đều có thể được sử dụng để xây dựng điều kiện so sánh A < v. Tuy nhiên, chỉ cần xem xét các giá trị thuộc tính riêng biệt trong tập huấn luyện làm vị trí phân tách ứng viên là đủ. Phân tách nhiều nhánh, bất kỳ tập hợp phạm vi giá trị thuộc tính nào cũng có thể được sử dụng, miễn là chúng loại trừ lẫn nhau và bao phủ toàn bộ phạm vi giá trị thuộc tính nằm giữa giá trị nhỏ nhất và lớn nhất quan sát được trong tập huấn luyện. Một phương pháp để xây dựng phân tách nhiều nhánh là áp dụng các chiến lược rời rạc hóa. Sau khi thực hiện rời rạc hóa, một giá trị thứ tự mới sẽ được gán cho mỗi khoảng đã rời rạc hóa, và điều kiện kiểm tra thuộc tính sau đó sẽ được định nghĩa dựa trên thuộc tính thứ tự mới này.



Hình 1.7: Điều kiện kiểm tra cho các thuộc tính liên tục

1.3. Các thước đo để chọn điều kiện kiểm tra thuộc tính - Measures for selecting an attribute test condition

Có nhiều thước đo có thể được sử dụng để đánh giá mức độ hiệu quả của một điều kiện kiểm tra thuộc tính. Những thước đo này thường ưu tiên các điều kiện kiểm tra có thể phân chia tập dữ liệu huấn luyện thành các tập con "tinh khiết" hơn trong các nút con, tức là các tập con này phần lớn có cùng một nhãn lớp. Việc có nút con tinh khiết hơn rất hữu ích vì nếu một nút chứa tất cả các mẫu huấn luyện từ cùng một lớp, thì không cần mở rộng thêm nữa. Ngược lại, một nút không tinh khiết (impure node) chứa dữ liệu huấn luyện thuộc nhiều lớp khác nhau có thể cần phải mở rộng qua nhiều cấp độ, làm tăng đáng kể độ sâu của cây quyết định. Những cây quyết định có kích thước lớn không được mong muốn vì chúng dễ bị quá khớp mô hình (overfitting), tức là mô hình có thể hoạt động kém trên các dữ liệu chưa thấy trước đó. Điều này có thể làm giảm hiệu suất phân loại trên dữ liệu chưa biết. Ngoài ra, các cây quyết định lớn hơn cũng khó giải thích hơn và tốn nhiều thời gian huấn luyện và kiểm tra hơn so với các cây nhỏ hơn. Trong phần tiếp theo, chúng tôi sẽ trình bày các phương pháp khác nhau để đo lường mức độ không tinh khiết của một nút và mức độ không tinh khiết tổng thể của các nút con, cả hai yếu tố này sẽ được sử dụng để xác định điều kiện kiểm tra thuộc tính tốt nhất cho một nút.

Thước đo độ không thuần khiết cho một nút (Impurity measure for a single node)

Độ không thuần khiết (impurity) của một nút đo lường mức độ khác biệt của các nhãn lớp đối với các mẫu dữ liệu thuộc về nút đó. Dưới đây là một số thước đo có thể được sử dụng để đánh giá độ không thuần khiết của một nút t:

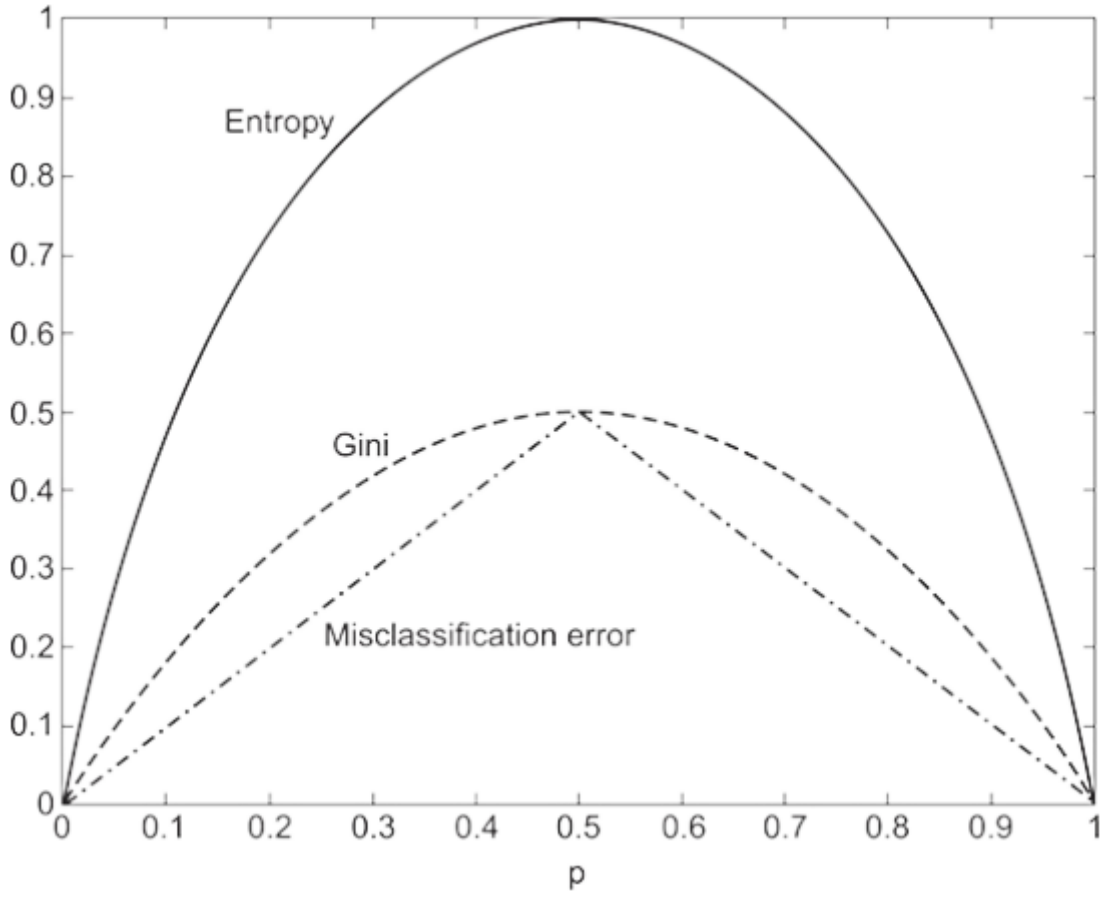
$$Entropy(t) = -\sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t) \tag{1}$$

$$Gini(t) = 1 - \sum_{i=0}^{c-1} [p(i|t)]^2 \tag{2}$$

$$Classification\ error(t) = 1 - \max_i [p(i|t)] \tag{3}$$

Trong đó, p(i|t) là tần suất tương đối của các mẫu huấn luyện thuộc lớp i tại nút ttt, c là tổng số lớp và $0\log_2 0 = 0$ trong các tính toán entropy. Cả ba thước đo này đều cho giá trị độ không thuần khiết bằng 0 nếu một nút chứa tất cả các mẫu từ một lớp duy nhất và đạt độ không thuần khiết tối đa nếu nút đó có tỷ lệ các lớp bằng nhau.

Hình 1.8 so sánh độ lớn tương đối của các thước đo độ không thuần khiết khi được áp dụng cho bài toán phân loại nhị phân. Vì chỉ có hai lớp, nên p0(t)+p1(t)=1. Trục ngang p biểu thị tỷ lệ các mẫu thuộc một trong hai lớp. Có thể quan sát thấy rằng cả ba thước đo đều đạt giá trị tối đa khi phân phối lớp là đồng đều (tức là p0(t)+p1(t)=0.5) và đạt giá trị tối thiểu khi tất cả các mẫu thuộc về một lớp duy nhất (tức là p0(t) hoặc p1(t) bằng 1). Các ví dụ sau đây minh họa cách giá trị của các thước đo độ không thuần khiết thay đổi khi phân phối lớp thay đổi.



Hình 1.8: So sánh giữa các thước đo độ nhiễu đối với bài toán phân loại nhị phân

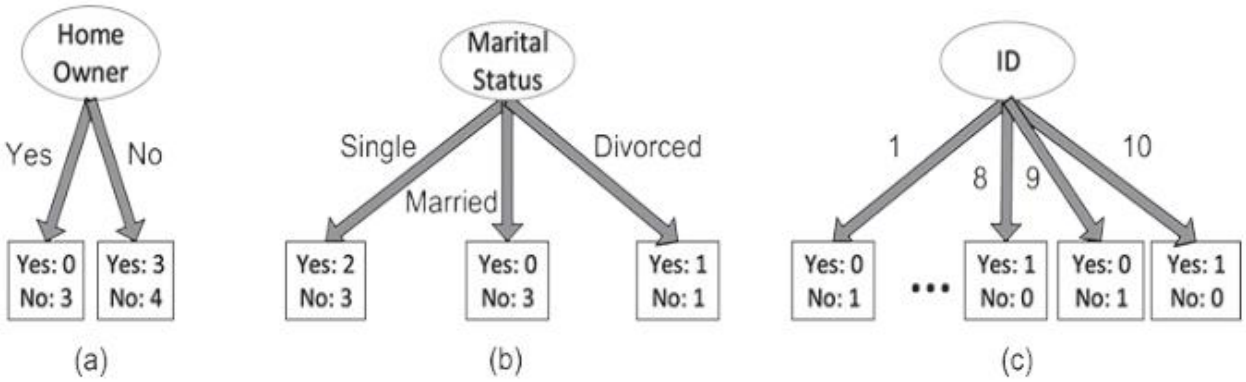
Node N1	Count	Gini=1-(0/6)2-(6/6)2=0
Class=0	0	Entropy=-(0/6) log2(0/6)-(6/6) log2(6/6)=0
Class=1	6	Error=1-max[0/6, 6/6]=0
Node N2	Count	Gini=1-(1/6)2-(5/6)2=0.278
Class=0	1	Entropy=-(1/6) log2(1/6)-(5/6) log2(5/6)=0.650
Class=1	5	Error=1-max[1/6, 5/6]=0.167
Node N3	Count	Gini=1-(3/6)2-(3/6)2=0.5
Class=0	3	Entropy=-(3/6) log2(3/6)-(3/6) log2(3/6)=1
Class=1	3	Entropy=1-max[6/6, 3/6]=0.5

Dựa trên các phép tính này, nút N1 có giá trị không thuần khiết thấp nhất, tiếp theo là N2 và N3. Ví dụ này, cùng với **Hình 1.8**, cho thấy sự nhất quán giữa các thước đo độ không thuần khiết, tức là nếu một nút N1 có entropy thấp hơn nút N2, thì chỉ số Gini và tỷ lệ lỗi của N1 cũng sẽ thấp hơn so với N2. Mặc dù có sự nhất quán này, nhưng thuộc tính được chọn làm tiêu chí phân tách bởi các thước đo độ không thuần khiết vẫn có thể khác nhau.

Độ không thuần khiết tổng thể của các nút con (Collective Impurity of Child Nodes)

Xét một điều kiện kiểm tra thuộc tính phân tách một nút chứa N mẫu huấn luyện thành k nút con, {v1,v2,…,vk}, trong đó mỗi nút con đại diện cho một phần dữ liệu thu được từ một trong k kết quả của điều kiện kiểm tra thuộc tính. Gọi N(vj) là số lượng mẫu huấn luyện liên kết với một nút con vj và I(vj) là giá trị độ không thuần khiết của nút đó. Vì mỗi mẫu huấn luyện trong nút cha đi đến nút vj với tỷ lệ N(vj)/N, nên độ không thuần khiết tổng thể của các nút con có thể được tính bằng cách lấy tổng trọng số của độ không thuần khiết các nút con, theo công thức sau:

$$I(children) = \sum_{j=1}^{k} \frac{N(v_j)}{N} I(v_j) \tag{4}$$

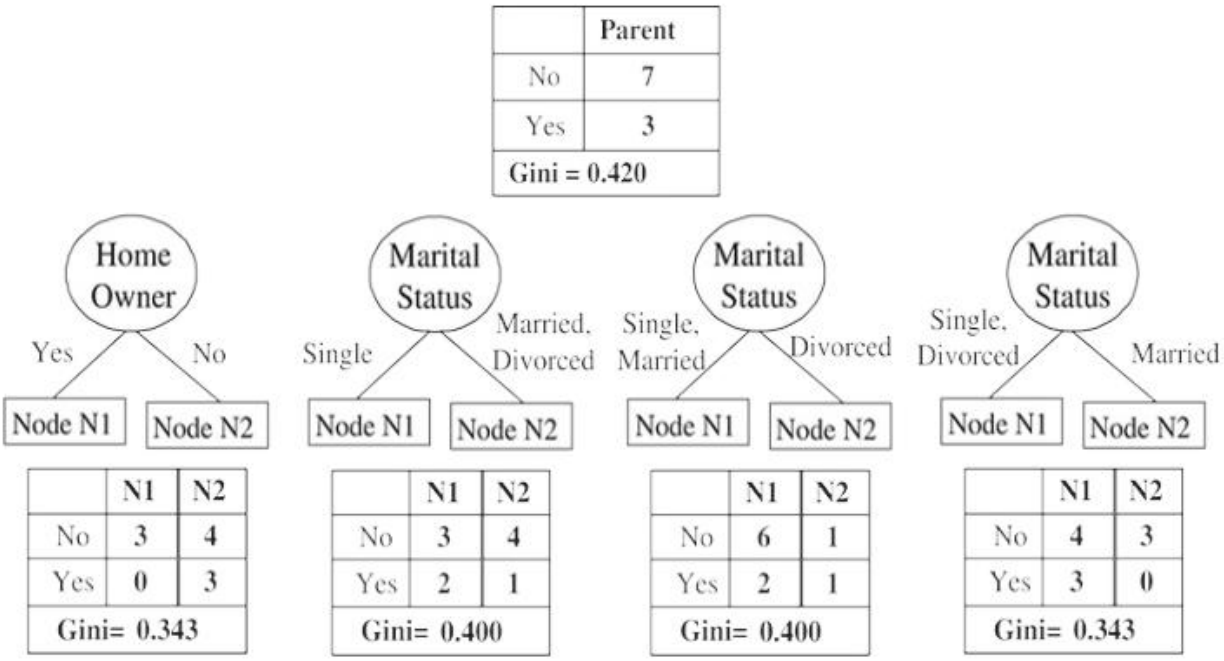


Hình 1.9: Ví dụ về điều kiện kiểm tra thuộc tính ứng viên

Xác định điều kiện kiểm tra thuộc tính tốt nhất (Identifying the best attribute test condition)

Để đánh giá mức độ hiệu quả của một điều kiện kiểm tra thuộc tính, chúng ta cần so sánh độ không thuần khiết của nút cha (trước khi phân tách) với độ không thuần khiết có trọng số của các nút con (sau khi phân tách). Sự khác biệt giữa hai giá trị này càng lớn, thì điều kiện kiểm tra càng tốt. Sự khác biệt này, Δ, còn được gọi là độ tăng (gain) trong độ tinh khiết của một điều kiện kiểm tra thuộc tính, được xác định theo công thức sau:

$$\Delta = I(parent) - I(child) \tag{5}$$



Hình 1.10: Tiêu chí phân tách cho bài toán phân loại người vay tiền sử dụng chỉ số Gini

Trong đó, I(parent) là độ không thuần khiết của một nút trước khi phân tách, và I(children) là độ không thuần khiết có trọng số sau khi phân tách. Có thể chứng minh rằng độ tăng (gain) luôn không âm, vì I(parent) ≥ I(children) đối với bất kỳ thước đo hợp lý nào được trình bày ở trên. Độ tăng càng cao, thì các lớp trong nút con càng tinh khiết so với nút cha. Tiêu chí phân tách trong thuật toán học cây quyết định chọn điều kiện kiểm tra thuộc tính có độ tăng cao nhất. Tối đa hóa độ tăng tại một nút tương đương với tối thiểu hóa độ không thuần khiết có trọng số của các nút con, vì I(parent) là không đổi đối với tất cả các điều kiện kiểm tra thuộc tính có thể có. Cuối cùng, khi entropy được sử dụng làm thước đo độ không thuần khiết, sự khác biệt trong entropy thường được gọi là "độ tăng thông tin" **(information gain),** Δinfo**.**

Trong phần tiếp theo, sẽ trình bày các phương pháp minh họa để xác định điều kiện kiểm tra thuộc tính tốt nhất, dựa trên thuộc tính định tính hoặc định lượng.

Phân tách các thuộc tính định tính (Splitting of qualitative attributes)

Hãy xem xét hai điều kiện phân tách ứng viên đầu tiên được minh họa trong **Hình 1.9**, liên quan đến các thuộc tính định tính là Chủ sở hữu nhà (Home Owner) và Tình trạng hôn nhân (Marital Status). Phân phối lớp ban đầu tại nút cha là (0.3, 0.7), vì trong dữ liệu huấn luyện có 3 mẫu thuộc lớp "Yes" và 7 mẫu thuộc lớp "No". Do đó,

I(parent) = -310log2 (310) – 710log2 (710) = 0.881

Mức độ tăng thông tin (information gain) cho Chủ sở hữu nhà và Tình trạng hôn nhân lần lượt được tính như sau:

Δinfo(Home Owner)=0.881−0.690=0.191Δinfo(Marital Status)=0.881−0.686=0.195

Vì tình trạng hôn nhân có độ tăng thông tin cao hơn, do có entropy có trọng số thấp hơn, nên nó sẽ được chọn làm thuộc tính để phân tách.

Phân tách nhị phân các thuộc tính định tính (Binary Splitting of Qualitative Attributes)

Hãy xem xét việc xây dựng một cây quyết định chỉ sử dụng phân tách nhị phân và chỉ số Gini làm thước đo độ không thuần khiết. **Hình 1.10** minh họa bốn tiêu chí phân tách ứng viên cho các thuộc tính Chủ sở hữu nhà (Home Owner) và Tình trạng hôn nhân (Marital Status). Vì trong tập huấn luyện có 3 khách hàng vay bị vỡ nợ và 7 khách hàng trả nợ đầy đủ (xem bảng trong **Hình 1.10**), chỉ số Gini của nút cha trước khi phân tách được tính là:

$1-(310)2-(710)2=0.420.1$

Nếu Chủ sở hữu nhà (Home Owner) được chọn làm thuộc tính phân tách, chỉ số Gini của nút con N1 và N2 lần lượt là 0 và 0.490. Chỉ số Gini trung bình có trọng số của các nút con được tính là:

$(310\times0)+(710\times0.490)=0.343$

Trong đó, các trọng số thể hiện tỷ lệ mẫu huấn luyện được phân vào mỗi nút con. Độ tăng Gini khi chọn Chủ sở hữu nhà làm thuộc tính phân tách là $0.420-0.343=0.077$. Tương tự, chúng ta có thể thực hiện phân tách nhị phân trên thuộc tính Tình trạng hôn nhân (Marital Status). Tuy nhiên, vì Tình trạng hôn nhân là một thuộc tính danh định có ba giá trị khác nhau, nên có ba cách nhóm các giá trị thuộc tính để tạo thành một phân tách nhị phân. Chỉ số Gini trung bình có trọng số của mỗi phân tách nhị phân ứng viên được minh họa trong **Hình 1.10.** Dựa trên các kết quả tính toán, Chủ sở hữu nhà (Home Owner) và phân tách nhị phân cuối cùng sử dụng Tình trạng hôn nhân (Marital Status) là hai ứng viên tốt nhất, vì cả hai đều tạo ra chỉ số Gini trung bình có trọng số thấp nhất. Phân tách nhị phân cũng có thể được sử dụng cho thuộc tính thứ tự (ordinal attributes), miễn là cách nhóm các giá trị thuộc tính tuân theo tính chất thứ tự của chúng.

Phân tách nhị phân các thuộc tính định lượng (Binary splitting of quantitative attributes)

Hãy xem xét bài toán xác định điểm phân tách nhị phân tốt nhất cho thuộc tính Thu nhập hàng năm (Annual Income $\leq \tau$) trong bài toán phân loại phê duyệt khoản vay đã đề cập trước đó. Như đã thảo luận trước đây, mặc dù τ có thể nhận bất kỳ giá trị nào trong khoảng từ giá trị nhỏ nhất đến giá trị lớn nhất của thu nhập hàng năm trong tập huấn luyện, nhưng chỉ cần xem xét các giá trị thu nhập hàng năm xuất hiện trong tập huấn luyện làm ứng viên phân tách là đủ. Đối với mỗi ứng viên τ, tập huấn luyện được quét một lần để đếm số lượng khách vay có thu nhập $\leq \tau$ và $> \tau$, cùng với tỷ lệ lớp của họ. Sau đó, ta có thể tính chỉ số Gini tại mỗi vị trí phân tách ứng viên và chọn giá trị τ có chỉ số Gini nhỏ nhất. Việc tính chỉ số Gini tại mỗi vị trí phân tách yêu cầu O(N) phép toán, trong đó N là số

lượng mẫu huấn luyện. Vì có tối đa N ứng viên phân tách, nên độ phức tạp tổng thể của phương pháp vét cạn này là $O(N^2)$. Tuy nhiên, có thể giảm độ phức tạp của bài toán này xuống O(N log N) bằng một phương pháp khác được mô tả như sau (xem minh họa trong **Hình 1.11**). Trước tiên, sắp xếp các mẫu huấn luyện theo thu nhập hàng năm, một thao tác một lần duy nhất có độ phức tạp O(N log N). Các vị trí phân tách ứng viên được lấy là trung điểm giữa hai giá trị thu nhập liên tiếp trong danh sách đã sắp xếp: $55,000, $65,000, $72,500,… Với ứng viên phân tách đầu tiên, không có mẫu nào có thu nhập ≤ $55,000, do đó chỉ số Gini của nút con với Annual Income < $55,000 bằng 0. Ngược lại, có 3 mẫu lớp "Yes" và 7 mẫu lớp "No" với thu nhập > $55,000. Chỉ số Gini của nút này là 0.420. Chỉ số Gini trung bình có trọng số cho vị trí phân tách đầu tiên (τ = $55,000) là: 0×0+1×0.420=0.420.0

	Class	No	No	No	Yes	Yes	Yes	No	No	No	No
		Annual Income (in '000s)									
Sorted Values →		60	70	75	85	90	95	100	120	125	220

Split Positions →	55		65		72.5		80		87.5		92.5		97.5		110		122.5		172.5		230	
	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini	0.420		0.400		0.375		0.343		0.417		0.400		*0.300*		0.343		0.375		0.400		0.420	

Hình 1.11: Phân tách các thuộc tính liên tục

Đối với ứng viên phân tách tiếp theo, khi τ = $65,000, phân phối lớp của các nút con có thể được cập nhật một cách đơn giản từ phân phối của ứng viên trước đó. Điều này là do khi τ tăng từ $55,000 lên $65,000, chỉ có một mẫu huấn luyện bị ảnh hưởng bởi thay đổi này. Bằng cách kiểm tra nhãn lớp của mẫu bị ảnh hưởng, có thể xác định được phân phối lớp mới. Ví dụ, khi τ tăng lên $65,000, có chỉ một khách vay trong tập huấn luyện có thu nhập $60,000 bị ảnh hưởng bởi thay đổi này. Vì nhãn lớp của khách vay này là "No", nên số lượng mẫu thuộc lớp "No" tăng từ 0 lên 1 (đối với Annual Income ≤ $65,000) và giảm từ 7 xuống 6 (đối với Annual Income > $65,000), như được minh họa trong **Hình 1.11**. Phân phối của lớp "Yes" không thay đổi. Chỉ số Gini cập nhật cho vị trí phân tách ứng viên này là 0.400.

Quy trình này được lặp lại cho đến khi chỉ số Gini của tất cả các ứng viên phân tách được tính toán. Vị trí phân tách tốt nhất tương ứng với vị trí có chỉ số Gini thấp nhất, xảy ra tại τ = $97,500. Vì chỉ số Gini tại mỗi vị trí phân tách có thể được tính với độ phức tạp O(1), nên tìm vị trí phân tách tốt nhất có độ phức tạp O(N) nếu tất cả các giá trị được lưu trữ theo thứ tự, với một thao tác sắp xếp một lần duy nhất có độ phức tạp O(N log N). Tổng độ phức tạp của phương pháp này là O(N log N), nhỏ hơn đáng kể so với $O(N^2)$ của phương pháp vét cạn (brute-force). Lượng tính toán có thể được giảm hơn nữa bằng cách chỉ xem xét các vị trí phân tách nằm giữa hai mẫu liên tiếp có nhãn lớp khác nhau. Ví dụ, không

cần xem xét các vị trí phân tách giữa $60,000 và $75,000, vì cả ba mẫu có thu nhập trong khoảng này ($60,000, $70,000 và $75,000) đều có cùng nhãn lớp. Việc chọn một vị trí phân tách trong khoảng này chỉ làm tăng độ không thuần khiết, thay vì giảm nó. Do đó, các ứng viên phân tách tại τ = $65,000 và τ = $72,500 có thể bị loại bỏ. Tương tự, các vị trí phân tách tại $87,500, $92,500, $110,000, $122,500 và $172,500 cũng có thể bị bỏ qua, vì chúng nằm giữa hai mẫu liên tiếp có cùng nhãn lớp. Chiến lược này giúp giảm số lượng vị trí phân tách cần xem xét từ 9 xuống còn 2 (không tính hai trường hợp biên τ = $55,000 và τ = $230,000).

Tỷ lệ thu được (Gain Ratio)

Một hạn chế tiềm ẩn của các thước đo độ không thuần khiết như entropy và chỉ số Gini là chúng có xu hướng ưu tiên các thuộc tính định tính (qualitative attributes) có nhiều giá trị khác nhau. **Hình 1.9** minh họa ba ứng viên phân tách cho tập dữ liệu được trình bày trong **Bảng 1.2**. Như đã đề cập trước đó, thuộc tính Tình trạng hôn nhân (Marital Status) là lựa chọn tốt hơn so với Chủ sở hữu nhà (Home Owner), vì nó tạo ra độ tăng thông tin (information gain) lớn hơn. Tuy nhiên, nếu so sánh với Mã khách hàng (Customer ID), thì Customer ID tạo ra phân vùng thuần khiết nhất với độ tăng thông tin tối đa, vì entropy và chỉ số Gini có trọng số của nó bằng 0 đối với các nút con. Tuy nhiên, Customer ID không phải là một thuộc tính tốt để phân tách, vì nó có giá trị duy nhất cho mỗi mẫu. Mặc dù một điều kiện kiểm tra sử dụng Customer ID có thể phân loại chính xác mọi mẫu trong tập huấn luyện, nhưng chúng ta không thể sử dụng nó để phân loại các mẫu mới có giá trị Customer ID chưa từng thấy trước đó trong quá trình huấn luyện. Ví dụ này cho thấy rằng độ không thuần khiết thấp không phải là điều kiện đủ để tìm ra một thuộc tính kiểm tra tốt cho một nút. Việc có quá nhiều nút con có thể khiến cây quyết định trở nên phức tạp hơn và dễ bị quá khớp hơn. Do đó, số lượng nút con do một thuộc tính phân tách tạo ra cũng cần được cân nhắc khi quyết định thuộc tính kiểm tra tốt nhất.

Có hai cách để khắc phục vấn đề này. Một cách là chỉ tạo ra các cây quyết định nhị phân, do đó tránh được khó khăn trong việc xử lý các thuộc tính có số lượng phân vùng khác nhau. Chiến lược này được áp dụng bởi các bộ phân loại cây quyết định như CART. Cách khác là sửa đổi tiêu chí phân tách để tính đến số lượng phân vùng được tạo ra bởi thuộc tính. Ví dụ, trong thuật toán cây quyết định C4.5, một thước đo được gọi là **tỷ lệ thu nhận (gain ratio)** được sử dụng để bù đắp cho các thuộc tính tạo ra số lượng lớn các nút con. Thước đo này được tính như sau:

$$Gain\ ratio = \frac{\Delta_{info}}{Split\ Info} = \frac{Entropy(Parent) - \sum_{i=1}^{k} \frac{N(v_i)}{N} Entropy(v_i)}{\sum_{i=1}^{k} \frac{N(v_i)}{N} \log_2 \frac{N(v_i)}{N}} \qquad (6)$$

Trong đó, $N(v_i)$ là số lượng mẫu dữ liệu được gán cho nút v_i và k là tổng số phân tách (splits). Thông tin phân tách (Split Information) đo lường entropy của việc phân

tách một nút thành các nút con và đánh giá xem phân tách này có tạo ra số lượng lớn các nút con có kích thước tương đương hay không.

1.4. Thuật toán để khai thác cây quyết định - Algorithm for Decision Tree Induction

Thuật toán ở **Hình 1.12** trình bày mã giả (pseudocode) cho thuật toán xây dựng cây quyết định. Đầu vào của thuật toán này là tập hợp các mẫu huấn luyện E cùng với tập thuộc tính F. Thuật toán hoạt động bằng cách đệ quy chọn thuộc tính tốt nhất để phân tách dữ liệu (Bước 7) và mở rộng các nút của cây (Bước 11 và 12) cho đến khi điều kiện dừng được thỏa mãn (Bước 1). Chi tiết của thuật toán này được mô tả dưới đây:

1. Hàm createNode() mở rộng cây quyết định bằng cách tạo một nút mới. Một nút trong cây quyết định có thể có điều kiện kiểm tra (test condition), được ký hiệu là node.test_cond, hoặc có nhãn lớp (class label), được ký hiệu là node.label.

2. Hàm find_best_split() xác định điều kiện kiểm tra thuộc tính để phân tách các mẫu huấn luyện liên kết với một nút. Thuộc tính được chọn để phân tách phụ thuộc vào thước đo độ không thuần khiết được sử dụng. Các thước đo phổ biến bao gồm entropy và chỉ số Gini.

3. Hàm Classify() xác định nhãn lớp sẽ được gán cho một nút lá. Đối với mỗi nút lá t, gọi p(i|t) là tỷ lệ mẫu huấn luyện thuộc lớp i trong nút t. Nhãn lớp được gán cho nút lá thường là lớp xuất hiện nhiều nhất trong các mẫu huấn luyện liên kết với nút đó.

Algorithm 4.1 A skeleton decision tree induction algorithm.

```
TreeGrowth (E, F)
 1: if stopping_cond(E,F) = true then
 2:    leaf = createNode().
 3:    leaf.label = Classify(E).
 4:    return leaf.
 5: else
 6:    root = createNode().
 7:    root.test_cond = find_best_split(E, F).
 8:    let V = {v|v is a possible outcome of root.test_cond }.
 9:    for each v ∈ V do
10:       E_v = {e | root.test_cond(e) = v and e ∈ E}.
11:       child = TreeGrowth(E_v, F).
12:       add child as descendent of root and label the edge (root → child) as v.
13:    end for
14: end if
15: return root.
```

Hình 1.12: Một thuật toán khung xương cho việc xây dựng cây quyết định

$$leaf.label = argmax_i p(i|t) \quad (7)$$

Trong đó, toán tử argmax trả về lớp i mà làm cực đại giá trị p(i|t). Bên cạnh việc cung cấp thông tin cần thiết để xác định nhãn lớp của một nút lá, nó cũng có thể được sử dụng như một ước lượng sơ bộ về xác suất rằng một mẫu được gán cho nút lá t thuộc về lớp i.

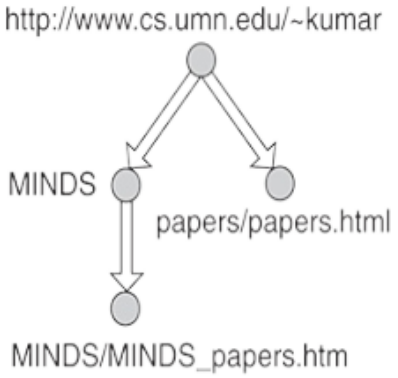
4. Hàm stopping_cond(): được sử dụng để kết thúc quá trình phát triển cây quyết định bằng cách kiểm tra xem tất cả các mẫu trong nút có cùng nhãn lớp hoặc giá trị thuộc tính không. Vì các bộ phân loại cây quyết định sử dụng phương pháp phân vùng đệ quy từ trên xuống để xây dựng mô hình, số lượng mẫu huấn luyện liên kết với một nút giảm dần khi độ sâu của cây tăng lên. Kết quả là, một nút lá có thể chứa quá ít mẫu huấn luyện, khiến việc quyết định nhãn lớp trở nên không có ý nghĩa về mặt thống kê. Điều này được gọi là vấn đề **phân mảnh dữ liệu (data fragmentation)**. Một cách để tránh vấn đề này là không cho phép phân tách nút nếu số lượng mẫu liên kết với nút giảm xuống dưới một ngưỡng nhất định. Một phương pháp có hệ thống hơn để kiểm soát kích thước của cây quyết định **(**số lượng nút lá**)**.

1.5. Ứng dụng ví dụ: Phát hiện Robot Web - Example Application: Web robot detection

Hãy xem xét nhiệm vụ phân biệt các mô hình truy cập của robot web với mô hình truy cập do người dùng tạo ra. Một robot web (còn được gọi là trình thu thập web - web crawler) là một chương trình phần mềm tự động truy xuất tệp từ một hoặc nhiều trang web bằng cách theo dõi các liên kết (hyperlinks) được trích xuất từ một tập hợp URL ban đầu. Các chương trình này được triển khai cho nhiều mục đích khác nhau, từ thu thập trang web cho công cụ tìm kiếm đến các hoạt động độc hại hơn, chẳng hạn như spam hoặc gian lận nhấp chuột (click frauds) trong quảng cáo trực tuyến.

Session	IP Address	Timestamp	Request Method	Requested Web Page	Protocol	Status	Number of Bytes	Referrer	User Agent
1	160.11.11.11	08/Aug/2004 10:15:21	GET	http://www.cs.umn.edu/~kumar	HTTP/1.1	200	6424		Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:15:34	GET	http://www.cs.umn.edu/~kumar/MINDS	HTTP/1.1	200	41378	http://www.cs.umn.edu/~kumar	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:15:41	GET	http://www.cs.umn.edu/~kumar/MINDS/MINDS_papers.htm	HTTP/1.1	200	1018516	http://www.cs.umn.edu/~kumar/MINDS	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:16:11	GET	http://www.cs.umn.edu/~kumar/papers/papers.html	HTTP/1.1	200	7463	http://www.cs.umn.edu/~kumar	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
2	35.9.2.2	08/Aug/2004 10:16:15	GET	http://www.cs.umn.edu/~steinbac	HTTP/1.0	200	3149		Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.7) Gecko/20040616

(a) Example of a Web server log.



(b) Graph of a Web session.

Attribute Name	Description
totalPages	Total number of pages retrieved in a Web session
ImagePages	Total number of image pages retrieved in a Web session
TotalTime	Total amount of time spent by Web site visitor
RepeatedAccess	The same page requested more than once in a Web session
ErrorRequest	Errors in requesting for Web pages
GET	Percentage of requests made using GET method
POST	Percentage of requests made using POST method
HEAD	Percentage of requests made using HEAD method
Breadth	Breadth of Web traversal
Depth	Depth of Web traversal
MultiIP	Session with multiple IP addresses
MultiAgent	Session with multiple user agents

(c) Derived attributes for Web robot detection.

Hình 1.13: Dữ liệu đầu vào để phát hiện robot web

Vấn đề phát hiện robot web có thể được xem như một bài toán phân loại nhị phân. Dữ liệu đầu vào cho nhiệm vụ phân loại là nhật ký máy chủ web (web server log), một ví dụ về điều này được thể hiện trong **Hình 1.13(a)**. Mỗi dòng trong tệp nhật ký tương ứng với một yêu cầu do một khách hàng (có thể là người dùng hoặc robot web) gửi đến máy chủ web. Các trường được ghi lại trong nhật ký web bao gồm địa chỉ IP của khách hàng, dấu thời gian của yêu cầu, URL của tệp được yêu cầu, kích thước của tệp và user agent, đây là một trường chứa thông tin nhận dạng về khách hàng.

Đối với người dùng thực, trường user agent chỉ định loại trình duyệt web hoặc thiết bị di động được sử dụng để tải tệp, trong khi đối với robot web, nó về mặt kỹ thuật sẽ chứa tên của chương trình thu thập dữ liệu. Tuy nhiên, robot web có thể che giấu danh tính thật của chúng bằng cách đặt trường user agent giống với các trình duyệt phổ biến. Do đó, user agent không phải là một trường đáng tin cậy để phát hiện robot web.

Bước đầu tiên trong việc xây dựng mô hình phân loại là xác định chính xác một tập dữ liệu và các thuộc tính liên quan. Một cách tiếp cận đơn giản là coi mỗi dòng trong nhật ký là một mẫu dữ liệu và sử dụng các trường tương ứng trong tệp nhật ký làm tập thuộc

tính của nó.Tuy nhiên, cách tiếp cận này không phù hợp vì một số lý do. Thứ nhất, nhiều thuộc tính có giá trị danh định và có phạm vi giá trị rất lớn. Ví dụ, số lượng địa chỉ IP khách hàng, URL và referrer duy nhất trong một tệp nhật ký có thể rất lớn. Những thuộc tính này không thích hợp để xây dựng cây quyết định vì thông tin phân tách của chúng quá cao (xem **Công thức (6)**). Ngoài ra, có thể không thể phân loại các mẫu kiểm tra chứa địa chỉ IP, URL hoặc referrer không có trong dữ liệu huấn luyện. Cuối cùng, nếu coi mỗi dòng trong nhật ký là một mẫu dữ liệu riêng biệt, ta sẽ bỏ qua trình tự của các trang web mà khách hàng truy cập. Đây là một thông tin quan trọng có thể giúp phân biệt việc truy cập của robot web với truy cập của người dùng thực.

Một phương án tốt hơn là xem xét mỗi phiên truy cập web như một mẫu dữ liệu. Một phiên truy cập web là một chuỗi các yêu cầu được thực hiện bởi một khách hàng trong một lần truy cập vào trang web. Mỗi phiên truy cập có thể được mô hình hóa như một đồ thị có hướng, trong đó các nút tương ứng với các trang web và các cạnh tương ứng với các liên kết siêu văn bản (hyperlinks) kết nối giữa các trang. **Hình 1.13(b)** hiển thị một biểu diễn đồ họa của phiên truy cập web đầu tiên được ghi lại trong tệp nhật ký. Mỗi phiên truy cập web có thể được đặc trưng bằng các thuộc tính ý nghĩa của đồ thị chứa thông tin giúp phân biệt các phiên khác nhau. **Hình 1.13(c)** cho thấy một số thuộc tính được trích xuất từ đồ thị, bao gồm độ sâu và độ rộng của cây tương ứng với điểm bắt đầu của phiên truy cập vào trang web. Ví dụ, độ sâu và độ rộng của cây trong **Hình 1.13(b)** đều bằng hai.

Các thuộc tính được trích xuất trong **Hình 1.13(c)** mang tính thông tin hơn so với các thuộc tính gốc trong tệp nhật ký vì chúng mô tả hành vi của khách hàng trên trang web. Sử dụng cách tiếp cận này, một tập dữ liệu gồm 2916 phiên truy cập đã được tạo, trong đó số lượng phiên do robot web (lớp 1) và người dùng thực (lớp 0) là cân bằng. 10% dữ liệu được dùng để huấn luyện, trong khi 90% còn lại được dùng để kiểm tra. Cây quyết định thu được được hiển thị trong **Hình 1.14,** với tỷ lệ lỗi 3.8% trên tập huấn luyện và 5.3% trên tập kiểm tra. Ngoài tỷ lệ lỗi thấp, cây quyết định cũng tiết lộ một số đặc tính thú vị giúp phân biệt robot web với người dùng thực:

1. Các phiên truy cập của robot web có xu hướng rộng nhưng nông, trong khi các phiên truy cập của người dùng thực thường tập trung hơn (hẹp nhưng sâu).

2. Robot web hiếm khi truy xuất các trang hình ảnh liên kết với một trang web.

3. Các phiên truy cập của robot web thường kéo dài và chứa số lượng lớn các trang được yêu cầu.

4. Robot web có xu hướng gửi yêu cầu lặp lại đối với cùng một trang web nhiều hơn so với người dùng thực, do các trang web mà người dùng thực đã truy cập thường được lưu trong bộ nhớ cache của trình duyệt.

1.6. Đặc điểm của cây quyết định - Characteristics of decision tree classifiers

Dưới đây là tóm tắt các đặc điểm quan trọng của các thuật toán xây dựng cây quyết định.

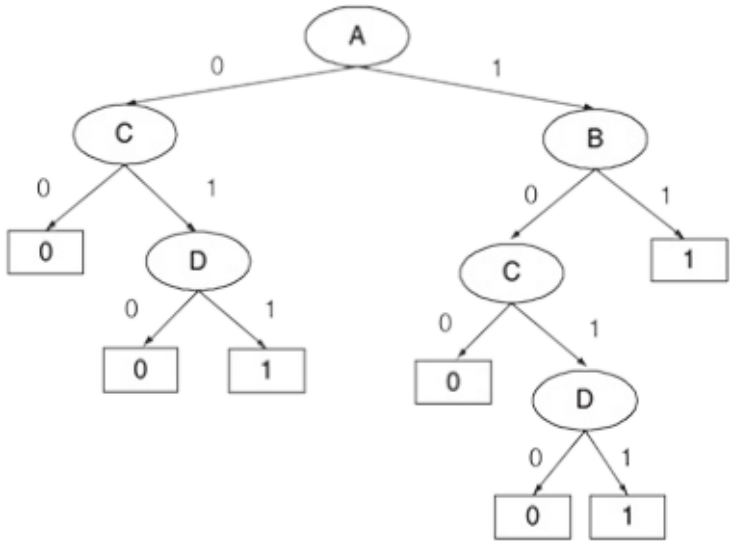
1. Khả năng áp dụng - Applicability: Cây quyết định là một phương pháp phi tham số để xây dựng mô hình phân loại. Phương pháp này không yêu cầu bất kỳ giả định nào trước đó về phân phối xác suất chi phối các lớp và thuộc tính của dữ liệu, do đó có thể áp dụng cho nhiều loại tập dữ liệu khác nhau. Nó cũng có thể áp dụng cho cả dữ liệu phân loại và dữ liệu liên tục mà không cần chuyển đổi các thuộc tính về cùng một dạng thông qua nhị phân hóa (binarization), chuẩn hóa (normalization) hoặc tiêu chuẩn hóa (standardization). Không giống như một số bộ phân loại nhị phân, cây quyết định có thể xử lý trực tiếp các bài toán phân loại đa lớp mà không cần phải chia nhỏ chúng thành nhiều bài toán phân loại nhị phân. Một đặc điểm hấp dẫn khác của bộ phân loại cây quyết định là các cây được tạo ra, đặc biệt là những cây có độ sâu ngắn, tương đối dễ giải thích. Độ chính xác của cây quyết định cũng có thể so sánh được với các kỹ thuật phân loại khác khi áp dụng cho nhiều tập dữ liệu đơn giản.

2. Tính biểu đạt - Expressivenes: Cây quyết định cung cấp một biểu diễn phổ quát cho các hàm có giá trị rời rạc. Nói cách khác, nó có thể mã hóa bất kỳ hàm nào có đầu ra là giá trị rời rạc. Điều này là do mỗi hàm có giá trị rời rạc có thể được biểu diễn dưới dạng một bảng gán (assignment table), trong đó mỗi tổ hợp duy nhất của các thuộc tính rời rạc được gán cho một nhãn lớp. Vì mọi tổ hợp thuộc tính đều có thể được biểu diễn dưới dạng một nút lá trong cây quyết định, nên ta luôn có thể tìm được một cây quyết định có nhãn lớp tại các nút lá khớp với bảng gán của hàm ban đầu. Cây quyết định cũng giúp biểu diễn gọn gàng hơn các hàm phức tạp, bằng cách nhóm một số tổ hợp thuộc tính lại trong cùng một nút lá. Ví dụ, **Hình 1.15** hiển thị bảng gán của hàm Boolean $(A \wedge B) \vee (C \wedge D)$ với bốn thuộc tính nhị phân, dẫn đến tổng cộng 2^4 = 16 tổ hợp có thể có. Cây quyết định trong **Hình 1.15** cho thấy một cách mã hóa nén của bảng gán này. Thay vì cần một cây đầy đủ với 16 nút lá, ta có thể mã hóa hàm này bằng một cây đơn giản hơn chỉ với 7 nút lá. Tuy nhiên, không phải mọi cây quyết định cho các thuộc tính rời rạc đều có thể được rút gọn. Một ví dụ đáng chú ý là hàm chẵn lẻ (parity function), trong đó giá trị đầu ra bằng 1 khi số lượng giá trị đúng trong các thuộc tính Boolean là số chẵn, và bằng 0 nếu là số lẻ. Việc mô hình hóa chính xác một hàm như vậy cần một cây quyết định đầy đủ với 2d nút lá, trong đó d là số lượng thuộc tính Boolean.

```
Decision Tree:
depth = 1:
| breadth> 7 :  class 1
| breadth<= 7:
| | breadth <= 3:
| | | ImagePages> 0.375:  class 0
| | | ImagePages<= 0.375:
| | | | totalPages<= 6:  class 1
| | | | totalPages> 6:
| | | | | breadth <= 1:  class 1
| | | | | breadth > 1:  class 0
| | width > 3:
| | | MultiIP = 0:
| | | | ImagePages<= 0.1333:  class 1
| | | | ImagePages> 0.1333:
| | | | breadth <= 6:  class 0
| | | | breadth > 6:  class 1
| | | MultiIP = 1:
| | | | TotalTime <= 361:  class 0
| | | | TotalTime > 361:  class 1
depth> 1:
| MultiAgent = 0:
| | depth > 2:  class 0
| | depth < 2:
| | | MultiIP = 1:  class 0
| | | MultiIP = 0:
| | | | breadth <= 6:  class 0
| | | | breadth > 6:
| | | | | RepeatedAccess <= 0.322:  class 0
| | | | | RepeatedAccess > 0.322:  class 1
| MultiAgent = 1:
| | totalPages <= 81:  class 0
| | totalPages > 81:  class 1
```

Hình 1.14: Mô hình cây quyết định phát hiện robot web

A	B	C	D	class
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

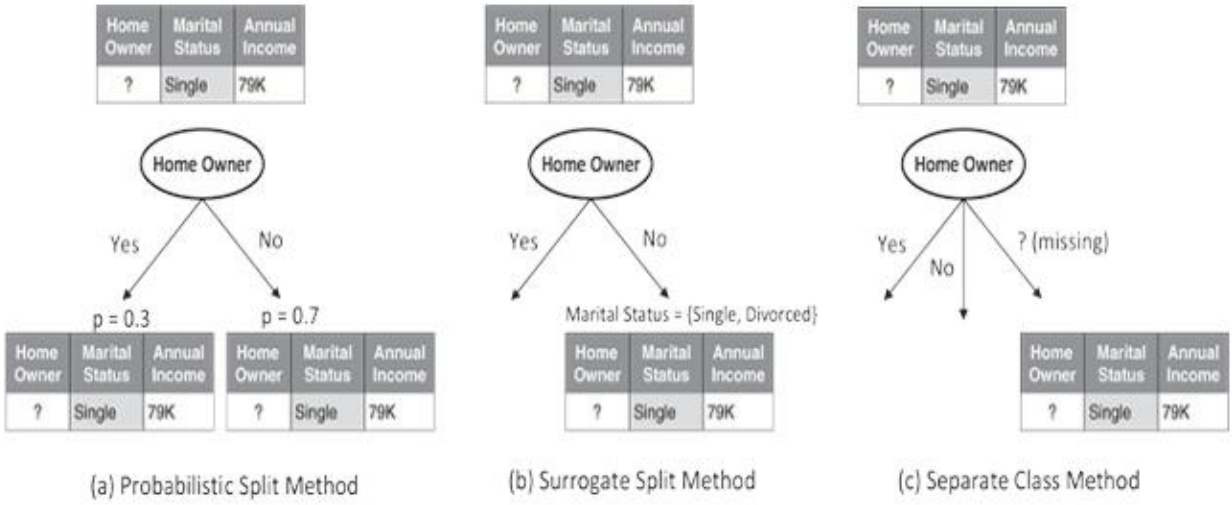


Hình 1.15: Cây quyết định cho hàm Boolean (A∧B)∨(C∧D)

4. Hiệu quả tính toán – Computational Efficiency: Vì số lượng cây quyết định có thể có là rất lớn, nhiều thuật toán cây quyết định sử dụng cách tiếp cận dựa trên heuristic để hướng dẫn quá trình tìm kiếm trong không gian giả thuyết rộng lớn. Ví dụ, thuật toán được trình bày trong **Mục 1.4** sử dụng chiến lược phân vùng đệ quy từ trên xuống theo cách tham lam (greedy, top-down, recursive partitioning strategy) để phát triển cây quyết định. Đối với nhiều tập dữ liệu, các kỹ thuật như vậy giúp nhanh chóng xây dựng một cây quyết định có độ chính xác khá tốt ngay cả khi kích thước tập huấn luyện nhỏ. Hơn nữa, khi cây quyết định đã được xây dựng, việc phân loại một mẫu kiểm tra cực kỳ nhanh chóng, với độ phức tạp tệ nhất là O(w), trong đó w là độ sâu tối đa của cây.

4. Xử lý giá trị bị thiếu – Handling Missing Values: Một bộ phân loại cây quyết định có thể xử lý các thuộc tính bị thiếu theo nhiều cách khác nhau, cả trong tập huấn luyện và tập kiểm tra. Khi có giá trị bị thiếu trong tập kiểm tra, bộ phân loại phải quyết định xem nên đi theo nhánh nào nếu giá trị của thuộc tính phân tách bị thiếu trong một mẫu kiểm tra. Một cách tiếp cận, được gọi là phương pháp phân tách xác suất (probabilistic split method), được sử dụng trong thuật toán C4.5. Phương pháp này phân phối mẫu dữ liệu vào mọi nhánh con của nút phân tách theo xác suất xuất hiện của giá trị bị thiếu. Ngược lại, thuật toán CART sử dụng phương pháp phân tách thay thế (surrogate split method), trong đó nếu giá trị của thuộc tính phân tách bị thiếu, mẫu dữ liệu được gán vào một trong các nhánh con dựa trên giá trị của một thuộc tính thay thế không bị thiếu và thuộc tính thay thế này có cách phân chia dữ liệu giống nhất với thuộc tính đang bị thiếu. Một cách tiếp cận khác, được gọi là phương pháp lớp riêng biệt (separate class method), được sử dụng bởi thuật toán CHAID. Trong phương pháp này, giá trị bị thiếu được xem là một lớp phân loại riêng biệt, khác với các giá trị thông thường của thuộc tính phân tách. **Hình 1.16** minh họa ba phương pháp trên để xử lý giá trị bị thiếu trong cây quyết định. Ngoài ra, có các chiến lược khác để xử lý giá trị bị thiếu trong bước tiền xử lý dữ liệu, chẳng hạn như điền giá trị bị thiếu bằng giá trị mode (nếu thuộc tính là dữ liệu phân loại) hoặc điền giá trị bị thiếu bằng

giá trị trung bình (nếu thuộc tính là dữ liệu liên tục) hoặc loại bỏ mẫu có giá trị bị thiếu trước khi đưa vào phân loại.



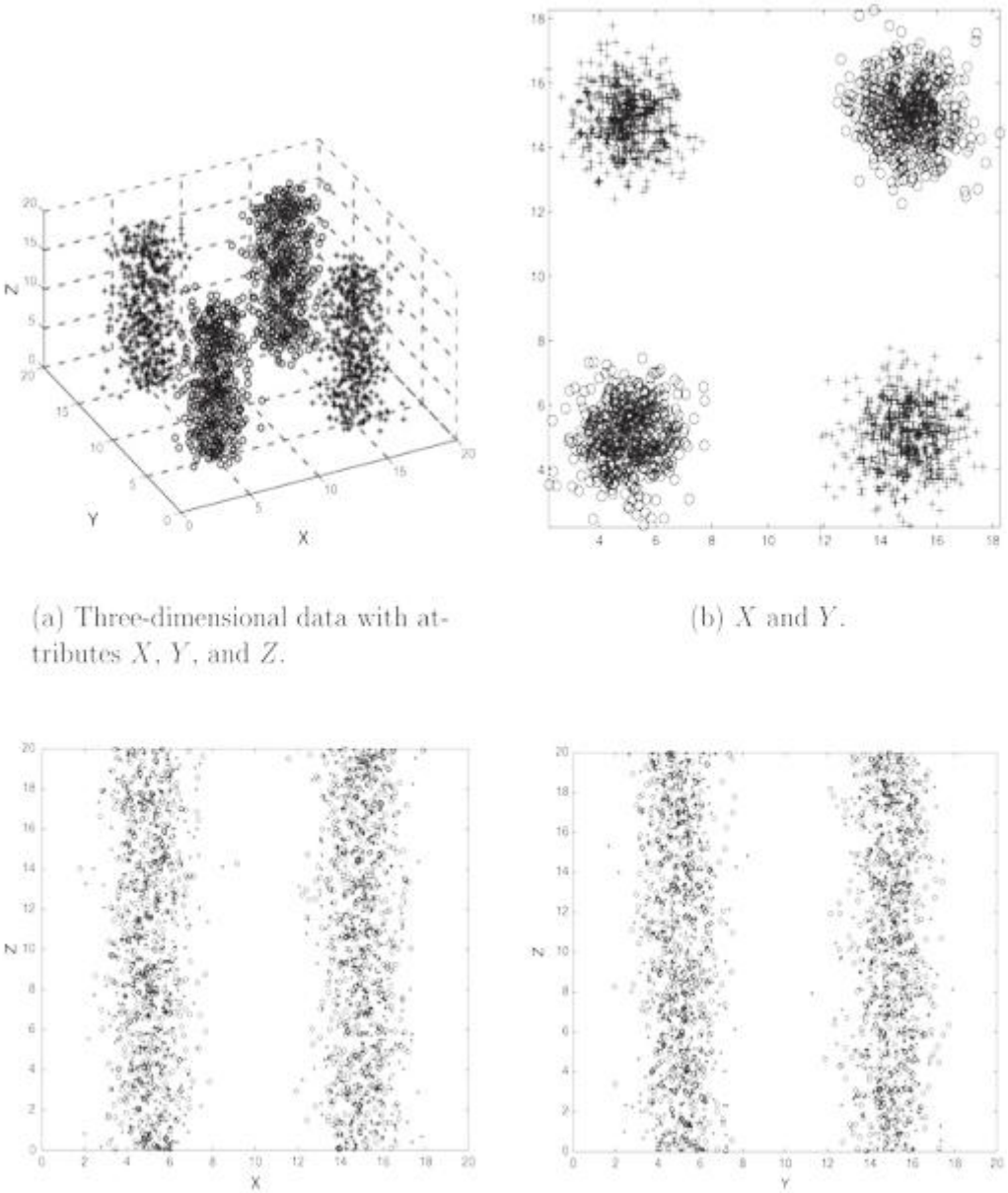
Hình 1.16: Các phương pháp xử lý giá trị thuộc tính bị thiếu trong bộ phân loại cây quyết định

Trong quá trình huấn luyện, nếu một thuộc tính v có giá trị bị thiếu trong một số mẫu huấn luyện tại một nút, chúng ta cần có cách đo mức độ tinh khiết (purity) nếu chọn v làm thuộc tính phân tách. Một cách đơn giản là loại bỏ các mẫu có giá trị bị thiếu của v và tính toán số lần xuất hiện của từng giá trị có thể có của v trong tất cả các nút con. Ngoài ra, nếu v được chọn làm điều kiện phân tách tại một nút, các mẫu có giá trị bị thiếu của v có thể được phân phối vào các nút con bằng một trong các phương pháp xử lý giá trị bị thiếu đã đề cập ở trên.

5. Xử lý tương tác giữa các thuộc tính – Handling Interactions among Attributes: Các thuộc tính được xem là có sự tương tác nếu chúng có thể phân biệt các lớp khi kết hợp với nhau, nhưng không có nhiều thông tin khi sử dụng riêng lẻ. Do tính chất tham lam (greedy) của các tiêu chí phân tách trong cây quyết định, các thuộc tính quan trọng này có thể bị bỏ qua nếu chúng không thể phân tách tốt một cách độc lập. Điều này có thể dẫn đến việc tạo ra các cây quyết định phức tạp hơn mức cần thiết. Do đó, cây quyết định có thể hoạt động kém hơn khi có sự tương tác giữa các thuộc tính.

Ví dụ minh họa, xét dữ liệu ba chiều trong **Hình 1.17(a)**, chứa 2000 điểm dữ liệu thuộc hai lớp, được ký hiệu là + và ○ trong sơ đồ. **Hình 1.17(b)** hiển thị phân bố của hai lớp trong không gian hai chiều với các thuộc tính X và Y, mô phỏng một phiên bản nhiễu của hàm XOR Boolean. Ta có thể thấy rằng hai lớp được phân tách tốt trong không gian này, nhưng không có thuộc tính nào (X hoặc Y) có đủ thông tin để phân biệt hai lớp nếu chỉ dùng riêng lẻ. Ví dụ, entropy của điều kiện phân tách $X \leq 10$ và $Y \leq 10Y$ đều gần bằng 1, cho thấy rằng cả X và Y đều không giảm được độ không thuần khiết (impurity) khi sử dụng một mình. Do đó, X và Y thể hiện sự tương tác giữa các thuộc tính. Dữ liệu cũng chứa thuộc tính thứ ba, Z, trong đó các lớp được phân bố đồng đều, như minh họa trong **Hình**

1.17(c) và **Hình 1.17(d)**. Vì vậy, entropy của bất kỳ phân tách nào liên quan đến Z đều gần bằng 1. Do đó, Z có nhiều khả năng bị chọn làm thuộc tính phân tách, thay vì hai thuộc tính thực sự có thông tin hữu ích là X và Y.



(a) Three-dimensional data with attributes X, Y, and Z.

(b) X and Y.

(c) X and Z.

(d) Y and Z.

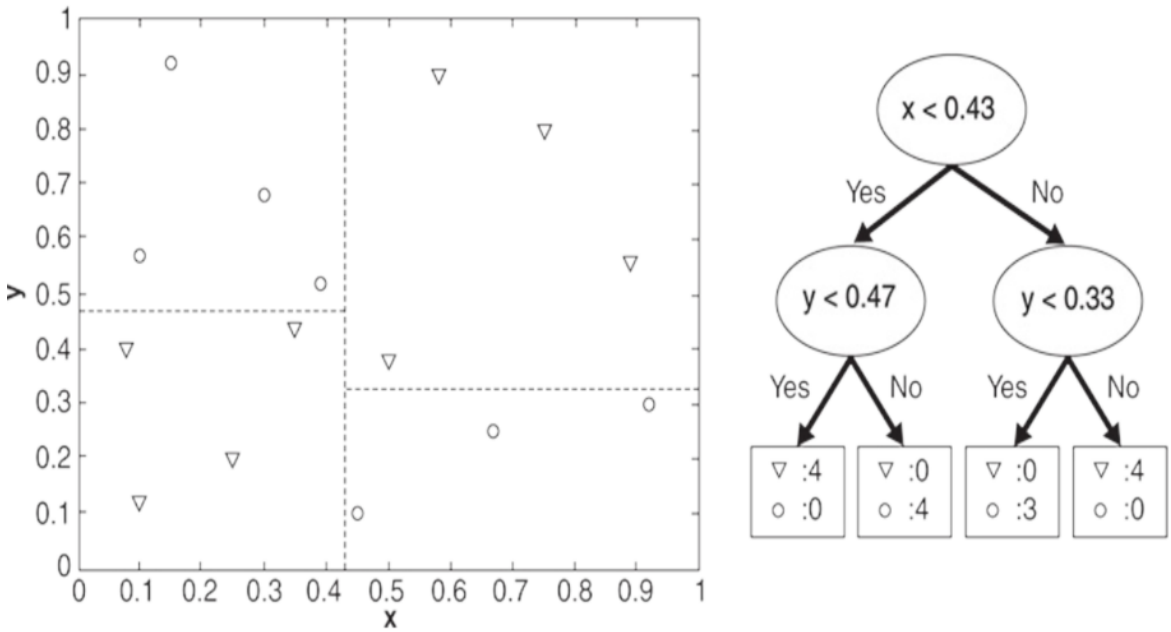
Hình 1.17: Dữ liệu XOR liên quan đến X và Y cùng một thuộc tính không liên quan Z

6. Xử lý các thuộc tính không liên quan – Handling Irrelevant Attributes: Một thuộc tính được coi là không liên quan nếu nó không hữu ích cho bài toán phân loại. Vì các thuộc tính không liên quan có mối liên hệ kém với nhãn lớp mục tiêu, chúng sẽ không giúp tăng độ tinh khiết (purity) và do đó sẽ bị bỏ qua bởi các thuộc tính liên quan hơn. Do đó, sự xuất hiện của một số lượng nhỏ các thuộc tính không liên quan sẽ không ảnh hưởng đến quá trình xây dựng cây quyết định. Tuy nhiên, không phải tất cả các thuộc tính cung cấp ít hoặc không có lợi ích đều bị coi là không liên quan (xem **Hình 1.17**). Nếu bài toán

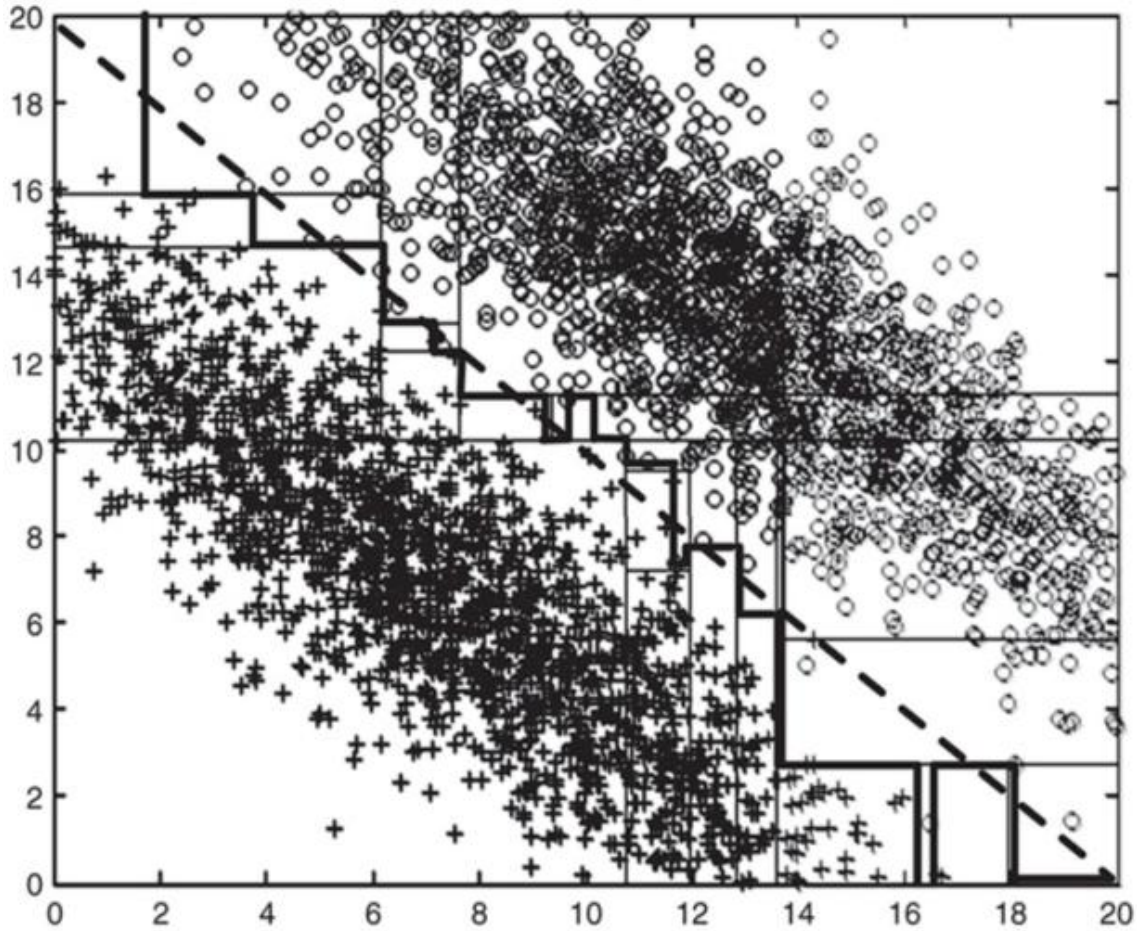
phân loại phức tạp (ví dụ: có sự tương tác giữa các thuộc tính) và có nhiều thuộc tính không liên quan, thì một số thuộc tính này có thể vô tình được chọn trong quá trình xây dựng cây, vì chúng có thể cung cấp độ lợi nhuận cao hơn một thuộc tính liên quan chỉ do may mắn. Các kỹ thuật chọn lọc thuộc tính (feature selection techniques) có thể giúp cải thiện độ chính xác của cây quyết định bằng cách loại bỏ các thuộc tính không liên quan trong giai đoạn tiền xử lý.

7. Xử lý các thuộc tính dư thừa – Handling Redundant Attributes: Một thuộc tính được coi là dư thừa nếu nó có mối tương quan mạnh với một thuộc tính khác trong tập dữ liệu. Vì các thuộc tính dư thừa mang lại lợi ích tương tự nhau về độ tinh khiết (purity) nếu chúng được chọn làm điều kiện phân tách, chỉ một trong số chúng sẽ được chọn làm điều kiện phân tách trong thuật toán cây quyết định. Do đó, cây quyết định có thể xử lý tốt sự hiện diện của các thuộc tính dư thừa.

8. Sử dụng phân tách trực giao – Using Rectilinear Splits: Các điều kiện kiểm tra được mô tả trong chương này chỉ liên quan đến một thuộc tính tại một thời điểm. Do đó, quá trình xây dựng cây có thể được xem là quá trình phân vùng không gian thuộc tính thành các vùng không chồng lấn, cho đến khi mỗi vùng chứa các bản ghi thuộc cùng một lớp. Biên giới giữa hai vùng lân cận của các lớp khác nhau được gọi là đường biên quyết định (decision boundary). **Hình 1.18** hiển thị cây quyết định cũng như đường biên quyết định cho một bài toán phân loại nhị phân. Vì điều kiện kiểm tra chỉ liên quan đến một thuộc tính tại một thời điểm, nên đường biên quyết định có dạng trực giao, tức là song song với các trục tọa độ. Điều này hạn chế khả năng thể hiện của cây quyết định, đặc biệt là khi biểu diễn đường biên quyết định của các tập dữ liệu có thuộc tính liên tục. **Hình 1.19** minh họa một tập dữ liệu hai chiều chứa hai lớp nhị phân, không thể được phân loại chính xác bằng cây quyết định thông thường, do điều kiện kiểm tra chỉ dựa trên các thuộc tính riêng lẻ. Trong tập dữ liệu này, các lớp nhị phân được tạo ra từ hai phân phối Gaussian lệch, với tâm tại (8,8) và (12,12). Đường biên quyết định thực sự được biểu diễn bằng đường chéo nét đứt, trong khi đường biên quyết định trực giao do cây quyết định tạo ra được thể hiện bằng đường liền nét dày. Ngược lại, cây quyết định xiên (oblique decision tree) có thể khắc phục giới hạn này bằng cách cho phép điều kiện kiểm tra sử dụng nhiều hơn một thuộc tính cùng lúc. Ví dụ, dữ liệu phân loại nhị phân trong **Hình 1.19** có thể được biểu diễn dễ dàng bằng một cây quyết định xiên, với nút gốc duy nhất có điều kiện kiểm tra: $x + y < 20$.



Hình 1.18: Ví dụ về một cây quyết định và các ranh giới quyết định của nó cho một tập dữ liệu hai chiều



Hình 1.19: Ví dụ về một tập dữ liệu không thể được phân vùng tối ưu bằng cây quyết định với các điều kiện kiểm tra trên từng thuộc tính riêng lẻ. Ranh giới quyết định thực sự được hiển thị bằng đường nét đứt

Mặc dù cây quyết định xiên có tính biểu đạt cao hơn và có thể tạo ra các cây gọn hơn, việc tìm điều kiện kiểm tra tối ưu lại tốn kém hơn về mặt tính toán.

9. Lựa chọn thước đo độ tạp nhiễm – Choice of Impurity Measure: Cần lưu ý rằng việc chọn thước đo độ tạp nhiễm thường không ảnh hưởng nhiều đến hiệu suất của các bộ phân loại cây quyết định, bởi vì hầu hết các thước đo độ tạp nhiễm đều khá nhất quán với nhau. Thay vào đó, chiến lược cắt tỉa cây (pruning strategy) có tác động lớn hơn đến cấu trúc cây quyết định cuối cùng so với việc lựa chọn thước đo độ tạp nhiễm.

1.7. Một ví dụ về xây dựng cây quyết định: bài toán play tennis [2]

1.7.1. Mô tả bài toán

"David là quản lý của một câu lạc bộ đánh Tennis nổi tiếng. Anh ta đang có rắc rối chuyện các thành viên đến hay không đến. Có ngày ai cũng muốn chơi tennis nhưng số nhân viên câu lạc bộ lại không đủ phục vụ. Có hôm, không hiểu vì lý do gì mà chẳng ai đến chơi, và câu lạc bộ lại thừa nhân viên.

Mục tiêu của David là tối ưu hóa số nhân viên phục vụ mỗi ngày bằng cách dựa theo thông tin dự báo thời tiết để đoán xem khi nào người ta sẽ đến chơi.

Để thực hiện điều đó, anh cần hiểu được tại sao khách hàng quyết định chơi và tìm hiểu xem có cách giải thích nào cho việc đó hay không.

Vậy là trong hai tuần, anh ta thu thập thông tin về:

- Quang cảnh (outlook): nắng (sunny), âm u (overcast) hoặc mưa (raining);
- Nhiệt độ (temperature): Hot, Cool, Mild
- Độ ẩm (humidity): High, Normal
- Có gió mạnh (windy) hay không: Strong, Weak
- Và tất nhiên là số người đến chơi golf vào hôm đó.

David thu được một bộ dữ liệu gồm 14 dòng và 5 cột" như trong **Hình 1.20**.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Hình 1.20: Dữ liệu của bài toán play tennis

1.7.2. Xây dựng cây quyết định

Hình 1.21 là các công thức cần thiết để có thể xây dựng cây quyết định.

- Entropy (S) is the minimum number of bits needed in order to classify an arbitrary example as **yes** or **no**

  Entropy(S) = $\sum_{i=1}^{c} -p_i \log_2 p_i$

  - Where S is a set of training examples,
  - c is the number of classes, and
  - p_i is the proportion of the training set that is of class i
  - For our entropy equation $0 \log_2 0 = 0$
- The information gain G(S, A), where A is an attribute:

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Hình 1.21: Các công thức để xây dựng cây quyết định

Các bước xây dựng cây quyết định:

- S có 9 Yes, 5 No hay S = [9^+, 5^-]

$$\boldsymbol{Entropy(S)} = Entropy([9^+, 5^-]) = -\frac{9}{14} \times \log_2 \frac{9}{14} - \frac{5}{14} \times \log_2 \frac{5}{14} \approx \mathbf{0.94}$$

- Values(Outlook) = {Sunny, Rain, Overcast}

S_{Sunny} = [2^+, 3^-] => $\boldsymbol{Entropy(S_{Sunny})} = -\frac{2}{5} \times \log_2 \frac{2}{5} - \frac{3}{5} \times \log_2 \frac{3}{5} \approx \mathbf{0.97}$

S_{Rain} = [3^+, 2^-] => $\boldsymbol{Entropy(S_{Rain})} = -\frac{3}{5} \times \log_2 \frac{3}{5} - \frac{2}{5} \times \log_2 \frac{2}{5} \approx \mathbf{0.97}$

$S_{Overcast}$ = [4^+, 0^-] => $\boldsymbol{Entropy(S_{Overcast})} = -\frac{4}{4} \times \log_2 \frac{4}{4} - \frac{0}{4} \times \log_2 \frac{0}{4} = \mathbf{0}$

$$\boldsymbol{InfoGain(S, Outlook)} = 0.94 - \frac{5}{14} \times 0.97 - \frac{5}{14} \times 0.97 - \frac{4}{14} \times 0 \approx \mathbf{0.247}$$

- Values(Temperature) = {Hot, Cool, Mild}

S_{Hot} = [2^+, 2^-] => $\boldsymbol{Entropy(S_{Hot})} = -\frac{2}{4} \times \log_2 \frac{2}{4} - \frac{2}{4} \times \log_2 \frac{2}{4} = \mathbf{1}$

S_{Cool} = [3^+, 1^-] => $\boldsymbol{Entropy(S_{Cool})} = -\frac{3}{4} \times \log_2 \frac{3}{4} - \frac{1}{4} \times \log_2 \frac{1}{4} \approx \mathbf{0.81}$

S_{Mild} = [4^+, 2^-] => $\boldsymbol{Entropy(S_{Mild})} = -\frac{4}{6} \times \log_2 \frac{4}{6} - \frac{2}{6} \times \log_2 \frac{2}{6} \approx \mathbf{0.91}$

$$\boldsymbol{InfoGain(S, Temperature)} = 0.94 - \frac{4}{14} \times 1 - \frac{4}{14} \times 0.81 - \frac{6}{14} \times 0.91 \approx \mathbf{0.03}$$

- Values(Humidity) = {High, Normal}

S_{High} = [3^+, 4^-] => $\boldsymbol{Entropy(S_{High})} = -\frac{3}{7} \times \log_2 \frac{3}{7} - \frac{4}{7} \times \log_2 \frac{4}{7} \approx \mathbf{0.99}$

S_{Normal} = [6^+, 1^-] => $\boldsymbol{Entropy(S_{Normal})} = -\frac{6}{7} \times \log_2 \frac{6}{7} - \frac{1}{7} \times \log_2 \frac{1}{7} \approx \mathbf{0.59}$

$$\boldsymbol{InfoGain(S, Humidity)} = 0.94 - \frac{7}{14} \times 0.99 - \frac{7}{14} \times 0.59 \approx \mathbf{0.15}$$

- Values(Wind) = {Weak, Strong}

S_{Weak} = [6^+, 2^-] => $\boldsymbol{Entropy(S_{Weak})} = -\frac{6}{8} \times \log_2 \frac{6}{8} - \frac{2}{8} \times \log_2 \frac{2}{8} \approx \mathbf{0.81}$

S_{Strong} = [3^+, 3^-] => $\boldsymbol{Entropy(S_{Strong})} = -\frac{3}{6} \times \log_2 \frac{3}{6} - \frac{3}{6} \times \log_2 \frac{3}{1} = \mathbf{1}$

$$\boldsymbol{InfoGain(S, Wind)} = 0.94 - \frac{8}{14} \times 0.81 - \frac{6}{14} \times 1 \approx \mathbf{0.048}$$

=> Vì độ lợi thông tin của Outlook là lớn nhất nên sẽ chọn thuộc tính này để làm nút gốc của cây quyết định.

Thuộc tính Outlook có 3 giá trị và sẽ xét lần lượt từng giá trị:

- Sunny:

$$\boldsymbol{Gain(S_{Sunny}, Humidity)}$$

$$= 0{,}97 - \frac{3}{5} \times \text{Entropy}\left(S_{\text{Sunny High}}\right) - \frac{2}{5} \times \text{Entropy}\left(S_{\text{Sunny Normal}}\right)$$

$$= 0{,}97 - \frac{3}{5} \times \left(-\frac{0}{3} \times \log_2 \frac{0}{3} - \frac{3}{3} \times \log_2 \frac{3}{3}\right) - \frac{2}{5} \times \left(-\frac{2}{2} \times \log_2 \frac{2}{2} - \frac{0}{2} \times \log_2 \frac{0}{2}\right)$$

$$= 0.97 - \frac{3}{5} \times 0 - \frac{2}{5} \times 0 = \mathbf{0.97}$$

$$\boldsymbol{Gain(S_{Sunny}, Temperature)}$$

$$= 0{,}97 - \frac{2}{5} \times \text{Entropy}\left(S_{\text{Sunny Hot}}\right) - \frac{2}{5} \times \text{Entropy}\left(S_{\text{Sunny Mild}}\right)$$

$$- \frac{1}{5} \times \text{Entropy}\left(S_{\text{Sunny Cool}}\right)$$

$$= 0.97 - \frac{2}{5} \times 0 - \frac{2}{5} \times 1 - \frac{1}{5} \times 0 = \mathbf{0.57}$$

$$\boldsymbol{Gain(S_{Sunny}, Wind)}$$

$$= 0{,}97 - \frac{2}{5} \times \text{Entropy}\left(S_{\text{Sunny Strong}}\right) - \frac{3}{5} \times \text{Entropy}\left(S_{\text{Sunny Weak}}\right)$$

$$= 0.97 - \frac{2}{5} \times 1 - \frac{3}{5} \times 0.92 = \mathbf{0.19}$$

=> Vì độ lợi thông tin đối với Humidity của Sunny là lớn nhất nên sẽ chọn thuộc tính này làm nút con của Sunny và đến đây dựa vào 2 thuộc tính của Humidity là High, Normal đã chia rõ các trường hợp Yes, No nên sẽ dừng lại tại đây ở nhánh này.

- Rain:

$$\boldsymbol{Gain(S_{Rain}, Temperature)}$$

$$= 0{,}97 - \frac{0}{5} \times \text{Entropy}\left(S_{\text{Rain Hot}}\right) - \frac{2}{5} \times \text{Entropy}\left(S_{\text{Rain Cool}}\right)$$

$$- \frac{3}{5} \times \text{Entropy}\left(S_{\text{Rain Mild}}\right)$$

$$= 0.97 - \frac{0}{5} \times 0 - \frac{2}{5} \times 1 - \frac{3}{5} \times 0.92 = \mathbf{0.018}$$

$$\boldsymbol{Gain(S_{Rain}, Humidity)}$$

$$= 0{,}97 - \frac{2}{5} \times \text{Entropy}\left(S_{\text{Rain High}}\right) - \frac{3}{5} \times \text{Entropy}\left(S_{\text{Rain Normal}}\right)$$

$$= 0.97 - \frac{2}{5} \times 1 - \frac{3}{5} \times 0.92 = \mathbf{0.018}$$

$$\boldsymbol{Gain(S_{Rain}, Wind)}$$

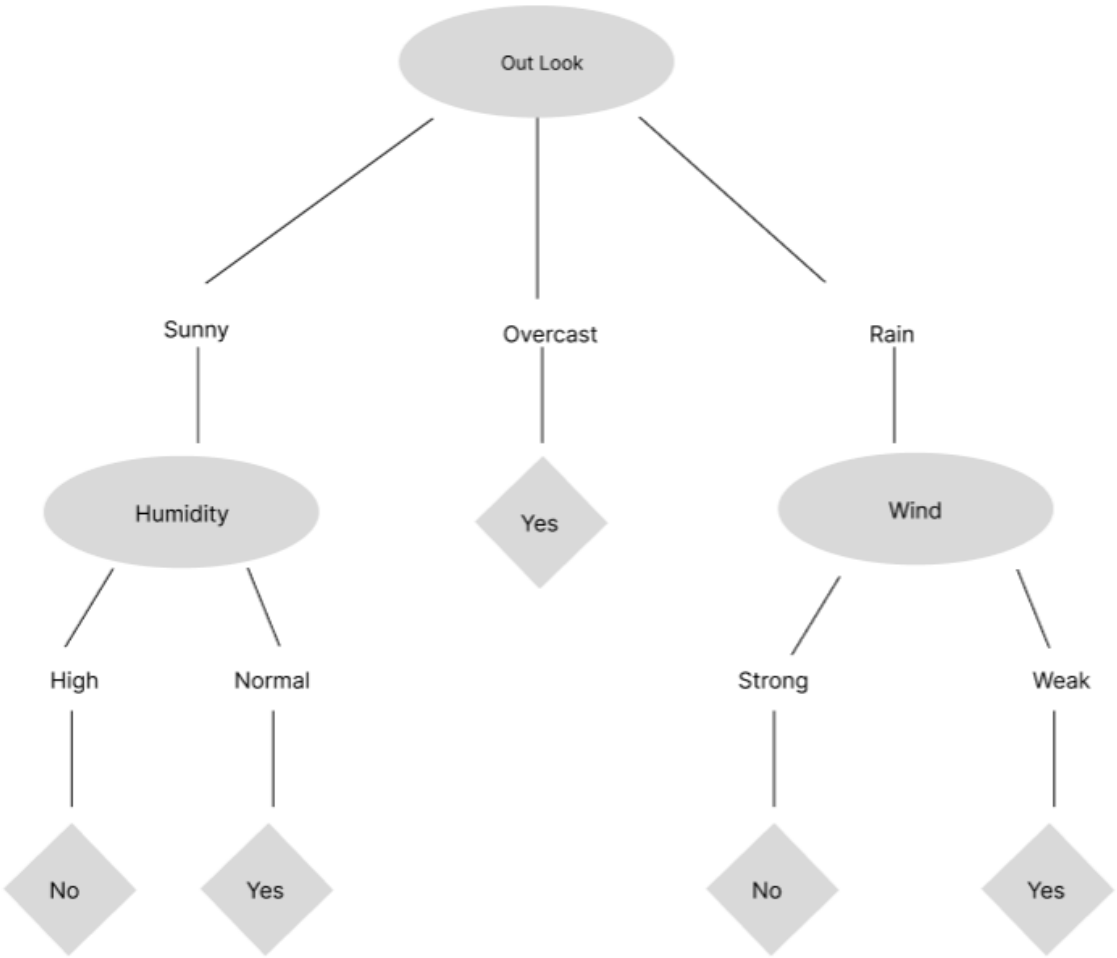
$$= 0{,}97 - \frac{2}{5} \times \text{Entropy}\left(S_{\text{Rain Strong}}\right) - \frac{3}{5} \times \text{Entropy}\left(S_{\text{Rain Weak}}\right)$$

$$= 0.97 - \frac{2}{5} \times 0 - \frac{3}{5} \times 0 = \mathbf{0.97}$$

=> Vì độ lợi thông tin đối với Wind của Rain là lớn nhất nên sẽ chọn thuộc tính này làm nút con của Sunny và đến đây dựa vào 2 thuộc tính của Wind là Strong, Weak đã chia rõ các trường hợp Yes, No nên sẽ dừng lại tại đây ở nhánh này.

- Overcast luôn cho Yes nên không cần thực hiện tính toán trên nhánh này.

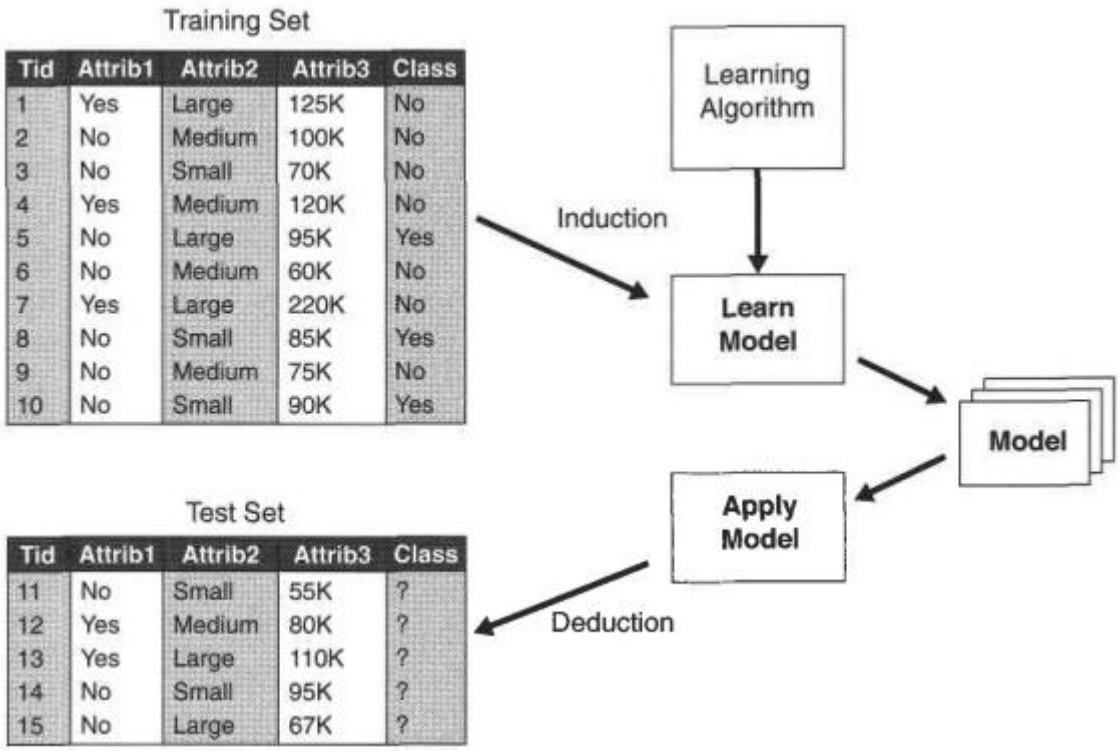
Từ các tính toán trên, ta sẽ vẽ được cây quyết định như ở **Hình 1.22**.



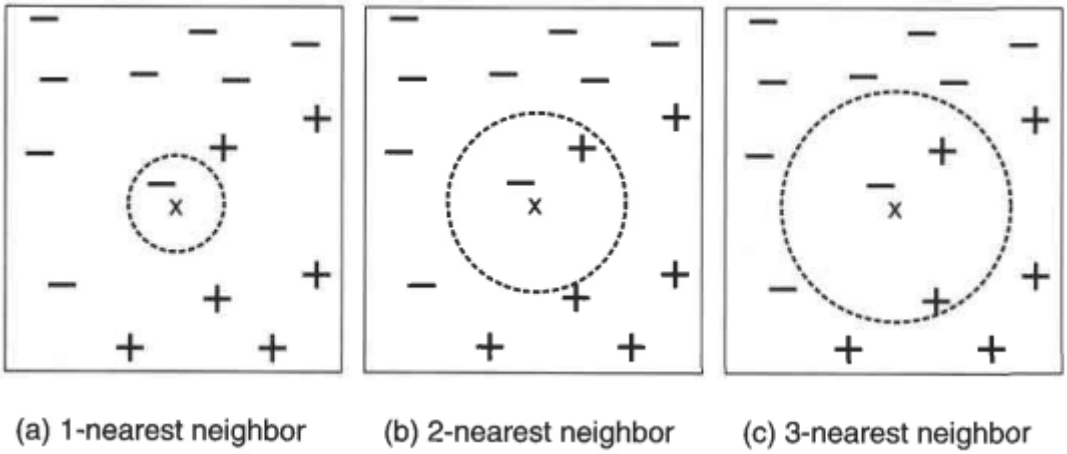
Hình 1.22: Cây quyết định xây dựng được sau khi tính toán

CHƯƠNG 2: MÔ HÌNH KNN – NEAREST-NEIGHBOR CLASSIFIERS

Khung phân loại được minh họa trong **Hình 1.23** bao gồm một quy trình hai bước: (1) Bước quy nạp để xây dựng mô hình phân loại từ dữ liệu và (2) Bước suy diễn để áp dụng mô hình vào các mẫu kiểm tra. Các bộ phân loại dựa trên cây quyết định và luật là những ví dụ về **người học tích cực (eager learners)** vì chúng được thiết kế để học một mô hình ánh xạ các thuộc tính đầu vào đến nhãn lớp ngay khi dữ liệu huấn luyện có sẵn. Một chiến lược đối lập là trì hoãn quá trình mô hình hóa dữ liệu huấn luyện cho đến khi cần phân loại các mẫu kiểm tra. Các kỹ thuật sử dụng chiến lược này được gọi là **người học lười biếng (lazy learners**). Một ví dụ về người học lười biếng là **bộ phân loại ghi nhớ (Rote classifier)**, bộ này ghi nhớ toàn bộ dữ liệu huấn luyện và chỉ thực hiện phân loại khi các thuộc tính của một mẫu kiểm tra khớp chính xác với một trong các mẫu huấn luyện. Một nhược điểm rõ ràng của phương pháp này là một số mẫu kiểm tra có thể không được phân loại vì chúng không khớp với bất kỳ ví dụ huấn luyện nào.



Hình 1.23: Cách tiếp cận tổng quát để xây dựng mô hình phân loại

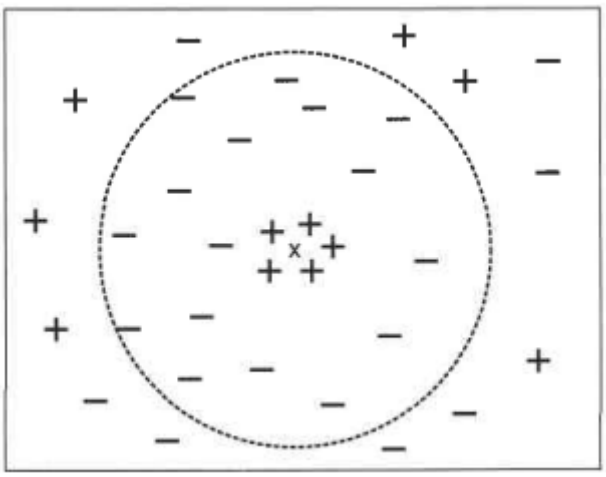


Hình 1.24: Các hàng xóm gần nhất thứ 1, 2 và 3 của một mẫu

Một cách để làm cho phương pháp này linh hoạt hơn là tìm tất cả các ví dụ huấn luyện có thuộc tính tương đối giống với mẫu kiểm tra. Những ví dụ này, được gọi là láng giềng gần nhất (nearest neighbors), có thể được sử dụng để xác định nhãn lớp của mẫu kiểm tra. Lý do sử dụng láng giềng gần nhất được minh họa rõ nhất bằng câu nói: "Nếu nó đi như vịt, kêu như vịt, và trông giống vịt, thì có lẽ nó là một con vịt." Bộ phân loại láng giềng gần nhất biểu diễn mỗi ví dụ dưới dạng một điểm dữ liệu trong không gian d chiều, trong đó d là số lượng thuộc tính. Khi có một mẫu kiểm tra, ta tính toán mức độ gần gũi của nó với các điểm dữ liệu khác trong tập huấn luyện, sử dụng một trong các phương pháp đo lường độ gần. K láng giềng gần nhất (k-nearest neighbors) của một ví dụ z là k điểm gần nhất với z.

Hình 1.24 minh họa các 1-, 2- và 3-láng giềng gần nhất của một điểm dữ liệu nằm ở trung tâm của mỗi vòng tròn. Dữ liệu đầu vào được phân loại dựa trên nhãn của các láng giềng gần nhất. Nếu các láng giềng có nhiều hơn một nhãn, ta sử dụng quy tắc đa số để gán nhãn. Trong **Hình 1.24 (a)**, điểm dữ liệu có l-láng giềng gần nhất là một ví dụ âm. Do đó, điểm dữ liệu được gán vào lớp âm. Nếu số lượng láng giềng gần nhất là ba, như trong **Hình 1.24 (c)**, thì vùng lân cận chứa hai ví dụ dương và một ví dụ âm. Sử dụng phương pháp bỏ phiếu đa số, điểm dữ liệu được gán vào lớp dương. Trong trường hợp có sự hòa giữa các lớp (xem **Hình 1.24 (b)**), ta có thể chọn ngẫu nhiên một trong hai lớp để phân loại điểm dữ liệu.

Thảo luận trước đó nhấn mạnh tầm quan trọng của việc chọn giá trị k phù hợp. Nếu k quá nhỏ, bộ phân loại láng giềng gần nhất có thể dễ bị quá khớp (overfitting) do nhiễu trong dữ liệu huấn luyện. Mặt khác, nếu k quá lớn, bộ phân loại láng giềng gần nhất có thể phân loại sai mẫu kiểm tra vì danh sách các láng giềng gần nhất của nó có thể bao gồm các điểm dữ liệu nằm quá xa khỏi khu vực lân cận của nó (xem **Hình 1.25**).



Hình 1.25: Phân loại k-láng giềng gần nhất với số lượng lớn

2.1.Algorithm

Một bản tóm tắt cấp cao về phương pháp phân loại láng giềng gần nhất được trình bày trong thuật toán ở **Hình 1.26**. Thuật toán này tính khoảng cách (hoặc độ tương đồng) giữa mỗi mẫu kiểm tra z = (x', y') và tất cả các mẫu huấn luyện (x, y) ∈ D để xác định danh sách láng giềng gần nhất của nó, Dz. Việc tính toán này có thể tốn kém nếu số lượng mẫu huấn luyện quá lớn. Tuy nhiên, các kỹ thuật lập chỉ mục hiệu quả có thể được sử dụng để giảm số lượng phép tính cần thiết để tìm láng giềng gần nhất của một mẫu kiểm tra.

Algorithm 5.2 The k-nearest neighbor classification algorithm.

1: Let k be the number of nearest neighbors and D be the set of training examples.
2: **for** each test example $z = (\mathbf{x}', y')$ **do**
3: Compute $d(\mathbf{x}', \mathbf{x})$, the distance between z and every example, $(\mathbf{x}, y) \in D$.
4: Select $D_z \subseteq D$, the set of k closest training examples to z.
5: $y' = \underset{v}{\operatorname{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$
6: **end for**

Hình 1.26: Thuật toán k-nn

Khi danh sách các láng giềng gần nhất đã được xác định, mẫu kiểm tra được phân loại dựa trên lớp chiếm đa số của các láng giềng gần nhất:

$$Majority\ Voting: y' = \underset{v}{\operatorname{argmax}} \sum_{(x_i, y_i) \in D_z} I(v = y_i) \quad (8)$$

Trong đó: v là một nhãn lớp, y_i là nhãn lớp của một trong các láng giềng gần nhất, I(·) là một hàm chỉ thị trả về giá trị 1 nếu đối số của nó đúng và 0 nếu sai.

Trong phương pháp bỏ phiếu theo đa số, mỗi láng giềng có cùng trọng số ảnh hưởng đến kết quả phân loại. Điều này làm cho thuật toán nhạy cảm với giá trị k, như đã thấy trong **Hình 1.24**. Một cách để giảm ảnh hưởng của k là gán trọng số cho mỗi láng giềng gần nhất xi theo khoảng cách của nó: $w_i = 1/d(x', x_i)^2$. Điều này có nghĩa là các mẫu huấn luyện ở xa z sẽ có ảnh hưởng yếu hơn đến phân loại so với các mẫu gần z.Sử dụng

phương pháp bỏ phiếu có trọng số theo khoảng cách (Distance-Weighted Voting), nhãn lớp có thể được xác định như sau:

$$Distance - Weighted\ Voting: y' = \underset{v}{\operatorname{argmax}} \sum_{(x_i, y_i) \in D_z} w_i \times I(v = y_i) \quad (9)$$

2.2. Đặc điểm của láng giềng gần nhất- Characteristics Nearest-Neighbor Classifiers)

Các đặc điểm của bộ phân loại K-láng giềng gần nhất được tóm tắt như sau:

- Phân loại K-láng giềng gần nhất (KNN) là một phần của phương pháp học dựa trên mẫu (instance-based learning), sử dụng các mẫu huấn luyện cụ thể để đưa ra dự đoán mà không cần duy trì một mô hình trừu tượng (hoặc mô hình) được suy ra từ dữ liệu. Các thuật toán học dựa trên mẫu yêu cầu một độ đo khoảng cách để xác định độ tương đồng hoặc khoảng cách giữa các mẫu, cùng với một hàm phân loại để trả về lớp dự đoán của một mẫu thử dựa trên mức độ gần với các mẫu khác.

- Bộ học thụ động (Lazy learners) như KNN không yêu cầu xây dựng mô hình. Tuy nhiên, việc phân loại một mẫu thử có thể tốn kém về tính toán vì cần tính toán khoảng cách giữa mẫu thử và tất cả các mẫu huấn luyện. Ngược lại, các bộ học chủ động (Eager learners) thường dành phần lớn tài nguyên tính toán để xây dựng mô hình. Một khi mô hình đã được xây dựng, việc phân loại một mẫu thử diễn ra rất nhanh.

- Bộ phân loại KNN đưa ra dự đoán dựa trên thông tin cục bộ, trong khi cây quyết định và bộ phân loại dựa trên luật cố gắng tìm ra một mô hình tổng quát phù hợp với toàn bộ không gian đầu vào. Do quyết định phân loại được thực hiện dựa trên thông tin cục bộ, KNN với giá trị k nhỏ có thể dễ bị ảnh hưởng bởi nhiễu.

- Bộ phân loại KNN có thể tạo ra các ranh giới quyết định có hình dạng bất kỳ. Những ranh giới này cung cấp một mô hình linh hoạt hơn so với cây quyết định và bộ phân loại dựa trên luật, vốn thường bị giới hạn bởi các ranh giới quyết định hình chữ nhật. Tuy nhiên, các ranh giới quyết định của KNN có thể có độ biến động cao, vì chúng phụ thuộc vào sự phân bố của tập huấn luyện. Việc tăng số lượng hàng xóm k có thể làm giảm độ biến động này.

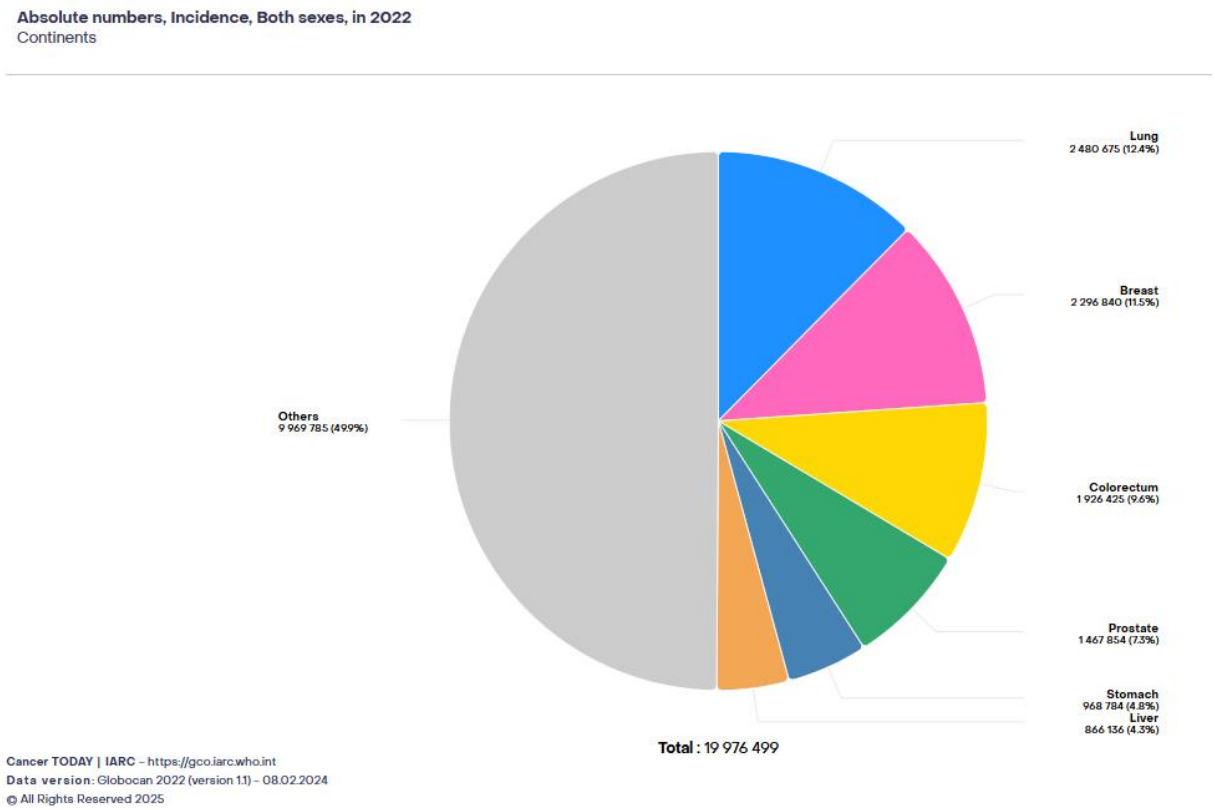
- Bộ phân loại KNN có thể tạo ra dự đoán sai nếu không áp dụng đúng các thước đo khoảng cách và các bước tiền xử lý dữ liệu. Ví dụ, giả sử chúng ta muốn phân loại một nhóm người dựa trên chiều cao (mét) và cân nặng (kg). Nếu chiều cao có biến động thấp (dao động từ 1.5m - 1.8m), trong khi cân nặng có biến động cao (từ 90 lb - 250 lb), thì thuộc tính chiều cao có thể không được xem xét đúng mức. Khi đó, khoảng cách giữa các mẫu có thể bị chi phối bởi sự khác biệt về trọng lượng thay vì chiều cao.

PHẦN 2: XÂY DỰNG ỨNG DỤNG HỖ TRỢ CHẨN ĐOÁN UNG THƯ PHỔI KHÔNG TẾ BÀO NHỎ

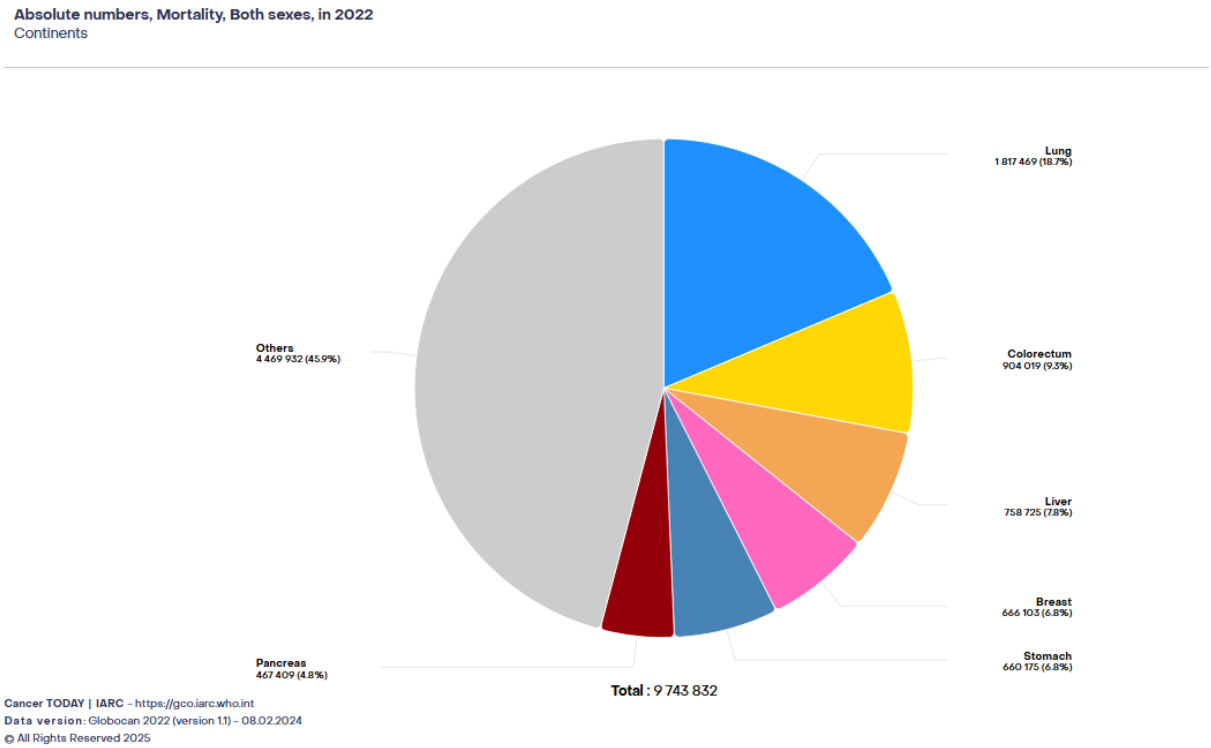
CHƯƠNG 1: GIỚI THIỆU VÀ MÔ TẢ BÀI TOÁN

1.1. Đặt vấn đề

Hiện này, ung thư phổi là một trong những căn bệnh nguy hiểm và có tỷ lệ tử vong cao nhất thế giới. Theo thống kê của GLOBOCAN vào năm 2022 [3], ung thư phổi là bệnh có số ca mắc bệnh và tử vong cao nhất trong tất cả các bệnh ung thư trên thế giới vào năm 2022, điều này được thể hiện dựa vào biểu đồ thống kê ở **Hình 2.1, Hình 2.2.**

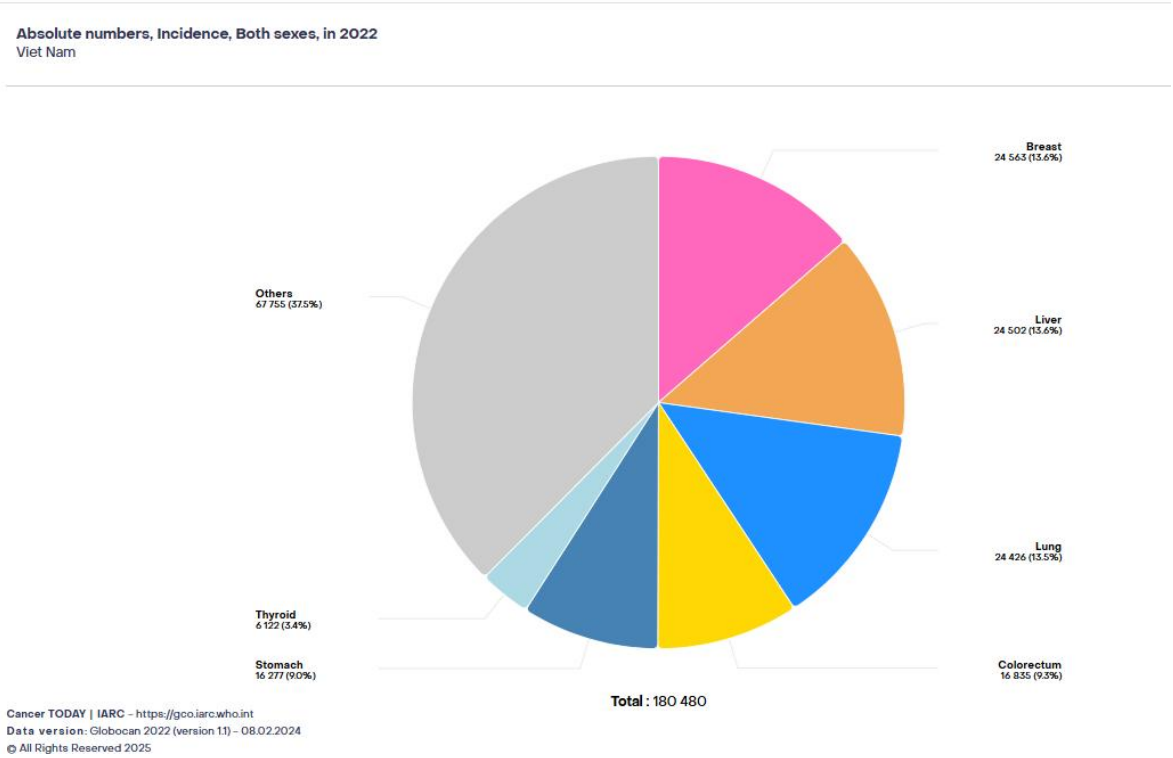


Hình 2.1: Biểu đồ thể hiện số lượng ca mắc ung thư trên thế giới năm 2022

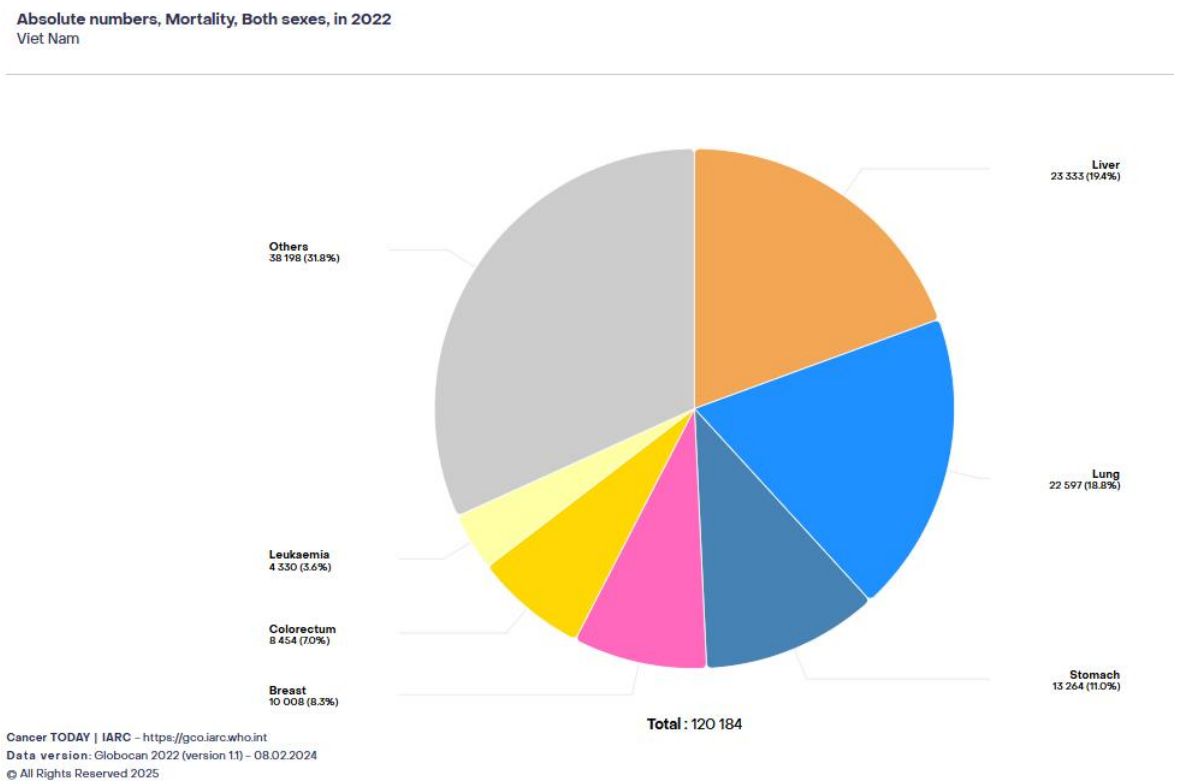


Hình 2.2: Biểu đồ thể hiện số lượng ca tử vong do ung thư trên thế giới năm 2022

Cũng theo số liệu về bệnh ung thư của GLOBOCAN năm 2022, tại Việt Nam ung thư phổi là bệnh ung thư phổ biến thứ ba, xếp sau ung thư vú và ung thư gan. Tuy nhiên, xét về số ca tử vong thì ung thư phổi lại chiếm thứ hai trong số những ca tử vong do mắc bệnh ung thư.



Hình 2.3: Biểu đồ thể hiện số lượng ca mắc ung thư tại Việt Nam năm 2022



Hình 2.4: Biểu đồ thể hiện số lượng ca tử vong do ung thư tại Việt Nam năm 2022

Theo "Current and Future Development in Lung Cancer Diagnosis" [4] và bản dịch tiếng Việt "Tổng quan Những tiến bộ trong chẩn đoán ung thư phổi: hiện tại và tương lai" [5], dựa vào cách phân chia theo mô bệnh học thì ung thư phổi cơ bản có thể chia làm hai loại là ung thư phổi tế bào nhỏ (SCLC) chiếm khoảng 10-15% tổng số ca bệnh và ung thư phổi không tế bào nhỏ (NSCLC) chiếm khoảng 80-85% tổng số ca bệnh. NSCLC lại bao gồm ung thư biểu mô tuyến (Adenocarcinoma) chiếm khoảng 40%, ung thư biểu mô tế bào vảy (Squamous Cell Carcinoma) chiếm khoảng 30% và ung thư biểu mô tế bào lớn (Large Cell Carcinoma) chiếm khoảng 10-15%. Điều này cho thấy việc phát hiện sớm ung thư phổi không tế bào nhỏ (NSCLC) là vô cùng quan trọng trong việc hỗ trợ điều trị. Hiện nay, có nhiều kỹ thuật hình ảnh giúp chẩn đoán như chụp cắt lớp vi tính (CT), chụp cộng hưởng từ (MRI) hay chụp cắt lớp phát xạ positron (PET/CT) nhưng chụp CT nói chung là phương pháp phổ biến do ảnh CT có độ phân giải không gian cao và độ tương phản tốt. Tuy nhiên, việc phân tích và chẩn đoán qua hình ảnh CT đòi hỏi phải có bác sĩ hoặc chuyên gia nhưng lại tốn nhiều thời gian.

Trong những năm gần đây với sự phát triển của công nghệ thông tin nói chung và trí tuệ nhân tạo (AI) nói riêng thì việc ứng dụng các mô hình máy học trong phân tích hình ảnh y khoa đang trở thành xu hướng quan trọng. Điều này cung cấp một công cụ hữu ích giúp cho các bác sĩ tiết kiệm được thời gian và hiệu quả hơn trong việc chẩn đoán ung thư phổi thông qua ảnh CT.

Từ các vấn đề nêu trên, đề tài "Xây dựng ứng dụng hỗ trợ chẩn đoán ung thư phổi" được đề xuất thực hiện. Đề tài tập trung vào việc sử dụng các mô hình máy học để phân

lớp ung thư phổi không tế bào nhỏ (NSCLC) từ ảnh CT bao gồm 3 loại ung thư biểu mô tuyến (Adenocarcinoma), ung thư biểu mô tế bào vảy (Squamous Cell Carcinoma) và ung thư biểu mô tế bào lớn (Large Cell Carcinoma). Tuy nhiên, kết quả này chỉ mang tính chất hỗ trợ các y bác sĩ trong việc chẩn đoán nhanh nên cần phải được kiểm nghiệm lại

1.2.Mục tiêu đề tài

Mục tiêu chính của đề tại là xây dựng được ứng dụng sử dụng mô hình học máy để chẩn đoán nguy cơ mắc ung thư phổi không tế bào nhỏ (NSCLC) dựa trên ảnh CT của bệnh nhân. Từ đó, giúp hỗ trợ bác sĩ và bệnh nhân trong việc đánh giá tình trạng sức khỏe cũng như có thể phát hiện sớm bệnh để kịp thời có những phương pháp điều trị hiệu quả.

1.3. Mô tả chi tiết đề tài

Ung thư phổi không tế bào nhỏ (NSCLC) chiếm phần lớn trong tổng số các ca ung thư phổi và có tỷ lệ tử vong cao nếu không được phát hiện kịp thời. Việc phân loại chính xác các loại NSCLC từ ảnh CT đóng một vai trò vô cùng quan trọng trong việc hỗ trợ các y bác sĩ đưa ra phương pháp điều trị hợp lý. Do đó, việc nghiên cứu và phát triển các mô hình máy học để có thể phân loại NSCLC đã trở thành một hướng đi tiềm năng, giúp nâng cao hiệu quả chẩn đoán và điều trị bệnh.

Đề tài "Xây dựng ứng dụng hỗ trợ chẩn đoán ung thư phổi" với mục tiêu hướng đến việc xây dựng các mô hình máy học có khả năng phân lớp ba loại ung thư chính trong NSCLC bao gồm ung thư biểu mô tuyến (Adenocarcinoma), ung thư biểu mô tế bào vảy (Squamous Cell Carcinoma), và ung thư biểu mô tế bào lớn (Large Cell Carcinoma**)** dựa trên ảnh CT. Giải pháp này sẽ giúp hỗ trợ cho các hệ thống y tế trong việc chẩn đoán chính xác hơn và giảm thời gian đánh giá hình ảnh cũng như tối ưu hóa hiệu quả điều trị cho bệnh nhân.

Để thực hiện đề tài, bài toán đã được xây dựng và huấn luyện mô hình mạng nơ-ron tích chập (**CNN**) gồm MobileNetV2, InceptionV3 và VGG16. Mục đích của các mô hình là trích xuất và phân tích các đặc trưng quan trọng từ ảnh CT để có thể phân lớp.

Quá trình huấn luyện được tiến hành trên tập dữ liệu train và kiểm tra hiệu quả mô hình trên tập dữ liệu test. Các độ đo được dùng để đánh giá mô hình bao gồm như accuracy, precision, recall, F1-score, support, AUC-ROC.

1.4. Hướng tiếp cận giải quyết của đề tài

Chuẩn bị dữ liệu:

- Thu thập dữ liệu: Sử dụng các tập dữ liệu Chest CT-Scan images Dataset[1].

- Tiền xử lý dữ liệu: chuẩn hóa kích thước hình ảnh về 224×224×3. Áp dụng các kỹ thuật tăng cường dữ liệu (data augmentation) như xoay ảnh ngẫu, dịch chuyển chiều ngang,

[1] https://www.kaggle.com/datasets/mohamedhanyyy/chest-ctscan-images/data

chiều dọc , biến dạng hình ảnh, phong to thu nhỏ ngẫu nhiên hình ảnh để tăng tính đa dạng của tập huấn luyện.

Xây dựng mô hình:

- Sử dụng các mô hình học sâu phổ biến trong xử lý ảnh:

+ VGG16: Một trong những mô hình CNN tiêu chuẩn, có cấu trúc đơn giản nhưng hiệu quả cao trong nhận diện ảnh.

+ InceptionV3: Một mô hình tiên tiến với các khối Inception giúp tối ưu hóa việc trích xuất đặc trưng.

+ MobileNetV2: Mô hình nhẹ, phù hợp với các ứng dụng triển khai trên thiết bị di động.

- Tích hợp thêm các khối Residual Block hoặc Squeeze-and-Excitation Block để cải thiện hiệu suất.

Huấn luyện mô hình:

- Chia dữ liệu thành các tập train 60%, validation 10% và test 30%.

- Cấu hình huấn luyện: hàm mất mát - categorical_crossentropy, tối ưu hóa: SGD hoặc Adam, sử dụng callback như EarlyStopping để tránh overfitting

- Theo dõi hiệu suất qua biểu đồ accuracy và loss.

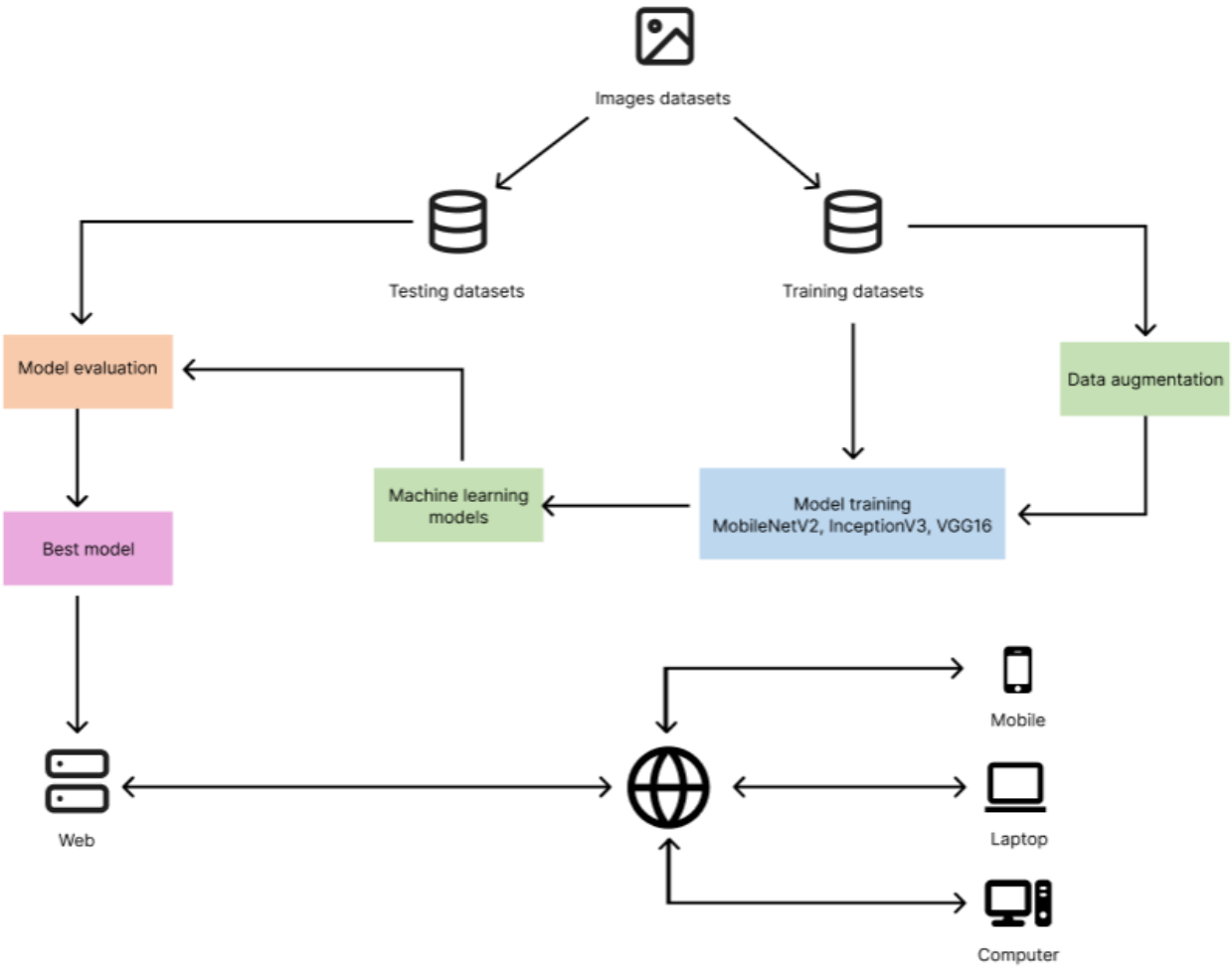
Đánh giá mô hình: Sau khi huấn luyện, các mô hình sẽ được đánh giá dựa trên: accuracy, precision, recall, F1-score, support, AUC-ROC để kiểm tra độ hiệu quả trong phân loại ung thư và từ đó chọn ra mô hình tốt nhất.

Triển khai ứng dụng: sau khi đã lựa chọn được mô hình tốt nhất sẽ được xây dựng trên nền tảng Web đơn giản với HTML, CSS, JavaScript và Flask.

CHƯƠNG 2: THIẾT KẾ VÀ CÀI ĐẶT GIẢI PHÁP

2.1. Kiến trúc tổng quát hệ thống

Trong phần này, sẽ trình bày về các các bước chính của hệ thống để có thể phân loại ung thư phổi không tế bào nhỏ. Quy trình thực hiện của mô hình được thể hiện như trong **Hình 2.5.** Đầu tiên, dữ liệu ảnh CT về bệnh ung thư phổi sẽ được thu thập. Sau đó, sẽ chia dữ liệu này thành hai tập là dữ liệu dùng để huấn luyện mô hình (training datasets) và dữ liệu dùng để kiểm thử mô hình sau huấn luyện (testing datasets). Tiếp theo, sẽ áp dụng các kỹ thuật tăng cường dữ liệu trên tập train như xoay ảnh ngẫu, dịch chuyển chiều ngang, chiều dọc , biến dạng hình ảnh, phong to thu nhỏ ngẫu nhiên hình ảnh. Mục đích của việc tăng cường này là làm cho tập dữ liệu trở nên phong phú. Kế tiếp sẽ huấn luyện mô hình và sau đó là đánh giá mô hình để chọn ra mô hình tốt nhất. Cuối cùng là triển khai mô hình lên ứng dụng web để hỗ trợ chẩn đoán ung thư phổi.



Hình 2.5: Kiến trúc tổng quát của hệ thống

Dữ liệu đóng vai trò rất quan trọng trong quá trình huấn luyện mô hình máy học để nhận dạng hình ảnh. Để xây dựng được mô hình có độ hiệu quả cao cần quan tâm đến tập dữ liệu có đa dạng và phong phú hay không. Để làm được điều đó thì quá trình thu thập dữ liệu thường gặp nhiều khó khăn, tốn thời gian và sức lực để sưu tầm cũng như phân chia dữ liệu cho hợp lý. Sau khi thu thập và xử lý tập dữ liệu, sẽ chia thành 3 phần với tỉ lệ 60%

train, 30% test và 10% validation. Để đánh giá chính xác hiệu quả của mô hình, nhóm đã tìm hiểu và nghiên cứu những mô hình thường dùng cho nhận diện và phân loại hình ảnh, từ đó chọn ra ba mô hình tiêu biểu là MobileNetV2, InceptionV3, VGG16. Với các thông số khác nhau từ những mô hình, ta sẽ dùng nó để huấn luyện và từ đó chọn ra mô hình tốt nhất. Sau đó sẽ tích hợp mô hình vào một ứng dụng web hỗ trợ chuẩn đoán ung thư phổi để nhận dạng những hình ảnh do người dùng cung cấp để chuẩn đoán ảnh đó thuộc loại ung thư gì.

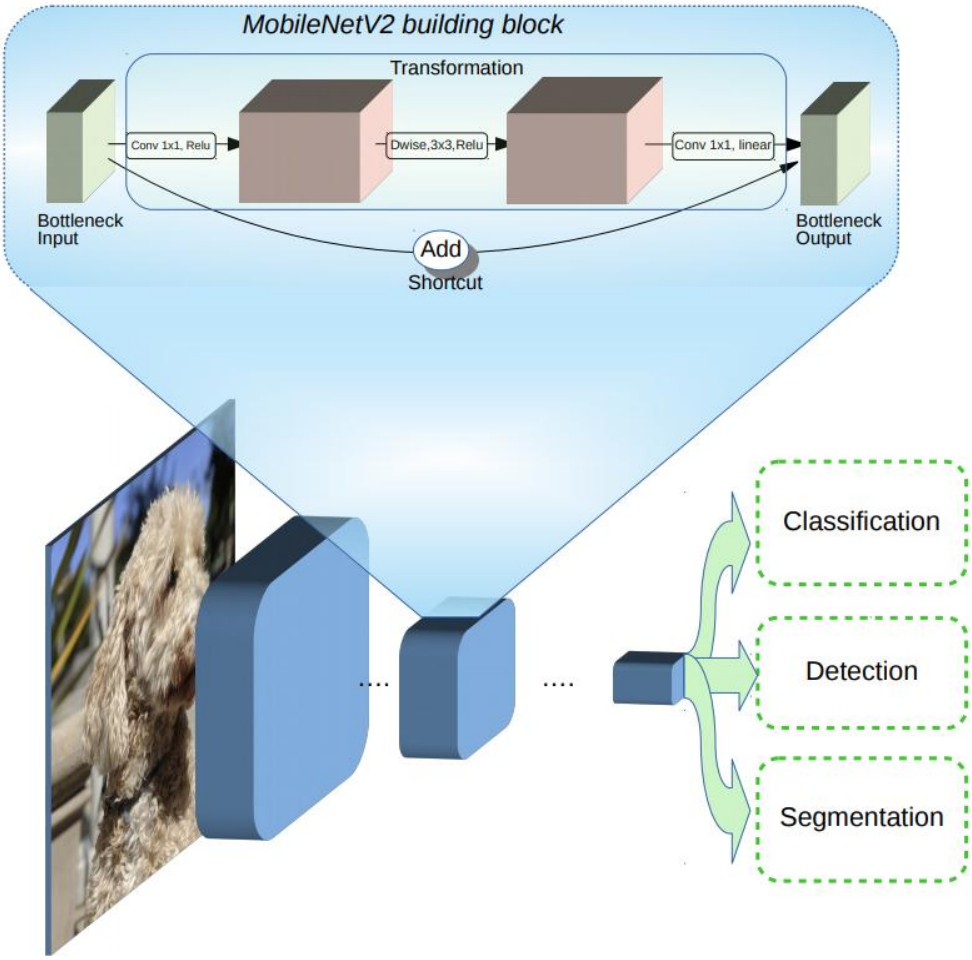
2.2.Xây dựng các mô hình

2.2.1. Mô hình MobileNetV2

MobileNetV2 là một kiến trúc mạng nơ-ron tích chập (CNN) được thiết kế đặc biệt để hoạt động hiệu quả trên các thiết bị di động. Điểm nổi bật của MobileNetV2 là sử dụng cấu trúc "Inverted Residual Block" với các kết nối tắt (shortcut connections) giữa các lớp bottleneck mỏng. Các lớp mở rộng trung gian sử dụng phép tích chập chiều sâu (depthwise convolution) nhẹ nhàng để lọc đặc trưng, tạo ra tính phi tuyến cần thiết. Kiến trúc của MobileNetV2 bao gồm một lớp tích chập ban đầu với 32 bộ lọc, tiếp theo là 19 lớp residual bottleneck. [6]

Mô hình này thường được sử dụng để giải quyết những bài toán phân loại hình ảnh, phát hiện các đối tượng trong ảnh hoặc video, nhận diện khuôn mặt trong hình ảnh, nhận diện văn bản,…

Hình 2.6 là một ví dụ minh họa cho kiến trúc của mô hình MobileNetV2.



Hình 2.6: Minh họa mô hình MobileNetV2[2]

Trong đề tài này, mô hình MobileNetV2 được xây dựng với kiến trúc như sau:

- Input Layer: Nhận ảnh đầu vào kích thước 224×224×3.

- Conv1: Tích chập thông thường, gồm 32 bộ lọc, làm kích thước ảnh giảm còn (112,112,32).

- Inverted Residual Blocks: Bao gồm các khối mở rộng (expand), tích chập chiều sâu (depthwise convolution), và thu nhỏ (project). Các khối này được lặp lại nhiều lần với số lượng kênh và kích thước giảm dần, sử dụng Batch Normalization và hàm kích hoạt ReLU.

- Các khối residual: Kết nối tắt (shortcut connections) giữa các lớp để cải thiện hiệu suất huấn luyện.

- Conv cuối cùng: Một lớp Conv2D cuối cùng với 1280 kênh, sau đó là Batch Normalization và ReLU.

- Global Average Pooling: Chuyển đổi đặc trưng từ dạng tensor về dạng vector.

---

[2]https://research.google/blog/mobilenetv2-the-next-generation-of-on-device-computer-vision-networks/

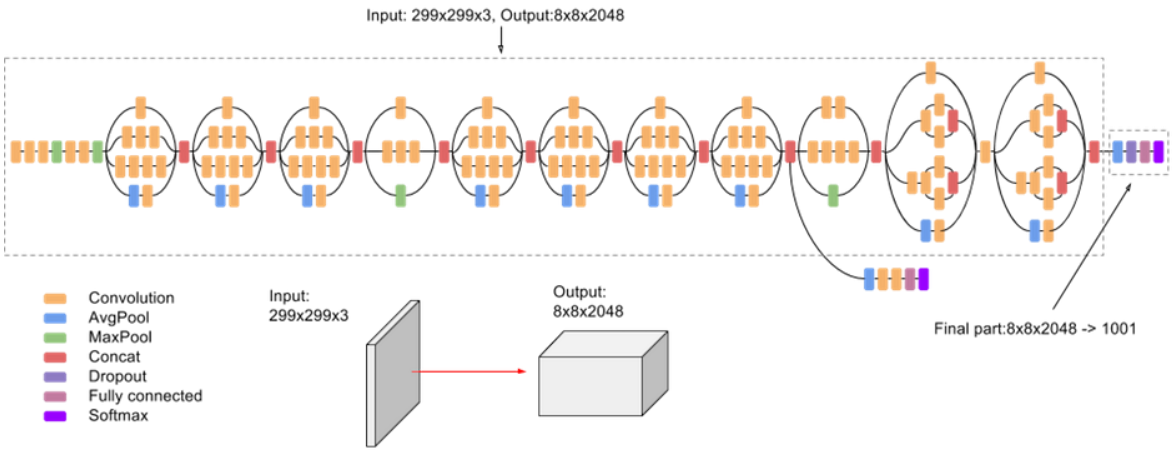
- Các lớp Dense (Fully Connected): Bao gồm các lớp Dense có số lượng neuron lần lượt là 1024, 512 và cuối cùng là 4 neuron tương ứng với số lớp phân loại đầu ra.

2.2.2. Mô hình InceptionV3

InceptionV3 là một kiến trúc mạng nơ-ron tích chập (CNN) được phát triển nhằm cải thiện hiệu quả tính toán và độ chính xác trong các tác vụ phân loại ảnh. Được giới thiệu lần đầu trong bài báo "Rethinking the Inception Architecture for Computer Vision" (2015), InceptionV3 nổi bật với việc sử dụng các kỹ thuật tối ưu như tích chập phân tách (factorized convolutions), tích chập bất đối xứng (asymmetric convolutions), và bộ phân loại phụ trợ (auxiliary classifier) giúp truyền tải thông tin nhãn sâu hơn vào mạng, kết hợp với chuẩn hóa theo batch (batch normalization) để tăng hiệu quả huấn luyện. [7]

InceptionV3 là một trong những mô hình rất phổ biến được sử dụng nhiều trong lĩnh vực học sâu (Deep Learning). Mô hình này được tạo ra nhằm đáp ứng các vấn đề về phân loại hình ảnh, nhận diện đối tượng trong ảnh hoặc video theo thời gian thực, các tác vụ thị giác máy tính phức tạp. Một trong những điểm mạnh của mô hình InceptionV3 là nó có thể tối ưu hóa về mặt hiệu suất tính toán mà không làm giảm đi độ chính xác.

Hình 2.7 là một ví dụ minh họa cho kiến trúc của mô hình InceptionV3.



Hình 2.7: Minh họa mô hình InceptionV3[3]

Trong đề tài này, mô hình InceptionV3 được xây dựng với kiến trúc như sau:

- Input: Ảnh đầu vào kích thước (224, 224, 3).

- Base Model (InceptionV3): Sử dụng kiến trúc InceptionV3 có sẵn từ thư viện Keras, loại bỏ phần phân loại cuối cùng (include_top=False), sử dụng trọng số huấn luyện trước trên ImageNet.

- GlobalAveragePooling2D: Chuyển đổi tensor đặc trưng thành vector đặc trưng.

- Các lớp Dense tùy chỉnh:

---

[3] https://paperswithcode.com/method/inception-v3

+ Dense(1024) với hàm kích hoạt ReLU.

+ Dense(512) với hàm kích hoạt ReLU.

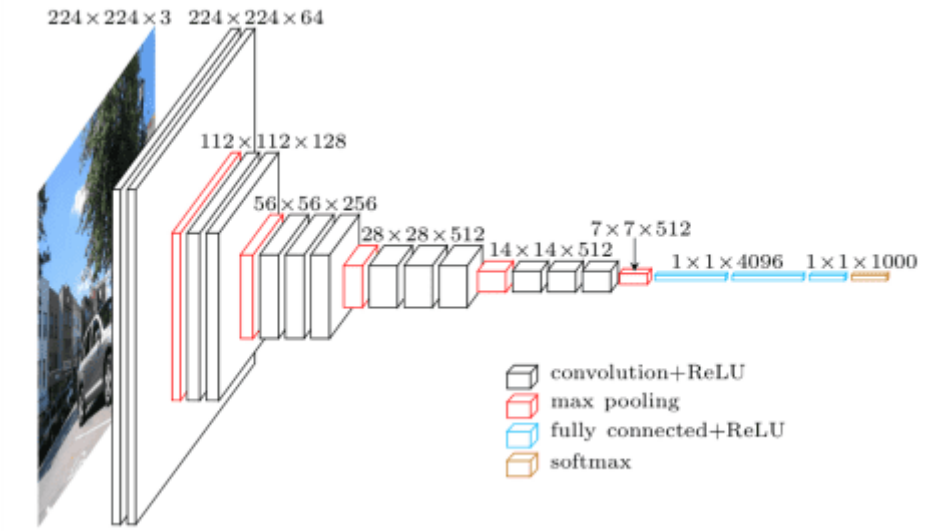
+ Dense(4) với hàm kích hoạt softmax để phân loại ảnh thành 4 lớp.

2.2.3. Mô hình VGG16

VGG16 là một kiến trúc mạng nơ-ron tích chập (CNN) nổi tiếng, được phát triển bởi nhóm Visual Geometry Group (VGG) tại Đại học Oxford. Với 16 lớp có trọng số, VGG16 nổi bật nhờ thiết kế đơn giản nhưng hiệu quả, sử dụng các bộ lọc kích thước nhỏ (3×3) và các lớp pooling để giảm độ phức tạp tính toán. Kiến trúc này đã đạt độ chính xác cao trong các tác vụ phân loại ảnh, đặc biệt trên bộ dữ liệu ImageNet với độ chính xác top-5 lên đến 92.7%. VGG16 cũng được ứng dụng rộng rãi trong học chuyển tiếp (transfer learning), giúp giải quyết nhiều bài toán thị giác máy tính như phân loại ảnh, nhận dạng đối tượng, và trích xuất đặc trưng. [4]

Đây là môt kiến trúc học sâu phổ biến được sử dụng trong các bài toán nhận diện hình ảnh, nhận diện khuôn mặt và các tác vụ thị giác máy tính phức tạp.

Hình 2.8 là một ví dụ minh họa cho kiến trúc của mô hình VGG16.



Hình 2.8: Minh họa mô hình VGG16[5]

Trong đề tài này, mô hình VGG16 được xây dựng với kiến trúc như sau:

- Input: Ảnh đầu vào kích thước (224, 224, 3).

- Base Model (VGG16): Sử dụng mô hình VGG16 đã được huấn luyện trước trên ImageNet, loại bỏ phần fully connected cuối cùng (include_top=False).

---

[4] https://blog.arunangshudas.com/vgg-architecture/

[5]https://viblo.asia/p/mo-phong-he-truy-van-hinh-anh-voi-machine-learning-semantic-search-phan-1-L4x5xwpalBM

- Residual Blocks: Hai khối residual với các lớp: Conv2D → BatchNormalization → ReLU → Conv2D → BatchNormalization → Squeeze-and-Excitation Block → Add → ReLU.

- Batch Normalization: Áp dụng sau các lớp residual.

- Flatten: Chuyển đổi tensor thành vector.

- Dense Layers:

  + Dense(1024) với hàm kích hoạt ReLU.

  + Dense(512) với hàm kích hoạt ReLU.

  + Dense(4) với hàm kích hoạt softmax để phân loại thành 4 lớp.

2.3.Giải pháp cài đặt

Ứng dụng được triển khai bằng ngôn ngữ lập trình Python với các thư viện hỗ trợ như sau:

- TensorFlow và Keras: xây dựng, huấn luyện và triển khai các mô hình học sâu để phân loại ung thư phổi không tế bào nhỏ qua ảnh CT.

- OpenCV: xử lý và tiền xử lý ảnh CT trước khi đưa vào mô hình.

- NumPy, Pandas: xử lý dữ liệu đầu vào và đầu ra của mô hình.

Flask là một micro web framework được viết bởi Python. Web framework là một khuôn khổ phần mềm được thiết kế để hỗ trợ phát triển các ứng dụng web bao gồm các dịch vụ web, tài nguyên web và các API web, cung cấp các chức năng cần thiết để tạo ra một ứng dụng website. Flask được phân loại là micro framework vì nó không yêu cầu các công cụ hoặc thư viện cụ thể, cung cấp một lõi chức năng đơn giản nhưng có thể mở rộng. Flask không có lớp trừu tượng cơ sở dữ liệu, xác thực biểu mẫu hoặc bất kỳ thành phần nào khác mà các thư viện bên thứ ba đã có từ trước cung cấp các chức năng phổ biến. Tuy nhiên, Flask hỗ trợ các tiện ích mở rộng có thể thêm các tính năng ứng dụng như thể chúng được triển khai trong chính Flask. Các tiện ích mở rộng tồn tại cho người lập bản đồ quan hệ đối tượng, xác thực biểu mẫu, xử lý tải lên, các công nghệ xác thực mở khác nhau và một số công cụ liên quan đến khuôn khổ chung. Đề tài sử dụng Flask như một framework chính để phát triển web service khai thác mô hình đề xuất đã huấn luyện.

Giao diện của web của ứng dụng được xây dựng bằng các công nghệ HTML, CSS, JavaScript.

CHƯƠNG 3: KIỂM THỬ VÀ ĐÁNH GIÁ

3.1. Mục tiêu kiểm thử

Mục tiêu của việc kiểm thử là đánh giá hiệu quả của các mô hình máy học trong việc hỗ trợ chẩn đoán ung thư phổi, đồng thời kiểm tra khả năng hoạt động của giao diện web trong việc nhận diện ảnh. Trong phần này sẽ thực hiện kiểm thử các vấn đề sau:

- Kiểm thử tính chính xác của các mô hình phân lớp trong việc nhận dạng ung phổi.
- Kiểm thử hoạt động của giao diện web trong việc nhận diện.

3.2. Kịch bản kiểm thử các mô hình

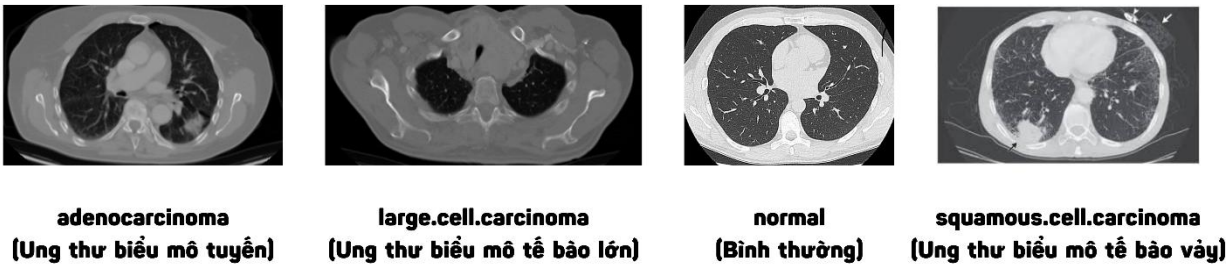
3.2.1. Mô tả tập dữ liệu

Trong đề tài này, việc nghiên cứu và huấn luyện mô hình được thực hiện trên tập dữ liệu **Chest CT-Scan images**[6], bao gồm 1000 ảnh ảnh CT ngực. Tất cả ảnh đều thuộc định dạng PNG, trong đó có 338 ảnh ở lớp adenocarcinoma (Ung thư biểu mô tuyến), 187 ảnh ở lớp large.cell.carcinoma (Ung thư biểu mô tế bào lớn), 215 ảnh ở lớp normal (Bình thường) và 260 ảnh ở lớp squamous.cell.carcinoma (Ung thư biểu mô tế bào vảy) được thể hiện như ở trong **Bảng 2.1.** Tập ảnh bao gồm nhiều hình ảnh có kích thước khác nhau nên đòi hỏi việc thay đổi kích thước ảnh về cùng cỡ trước khi đưa vào mô hình huấn luyện. Mẫu hình ảnh của các lớp được minh họa trong **Hình 2.9**.

Bảng 2.1: Số lượng samples ở mỗi lớp của tập dữ liệu Chest CT-Scan images

Class (Lớp)	Samples (Số lượng ảnh)
adenocarcinoma (Ung thư biểu mô tuyến)	338
large.cell.carcinoma (Ung thư biểu mô tế bào lớn)	187
normal (Bình thường)	215
squamous.cell.carcinoma (Ung thư biểu mô tế bào vảy)	260
Total (Tổng)	1000

[6] Chest CT-Scan images Dataset



Hình 2.9: Samples của các lớp trong tập dữ liệu

3.2.2. Môi trường thực nghiệm

Nghiên cứu được thực hiện dựa trên nền tảng ngôn ngữ lập trình Python sử dụng các thư viện mã nguồn mở hỗ trợ cho các mô hình machine learning (máy học) như: Tensorflow Keras, OpenCV, NumPy, Pandas cho mạng nơ-ron hỗ trợ chạy mô hình trên cả CPU và GPU. Đề tài được thực nghiệm trên 2 môi trường khác nhau, cụ thể là:

- Quá trình xây dựng kiến trúc của các mô hình, đồng thời thực hiện đánh giá chọn lọc mô hình tốt nhất được thực hiện trên Google Colab (Google Colaboratory), một sản phẩm của dự án Google Research với mục đích hỗ trợ chạy code Python trực tiếp thông qua trình duyệt web, phù hợp cho việc nghiên cứu máy học, phân tích dữ liệu và giáo dục.

- Nhiệm vụ huấn luyện mô hình để sử dụng trong hệ thống web cũng như việc xây dựng website chẩn đoán bệnh được thực hiện trên Windows 11 Home Single Language, CPU Intel(R) Core(TM) i5-1155G7, RAM: 16Gb, với điều kiện máy đã được cài đặt Python và các thư viện hỗ trợ.

3.2.3. Cơ sở đánh giá

Đề tài sử dụng các độ đo để đánh giá hiệu quả huấn luyện của mô hình là accuracy, precision, recall, F1-score, support và AUC-ROC.

- Accuracy - Độ chính xác: là tỷ lệ các dự đoán đúng so với tổng số mẫu. Được định nghĩa theo công thức sau:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{10}$$

- Recall - Độ nhạy là tỷ lệ số lượng mẫu mô hình phân loại đúng (True Positive) so với tổng số mẫu thực sự (True Positive + False Negative). Được định nghĩa theo công thức:

$$Recall = \frac{TP}{TP + FN} \tag{11}$$

- Precision - Độ chính xác: Là tỷ lệ số lượng mẫu được mô hình dự đoán đúng với thực tế(True Positive) so với tổng số mẫu mà mô hình dự đoán (True Positive + False Positive). Được xác định qua công thức:

$$Precision = \frac{TP}{TP + FP} \tag{12}$$

- F1 - score là chỉ số trung hòa giữa độ nhạy **Recall** và độ chính xác **Precision**. Được xác định theo công thức:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (13)$$

Trong đó :

+ **TP (True Positive)**: dự đoán đúng khi nhãn thực tế là dương

+ **TN (True Negative)**: dự đoán đúng khi nhãn thực tế là âm

+ **FP (False Positive)**: dự đoán sai khi nhãn thực tế là âm nhưng dự đoán là dương.

+ **FN (False Negative)**: dự đoán sai khi nhãn thực tế là dương nhưng dự đoán là âm.

- Ma trận nhầm lẫn (Confusion Matrix): là một bảng cho thấy số lượng dự đoán đúng và sai của mô hình đối với từng lớp.

Khi tỷ lệ giữa số lượng thuộc các lớp trong tập dữ liệu thay đổi trong một bộ thử nghiệm, một mô hình có thể không còn thực hiện tối ưu, hoặc thậm chí khó có thể chấp nhận được. Tuy nhiên, đường cong ROC (Receiver Operating Characteristic curve, Đường cong ROC – AUC (**Hình 2.10**) là bất biến đối với những thay đổi trong phân phối lớp. Hay nói cách khác - diện tích dưới đường cong ROC - AUC cũng bất biến khi có những thay đổi trong phân phối lớp. Có thể nói AUC là thước đo để đánh giá mô hình tốt hơn ACC. Vì thế chúng ta sử dụng thêm độ đo AUC để có thể đánh giá, trực quan hóa và so sánh với ACC.

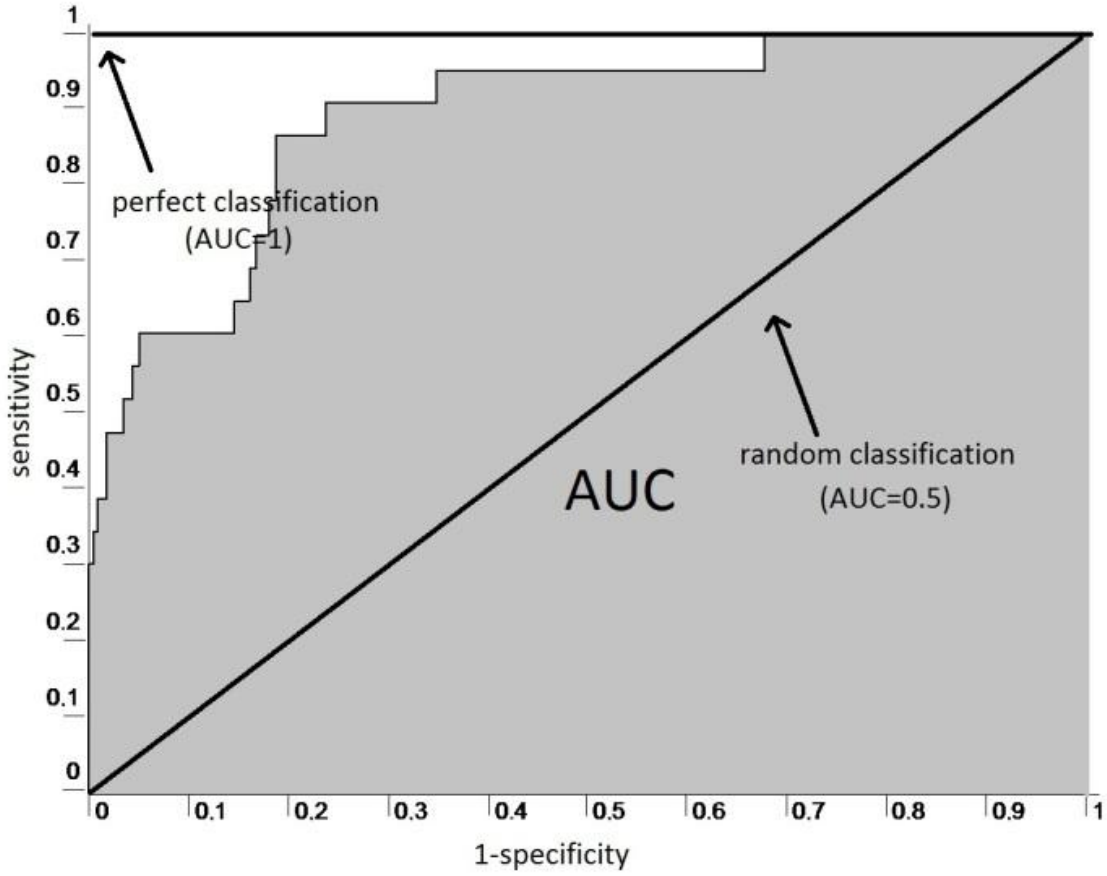
Để tính giá trị AUC, trước hết, ta tính 2 độ đo khác từ Confusion matrix trên là TPR và FPR.

$$True\ Positive\ Rate - TPR = \frac{TP}{TP + FN} \quad (14)$$

Tỷ lệ giữa số lượng dự đoán đúng chính xác với tổng số dự đoán. Có thể thấy TPR càng cao thì dự đoán người có bệnh càng chính xác.

$$False\ Positive\ Rate - FPR = \frac{FP}{FB + TN} \quad (15)$$

Tỷ lệ giữa số lượng dự đoán sai người không có bệnh thành có bệnh với tổng số dự đoán cho người không có bệnh. Có thể thấy FPR càng cao thì dự đoán người không có bệnh phân loại càng sai

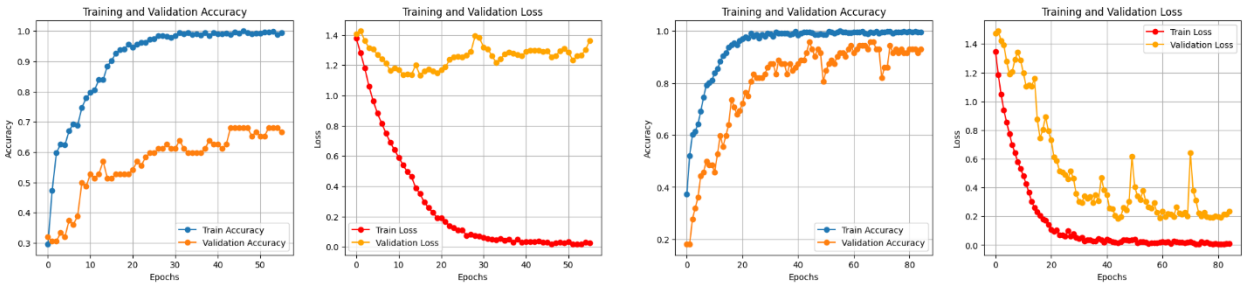


Hình 2.10: Đường cong ROC – AUC

3.2.4. Kết quả thực nghiệm

Để đánh giá hiệu suất của các mô hình phân lớp ung thư phổi không tế bào nhỏ, tập dữ liệu được chia thành ba phần với tỷ lệ 60% cho train, 10% cho validation và 30% cho test. Qua quá trình huấn luyện và đánh giá các mô hình thu được kết quả như sau:

- Biểu đồ Accuracy và Loss trong quá trình huấn luyện ở **Hình 2.11** cho thấy ban đầu Loss trên cả tập Train lẫn Validation đều cao và độ chính xác thấp. Điều này cho thấy mô hình chưa học được gì đáng kể. Giữa quá trình huấn luyện, độ chính xác (Accuracy) trên tập Train và Validation tăng dần. Loss trên cả hai tập giảm đều cho thấy mô hình đang học tốt từ dữ liệu. Cuối quá trình huấn luyện, Train Accuracy cao hơn Val Accuracy, Val Loss dao động có thể là dấu hiệu mô hình đạt đến mức bão hòa hoặc bị nhiễu trong dữ liệu kiểm định.



a. MobileNetV2 b. InceptionV3

c. VGG16

Hình 2.11: Biểu đồ Accuracy và Loss của các mô hình

Từ **Hình 2.11** có thể thấy:

+ Ở mô hình MobileNetV2, Accuracy trên tập train đạt gần 100% khá sớm nhưng độ chính xác trên tập validation chỉ đạt khoảng 60%. Điều này thể hiện rõ ràng mô hình đang bị overfitting. Loss trên tập train giảm mạnh nhưng loss trên tập validation dao động lớn và không giảm rõ ràng.

+ Ở mô hình InceptionV3, Accuracy ổn định và khá tốt, đạt khoảng 90% trên tập validation, chênh lệch với tập train không lớn, thể hiện khả năng tổng quát tốt hơn. Loss trên cả tập train và validation đều giảm tốt nhưng vẫn còn dao động nhất định ở tập validation.

+ Ở mô hình VGG16, Accuracy rất tốt và ổn định, đạt trên 90% và tiến tới 96% ở tập validation. Loss của VGG16 giảm nhanh và ổn định, ít dao động, cho thấy khả năng học tập ổn định nhất trong ba mô hình.

- Dưới đây là báo cáo đánh giá classification report (Precision, Recall, F1-score) trên tập test của 3 mô hình thể hiện ở **Hình 2.12**. Các lớp 0, 1, 2, 3 tương đương với các phân lớp adenocarcinoma (ung thư biểu mô tuyến), large.cell.carcinoma(ung thư biểu mô tế bào lớn), normal (bình thường) và squamous.cell.carcinoma(ung thư biểu mô tế bào vảy).

	precision	recall	f1-score	support
0	0.69	0.87	0.77	120
1	0.48	0.98	0.65	51
2	1.00	0.98	0.99	54
3	1.00	0.08	0.14	90
accuracy			0.68	315
macro avg	0.79	0.73	0.64	315
weighted avg	0.80	0.68	0.61	315

a. MobileNetV2

	precision	recall	f1-score	support
0	0.89	0.92	0.91	120
1	0.86	0.96	0.91	51
2	1.00	0.98	0.99	54
3	0.94	0.86	0.90	90
accuracy			0.92	315
macro avg	0.92	0.93	0.92	315
weighted avg	0.92	0.92	0.92	315

b. InceptionV3

	precision	recall	f1-score	support
0	0.94	0.95	0.95	120
1	0.89	0.98	0.93	51
2	1.00	0.98	0.99	54
3	0.99	0.93	0.96	90
accuracy			0.96	315
macro avg	0.96	0.96	0.96	315
weighted avg	0.96	0.96	0.96	315

c. VGG16

Hình 2.12: Báo cáo đánh giá (Classification report) của các mô hình

Từ **Hình 2.12**, có thể thấy được rằng:

+ Ở mô hình MobileNetV2: Precision và recall thấp nhất là ở lớp 3 - squamous.cell.carcinoma (ung thư biểu mô tế bào vảy) recall cực kỳ thấp cụ thể là 0.08, f1-score chỉ đạt 0.14, mô hình gần như không nhận diện được lớp này. Accuracy chung chỉ 68%.

+ Ở mô hình InceptionV3: Kết quả tốt hơn nhiều so với MobileNetV2, precision, recall và f1-score đều từ 0.86 trở lên. Accuracy chung là 92%, khá cao và cân bằng tốt giữa các lớp.

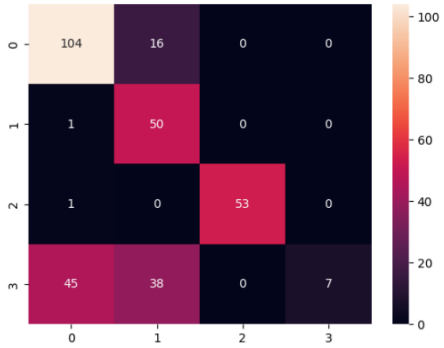
+ Ở mô hình VGG16: Cho kết quả rất cao với precision, recall, f1-score đạt gần như tuyệt đối, thấp nhất là 0.89 và hầu hết các lớp đều xấp xỉ 1.00. Accuracy tổng thể lên tới 96%, cao nhất trong ba mô hình.

- **Hình 2.13** bên dưới thể hiện ma trận nhầm lẫn của 3 mô hình. Từ đó có thể đưa ra một số nhận xét như sau:

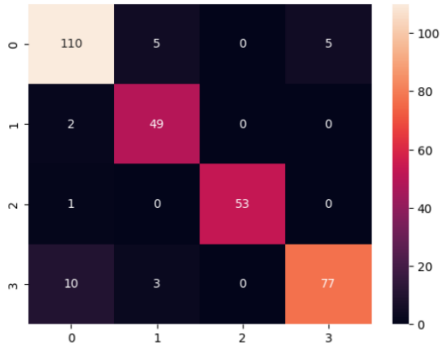
+ Ở mô hình MobileNetV2: Ma trận nhầm lẫn cho thấy lớp 3 - squamous.cell.carcinoma (ung thư biểu mô tế bào vảy) bị nhận dạng sai hầu hết thành lớp 0 - adenocarcinoma (ung thư biểu mô tuyến) và 1- large.cell.carcinoma(ung thư biểu mô tế bào lớn), rất kém hiệu quả.

+ Ở mô hình InceptionV3: Có độ chính xác khá cao và nhầm lẫn ít hơn. Chủ yếu xảy ra ở lớp 0 - adenocarcinoma (ung thư biểu mô tuyến) và lớp 3 - squamous.cell.carcinoma(ung thư biểu mô tế bào vảy) nhưng ít hơn đáng kể so với mô hình MobileNetV2.

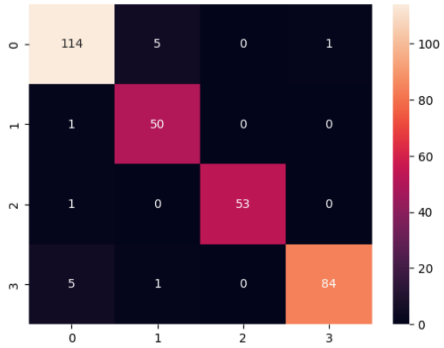
+ Ở mô hình VGG16: Có ít nhầm lẫn nhất với rất ít sai sót. Ma trận nhầm lẫn gần như chéo thể hiện độ chính xác và khả năng nhận dạng chính xác nhất trong 3 mô hình.



a. MobileNetV2



b. InceptionV3



c. VGG16

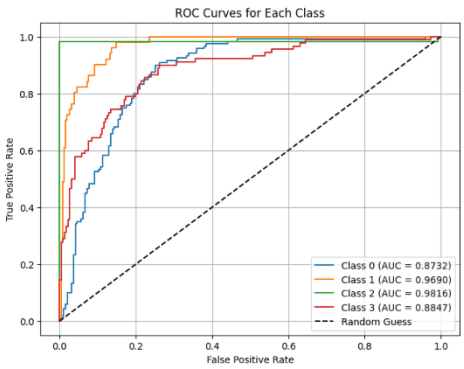
Hình 2.13: Ma trận nhầm lẫn (Confusion Matrix) của các mô hình

- **Hình 2.14** bên dưới là đường cong ROC và chỉ số AUC của 3 mô hình.

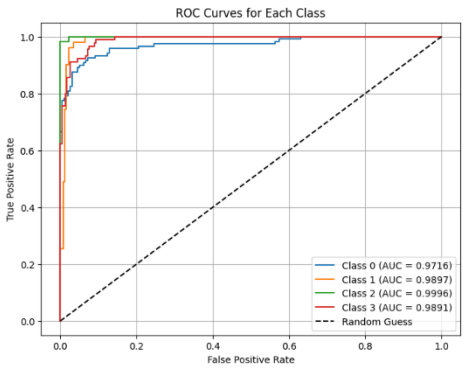
+ Ở mô hình MobileNetV2: AUC khá thấp so với hai mô hình kia, lớp 3 - squamous.cell.carcinoma (ung thư biểu mô tế bào vảy) AUC chỉ khoảng 0.88 cho thấy khả năng phân biệt thấp ở lớp này.

+ Ở mô hình InceptionV3: Có AUC rất cao, các lớp đều đạt gần hoặc trên 0.97 cho thấy khả năng phân biệt các lớp tốt.

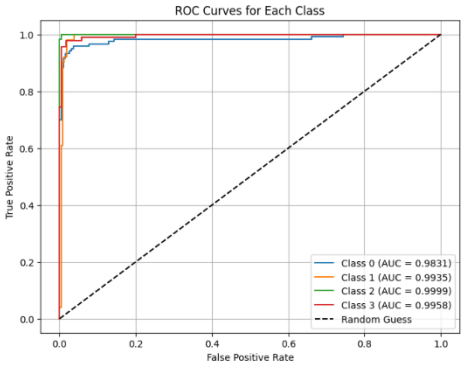
+ Ở mô hình VGG16: Có AUC cao nhất hầu hết xấp xỉ 1.00 chứng minh khả năng phân loại cực kỳ tốt.



a. MobileNetV2



b. InceptionV3



c. VGG16

Hình 2.14: Đường cong ROC và chỉ số AUC của các mô hình

- Ba mô hình MobileNetV2, InceptionV3 và VGG16 được huấn luyện bằng cách sử dụng lựa chọn giá trị batch_size = 64 và số vòng lặp (epoch) tăng dần ảnh hưởng đến sự thay đổi tốc độ học và đồng thời cũng ảnh hưởng đến tỉ lệ lỗi, độ chính xác, thời gian huấn luyện của mô hình. Kết quả thực nghiệm cho thấy giá trị batch_size = 64 và số vòng lặp (epoch) = 100 (có sử dụng EarlyStopping với patience=40 để dừng huấn luyện khi mô hình không cải thiện) cho kết quả cao nhất là xấp xỉ 96%. **Bảng 2.2** mô tả kết quả huấn luyện và kiểm tra.

Bảng 2.2: So sánh độ chính xác giữa ba mô hình

Mô hình	**Batch_size**	**Epoch**	**Epoch thực tế**	**Số lượng mẫu**		**ACC (%)**	**ROC Area (AUC)**
				Huấn luyện	**Kiểm tra**		
MobileNetV2	64	100	56	685	315	67.94	0.9271
InceptionV3	64	100	85	685	315	91.75	0.9875
VGG16	64	100	80	685	315	95.56	0.9931

Kết luận: Dựa trên tất cả các cơ sở đánh giá ở trên, có thể xếp hạng hiệu quả các mô hình như sau:

+ VGG16**:** Tốt nhất, phù hợp nhất để ứng dụng vào bài toán thực tế.

+ InceptionV3: Hiệu suất tốt, có thể dùng trong trường hợp cần mô hình nhẹ hơn so với VGG16.

+ MobileNetV2: Kém hiệu quả rõ rệt không nên dùng nếu không có yêu cầu đặc biệt về tốc độ và kích thước mô hình nhỏ gọn.

Từ đó kết luận đó, VGG16 và InceptionV3 sẽ được được chọn làm mô hình chính cho bài toán này và triển khai trên giao diện web ở phần kế tiếp.

3.3.Kịch bản kiểm thử giao diện web

Dưới đây sẽ là một số kịch bản kiểm thử giao diện web của ứng dụng đã xây dựng:

3.3.1. Kiểm thử hiển thị giao diện ban đầu

Bước thực hiện: Truy cập vào giao diện web.

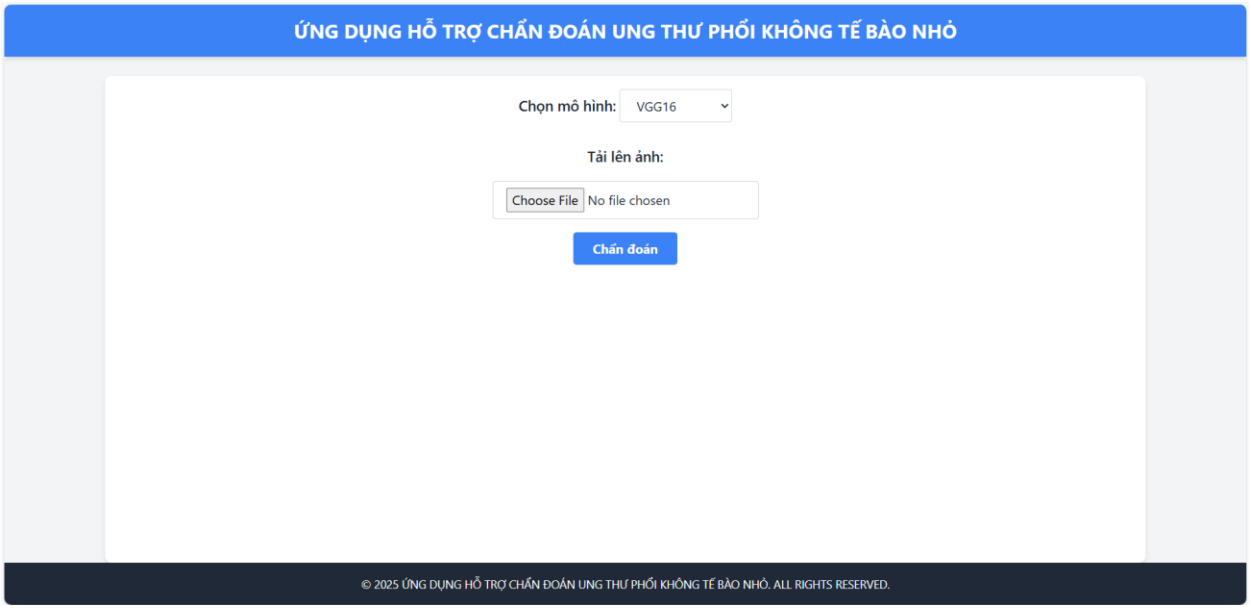
Kết quả mong đợi được thể hiện như ở **Hình 2.15**:

- Tiêu đề "ỨNG DỤNG HỖ TRỢ CHẨN ĐOÁN UNG THƯ PHỔI KHÔNG TẾ BÀO NHỎ" được hiển thị rõ ràng, đúng chính tả.

- Dropdown "Chọn mô hình" hiển thị mặc định là mô hình VGG16.

- Nút "Choose File" hiển thị trạng thái mặc định "No file chosen".

- Nút "Chẩn đoán" hiển thị rõ ràng và trạng thái clickable.



Hình 2.15: Giao diện ban đầu

3.3.2. Kiểm thử chức năng chọn mô hình

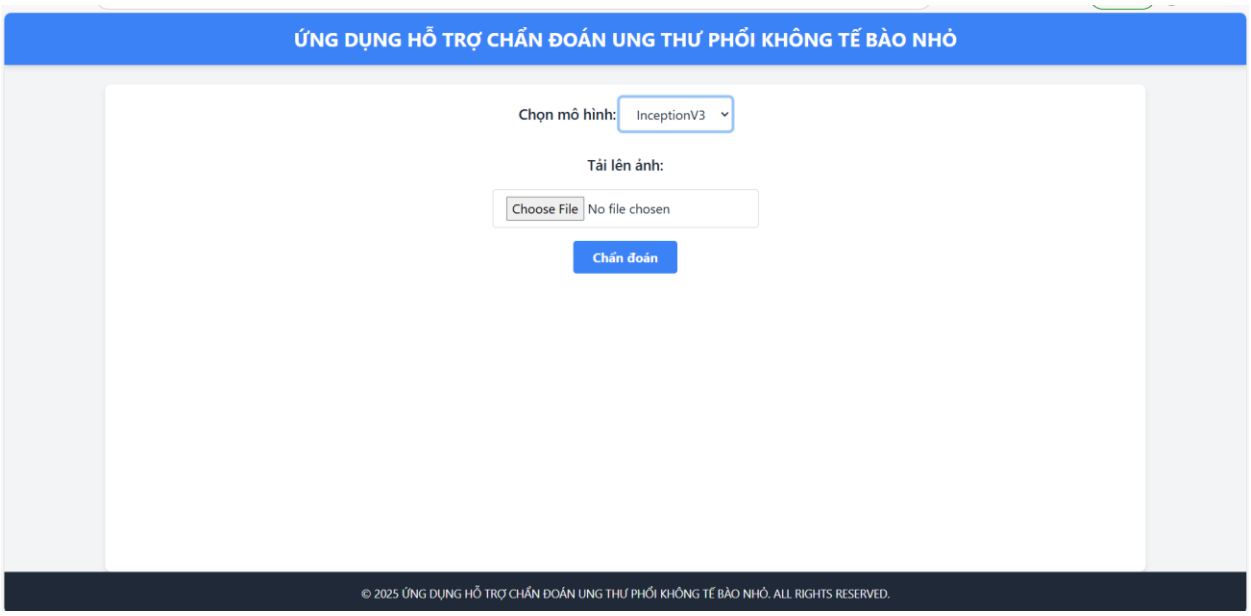
Bước thực hiện: Chọn vào dropdown "Chọn mô hình" sau đó chọn từng mô hình

Kết quả mong đợi**:**

- Dropdown hiển thị đầy đủ các mô hình cho phép lựa chọn như ở **Hình 2.16**.

- Khi chọn mô hình, giá trị hiển thị trên dropdown thay đổi đúng theo lựa chọn như ở **Hình 2.17**.

Hình 2.16: Giao diện khi chọn Dropdown



Hình 2.17: Giao diện khi chọn mô hình khác

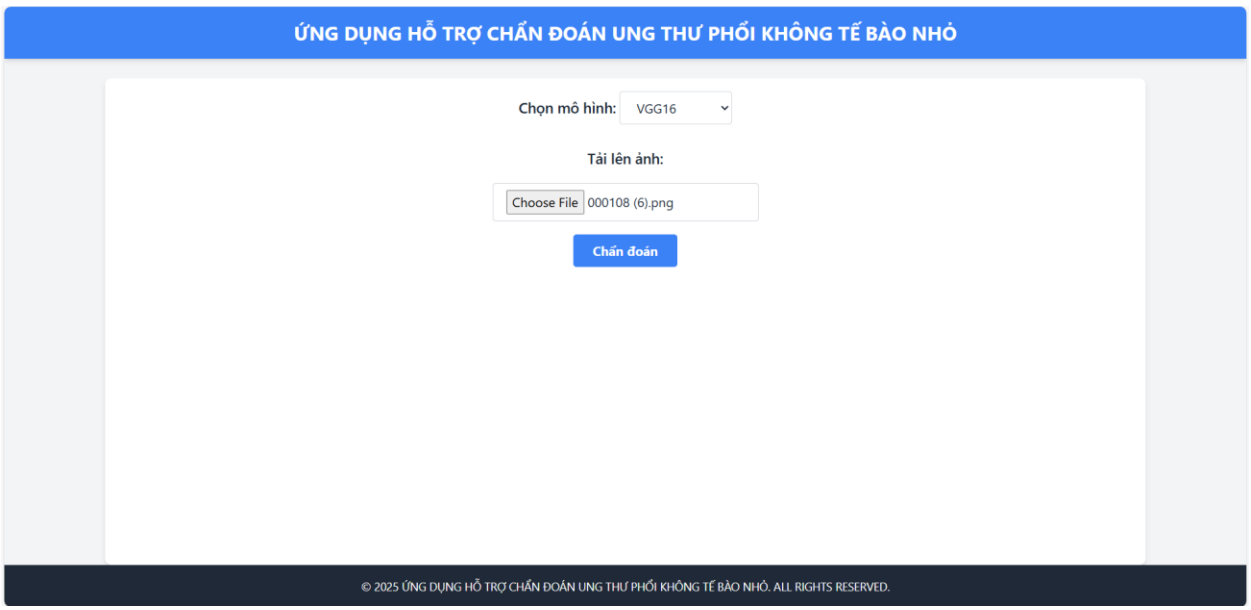
3.3.3. Kiểm thử tải lên file ảnh hợp lệ

Bước thực hiện: Nhấn nút "Choose File" và chọn một file ảnh hợp lệ (định dạng jpg, jpeg, png).

Kết quả mong đợi thể hiện như ở **Hình 2.18**:

- Tên file được hiển thị ngay cạnh nút "Choose File".

- Không có lỗi xảy ra khi upload ảnh.



Hình 2.18: Giao diện khi tải lên một ảnh

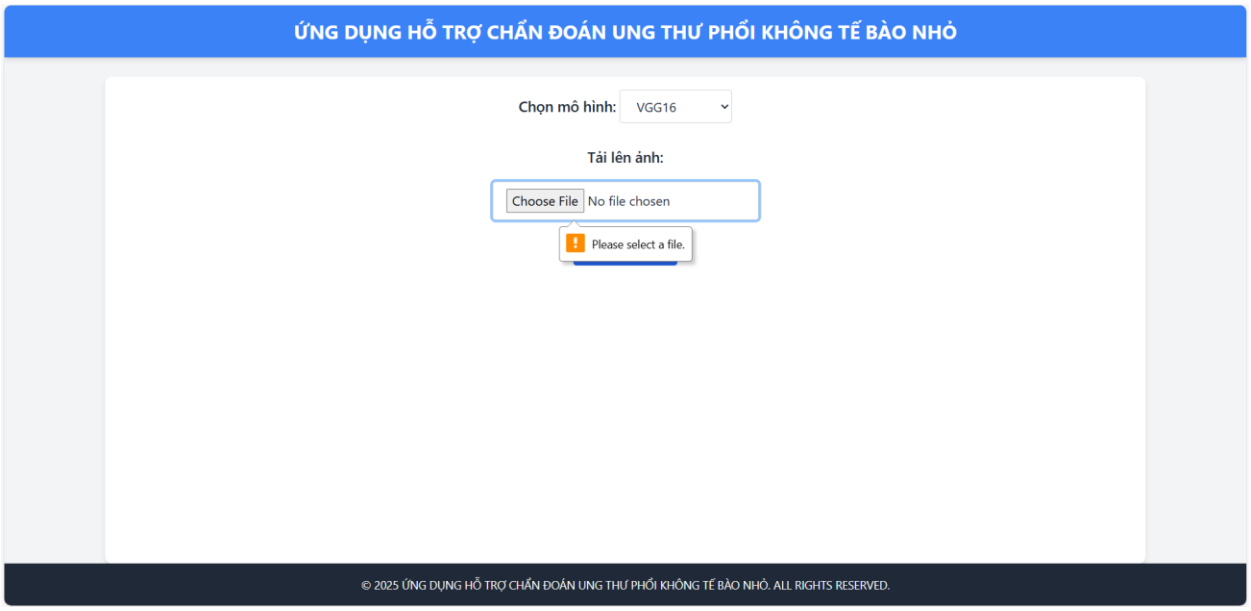
3.3.4. Kiểm thử nút "Chẩn đoán" khi chưa chọn ảnh

Bước thực hiện**:** Chọn một mô hình bất kỳ, không chọn file ảnh và nhấn nút "Chẩn đoán".

Kết quả mong đợi thể hiện như ở **Hình 2.19:**

- Hiển thị thông báo nhắc người dùng chọn ảnh trước khi tiến hành chẩn đoán.

- Không thực hiện tiến trình chẩn đoán.



Hình 2.19: Giao diện khi không chọn ảnh nhưng chẩn đoán

3.3.5. Kiểm thử nút "Chẩn đoán" với đầy đủ thông tin

Bước thực hiện**:**

- Chọn một mô hình ngẫu nhiên, ví dụ VGG16.

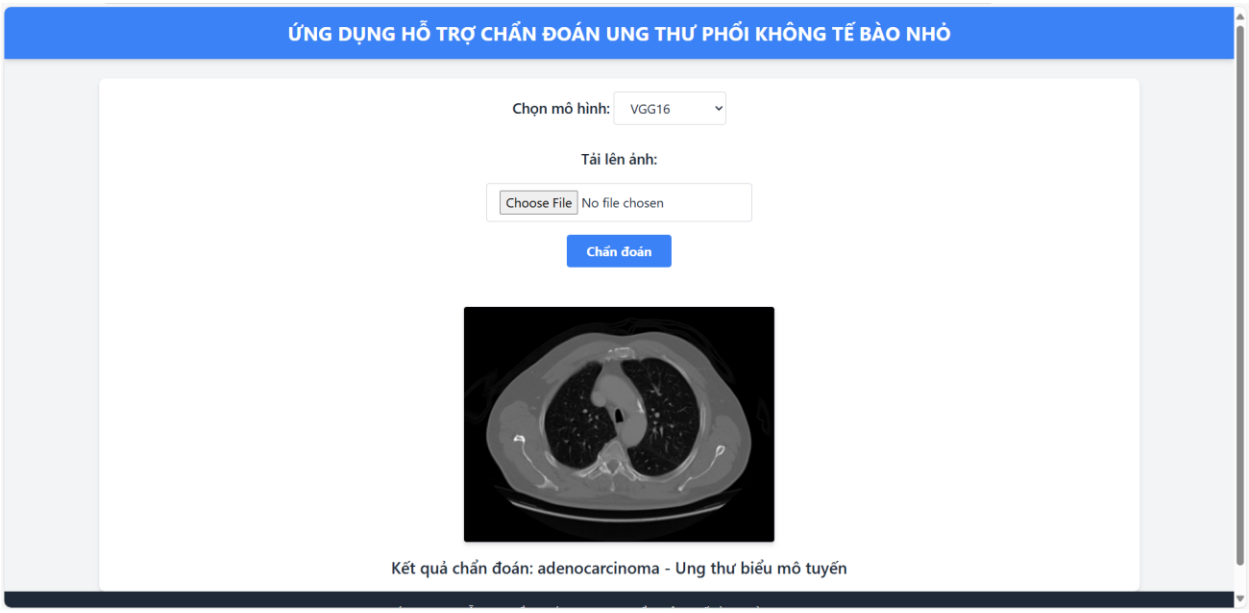
- Tải lên ảnh CT hợp lệ.

- Nhấn nút "Chẩn đoán".

Kết quả mong đợi thể hiện như ở **Hình 2.20:**

- Hệ thống chuyển sang xử lý (loading) rõ ràng cho người dùng nhận biết.

- Sau khi xử lý, trả về kết quả chẩn đoán hiển thị rõ ràng trên giao diện.



Hình 2.20: Giao diện sau khi chẩn đoán thành công

3.4. Kết quả kiểm thử

3.4.1. Kết quả kiểm thử các mô hình

Kết quả khi kiểm thử hai mô hình được triển khai trên giao diện web là InceptionV3 và VGG16 bằng cách sử dụng một tập dữ liệu đánh giá độc lập hoàn toàn với tập huấn luyện. Kết quả của quá trình kiểm thử của mô hình VGG16 được thể hiện ở **Bảng 2.3** và của mô hình InceptionV3 được thể hiện ở **Bảng 2.4**.

Bảng 2.3: Kết quả kiểm thử mô hình VGG16

Tên lớp	Số lượng ảnh dự đoán	Số lượng ảnh dự đoán đúng	Tổng
adenocarcinoma (Ung thư biểu mô tuyến)	20	18	75/80
large.cell.carcinoma (Ung thư biểu mô tế bào lớn)	20	20	
normal (Bình thường)	20	19	
squamous.cell.carcinoma (Ung thư biểu mô tế bào vảy)	20	18	

Bảng 2.4: Kết quả kiểm thử mô hình InceptionV3

Tên lớp	Số lượng ảnh dự đoán	Số lượng ảnh dự đoán đúng	Tổng
adenocarcinoma (Ung thư biểu mô tuyến)	20	10	31/80
large.cell.carcinoma (Ung thư biểu mô tế bào lớn)	20	12	
normal (Bình thường)	20	9	
squamous.cell.carcinoma (Ung thư biểu mô tế bào vảy)	20	0	

Từ **Bảng 2.3** và **Bảng 2.4**, có thể thấy được rằng mô hình VGG16 có tổng số lượng ảnh dự đoán đúng là 75/80, đây là một kết quả khá cao và có thể chấp nhận được. Còn mô hình InceptionV3 có tổng số lượng ảnh dự đoán đúng là 31/80, kết quả này thấp hơn đáng kể so với VGG16. Mặc dù khi đánh giá trên tập dữ liệu test thì mô hình có độ chính xác là 91.75%, tuy nhiên khi kiểm thử thực tế trên giao diện web với tập dữ liệu ngẫu nhiên thì mô hình lại đạt kết quả khá thấp. Điều này chứng tỏ rằng mô hình InceptionV3 hoạt động chưa tốt. Tóm lại, mô hình VGG16 cho hiệu suất vượt trội hơn so với mô hình InceptionV3. Điều này chứng tỏ VGG16 phù hợp hơn trong việc phân loại các loại mô ung thư. Mô hình InceptionV3 cần được cải thiện hoặc điều chỉnh các tham số để tăng hiệu quả phân loại.

Trong trường hợp ứng dụng thực tế, nên ưu tiên sử dụng VGG16 để đảm bảo độ chính xác cao hơn khi chẩn đoán.

3.4.2. Kết quả kiểm thử giao diện web

Bảng 2.5: Kết quả kiểm thử giao diện web

Kịch bản kiểm thử giao diện web	Đánh giá
Hiển thị giao diện ban đầu	Đạt
Chức năng chọn mô hình	Đạt
Tải lên file ảnh hợp lệ	Đạt
Nút "chẩn đoán" khi chưa chọn ảnh	Đạt
Nút "chẩn đoán" với đầy đủ thông tin	Đạt

Từ **Bảng 2.5**, có thể thấy được rằng cơ bản đề tài đã xây dựng được giao diện web có nhúng mô hình máy học để chẩn đoán ung thư phổi không tế bào nhỏ.

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

4.1.1. Kết quả đạt được

Tìm hiểu và xây dựng được các mô hình máy học có khả năng phân loại ung thư phổi không tế bài nhỏ. Đã triển khai và huấn luyện thành công các mô hình MobileNetV2, InceptionV3 và VGG16. Thực hiện đánh giá trên tập dữ liệu hình ảnh thì nhận thấy mô hình VGG16 tốt nhất với độ chính xác xấp xỉ 96%.

Xây dựng được giao diện web có nhúng mô hình máy học Người dùng có thể tải lên hình ảnh CT lên và nhấn chẩn đoán thì sẽ nhận được kết quả. Kết quả này chỉ mang tính chất hỗ trợ các y bác sĩ trong việc chẩn đoán nhanh nên cần phải được kiểm nghiệm lại.

4.1.2. Hạn chế của đề tài

Sau khi xây dựng ứng dụng để dự đoán và phân loại ung thư phổi, bên cạnh việc đạt được một số yêu cầu nhất định thì đề tài vẫn còn những hạn chế nhất định như sau:

- Dữ liệu chỉ đến từ một nguồn nhất định làm giảm khả năng tổng quát của mô hình. Việc dữ liệu bị giới hạn sẽ làm khả năng học của mô hình bị hạn chế khi gặp những hình ảnh có độ nhận diện khó.

- Khả năng nhận diện thấp đối với một số hình ảnh, đặc biệt là những hình ảnh sử dụng trí tuệ nhân tạo tạo ra hoặc những hình ảnh có độ tối quá mức. Điều này có thể sẽ dẫn đến quá trình dự đoán và phân loại bị sai sót.

- Ảnh hưởng của nhiễu nền: các yếu tố bên ngoài như những vật thể lạ được chụp kèm theo hình ảnh hoặc độ phân giải hình ảnh quá thấp sẽ dẫn đến quá trình dự đoán sai.

- Mặc dù mô hình đã được tối ưu hóa, nhưng trong một số trường hợp, thời gian dự đoán vẫn có thể chưa đáp ứng yêu cầu của các ứng dụng thực tế, đặc biệt là đối với những hệ thống yêu cầu phản hồi thời gian thực.

- Yêu cầu về máy tính: việc huấn luyện mô hình được thực hiện trên colab sử dụng GPU để đẩy nhanh quá trình huấn luyện mô hình, giúp nó học nhanh hơn.

- Đề tài chỉ tập trung vào việc nhận diện các hình ảnh liên quan về bệnh ung thư phổi không tế bào nhỏ (NSCLC) chiếm khoảng 80-85% tổng số ca bệnh. Các hình ảnh liên quan đến bệnh ung thư phổi tế bào nhỏ chiếm khoảng 10-15% vẫn chưa dự đoán được do mô hình chưa trải qua quá trình huấn luyện.

- Hiện tại đề tài chỉ có khả năng nhận dạng dựa trên ảnh CT, các ảnh chụp khác về bệnh ung thư phổi thì ứng dụng chưa nhận dạng được.

4.2.Hướng phát triển

Đề tài sẽ có những hướng để phát triển và khắc phục các hạn chế như sau:

- Mở rộng và đa dạng hóa nguồn dữ liệu:

+ Thu thập dữ liệu từ nhiều nguồn khác nhau như các bệnh viện, trung tâm nghiên cứu hoặc sử dụng các bộ dữ liệu công khai lớn hơn.

+ Sử dụng kỹ thuật data augmentation để tăng cường dữ liệu huấn luyện, giúp mô hình học được nhiều đặc điểm hơn.

+ Kết hợp dữ liệu từ nhiều thiết bị chụp khác nhau để giúp mô hình tổng quát tốt hơn.

- Cải thiện khả năng nhận diện hình ảnh phức tạp:

+ Tích hợp các mô hình học sâu tiên tiến hơn như Transformer-based models (ViT, Swin Transformer) để tăng khả năng nhận diện hình ảnh do AI tạo ra hoặc hình ảnh có độ tối cao.

+ Sử dụng preprocessing techniques như điều chỉnh độ sáng, tăng độ tương phản hoặc lọc nhiễu để cải thiện chất lượng ảnh đầu vào.

- Giảm ảnh hưởng của nhiễu nền và cải thiện độ chính xác dự đoán:

+ Áp dụng kỹ thuật segmentation (phân vùng ảnh) để chỉ tập trung vào khu vực chứa phổi và loại bỏ nhiễu nền.

+ Kết hợp attention mechanism để mô hình có thể tập trung vào những vùng quan trọng trong ảnh CT.

+ Nâng cấp độ phân giải hình ảnh bằng các phương pháp super-resolution để đảm bảo chất lượng ảnh khi dự đoán.

- Cải thiện thời gian dự đoán để đáp ứng yêu cầu thực tế:

+ Tối ưu hóa mô hình bằng cách sử dụng quantization hoặc pruning để giảm số lượng tham số mà không làm giảm hiệu suất.

+ Nếu triển khai trên hệ thống yêu cầu thời gian thực, có thể áp dụng các mô hình nhẹ hơn như EfficientNet,… thay vì VGG16, InceptionV3.

- Nâng cấp hạ tầng tính toán và triển khai mô hình:

+ Mở rộng khả năng huấn luyện trên các cụm GPU mạnh hơn thay vì chỉ sử dụng Google Colab.

+ Tích hợp mô hình vào các nền tảng cloud computing như Google Cloud AI, AWS SageMaker, Azure ML để hỗ trợ dự đoán trực tuyến.

- Mở rộng phạm vi dự đoán để nhận diện cả ung thư phổi tế bào nhỏ:

+ Thu thập thêm dữ liệu liên quan đến ung thư phổi tế bào nhỏ (SCLC) để mô hình có thể dự đoán được cả hai loại ung thư phổi.

+ Sử dụng multi-class classification models để phân biệt NSCLC và SCLC thay vì chỉ nhận diện một loại.

- Hỗ trợ nhiều loại hình ảnh y tế hơn, không chỉ ảnh CT:

+ Mở rộng mô hình để hỗ trợ các loại ảnh khác như X-ray, MRI, PET scan nhằm giúp bác sĩ có nhiều lựa chọn hơn trong việc chẩn đoán.

+ Áp dụng kỹ thuật multi-modal learning, kết hợp dữ liệu từ nhiều loại hình ảnh khác nhau để cải thiện độ chính xác.

TÀI LIỆU THAM KHẢO

[1] P. N. Tan, M. Steinbach, A. Karpatne, and V. Kumar, *Introduction to Data Mining*. in What's New in Computer Science Series. Pearson, 2019. [Online]. Available: https://books.google.com.vn/books?id=_ZQ4MQEACAAJ

[2] "PGS. TS Nguyễn Thái Nghe, Slide bài giảng Nghiệp vụ thông minh."

[3] "900-world-fact-sheet.pdf." Accessed: Feb. 20, 2025. [Online]. Available: https://gco.iarc.who.int/media/globocan/factsheets/populations/900-world-fact-sheet.pdf

[4] R. Nooreldeen and H. Bach, "Current and Future Development in Lung Cancer Diagnosis," *Int. J. Mol. Sci.*, vol. 22, no. 16, p. 8661, Aug. 2021, doi: 10.3390/ijms22168661.

[5] S. User, "Tổng quan Những tiến bộ trong chẩn đoán ung thư phổi: hiện tại và tương lai," Hội Hô Hấp TP.HCM. Accessed: Feb. 20, 2025. [Online]. Available: https://www.hoihohaptphcm.org/index.php/chuyende/benh-phoi/762-tong-quan-nhung-tien-bo-trong-chan-doan-ung-thu-phoi-hien-tai-va-tuong-lai

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," Mar. 21, 2019, *arXiv*: arXiv:1801.04381. doi: 10.48550/arXiv.1801.04381.

[7] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," Dec. 11, 2015, *arXiv*: arXiv:1512.00567. doi: 10.48550/arXiv.1512.00567.